

DSM Native Branch Full Change Dump

Generated from git diff with syntax coloring

Generated on 2026-04-30 11:38:37 CEST

Range

```
Branch: dsm_native
Base: 6bc051ece4 6bc051ece4bc49d398bbafd9944a922386b78a6f
Head: 88b9d147b0 88b9d147b082f0f67f08350ccbeac4da36d757ca
Range: 6bc051ece4..88b9d147b0
88b9d147b082f0f67f08350ccbeac4da36d757ca (origin/dsm_native, origin/dsm-nb-transition, dsm_native) RM DSM: refresh BEACON native references
34eb79dabc83b4ef64721e75977f6f387a7520e (vgk2/dsm-nb-transition, github/dsm_native, github/dsm-nb-transition) RM DSM: bound micro water content by porosity ceiling
8712c6d7ce0efcbac52148e3e7ca014cad7e247c DSM native decks: use Bishop chi=S_L (power law exponent 1)
6d6f795326257ca73d799834bd1277b0ef59feed6 (ogs-2/dsm-nb-transition) refactor(rm): remove DSM role-mapping transition scaffolding
f6c0e2f8eb4d0c4c548952f8c5fb5564c3calca7 RichardsMechanics DSM: remove notebook/Mathematica naming remnants
c061b6d21c75afe91ba3745d2e5da36b97a289cd RichardsMechanics: rename DSM notebook scope to dsm_micromacro
87dclae9bda7ab671320cb8c46982dc692c65410 Rename notebook reference identifiers to MathematicaReference naming
f7043ddd5a57a0ed424d25b825ed039d74738f4c Rename remaining VK helper symbols to neutral transition names
a57b0377820d9b277a457f572664218f58dbaa8 Rename VK type/test identifiers to conflict-free descriptive names
f770f11145e9ed47b29c3232d690c6c02b3b856a Rename RM local symbols to descriptive transition names
ea0e6cee4333d1d87ebd9ad9c3d85da2106bc402 Rename VK exchange shorthand and placeholder helper names
7da183661c9810a1d875d6f6444177d5b7b0154 Remove transition audit relic logging from native RM
1f9d0387a01c6fe94d229763949dbf3488e26fb4 Fix native DSM segfault without potential_exchange and refresh notebook-role references
8fbd9f8a33e0d54be5b30922f86dfe653ae5cc41 Rename VK benchmark/test data filenames to notebook naming
07c3484ba8454ad1176aa202a7cd5281e7233503 Rename VK file names in native DSM source/test/scripts
c9b6e3bf7171b474d9efcc7a7bd907d1484ea6d Rename VK output/config identifiers in native DSM bridge
bf625d5d1db55f94d3b340fef6979e90cc5e58 Apply swelling-stress mechanical strain path for VK exchange mode
0ab7ca0f22f907daf2a7f3921c4f09c2741dd6f RichardsMechanics: align notebook-role VK porosity and n_l bounds with bridge
c3de0b46cf68ecaff2c9c4cc3522a64224d9023 Align VK notebook mass-storage role mapping for dense D0 parity
d46e11ac00d09f4a746468b36b2088e57a849ff RM: add native notebook strain replay parity
65e3001fafc08c743e4147ac95efecd830f967 RM: add native notebook overlap replay state
762cc44b8a83b25454d5a85e9f8d218b501877c RM: add notebook-role history trend comparison
efae2aa381769df19efa1409bc902295e2d6d80 RM: add notebook-role history extraction
7249a0615079002a06220b900405bda8f872f22a meta: add project context guidance
8b12c35b29408a5d76bf30931c12fe75abdbb15 RM: align notebook mass-storage residual check
f34330364da8dadedb8f9e6a7af512e167d2be RM: trim notebook-role input gains and redundant test
555d8902e4989365d0501326f3d7efb76d3a52b RM: promote coupled notebook mass-storage solve and notebook-only swelling
c2e72b894543030acbf23b91f9cc95aab69f1efb RM: centralize notebook-style mass-storage writeback
c1bd5d261cd4472c6a61003979239174ec623347 RM: align notebook-role reference case with coupled storage
ef9edf5e05691001b329ac17b0c06312fdda729 RM: continue notebook-role alignment and 1c density fix
2a712d7552021e016a062f3bac488d1e9211291f RM: add notebook-style VK micro-mass storage mode
88bce38c7eab49999ebbc06365d0cc3f3d2fabf RM: add current-solid-fraction VK micro-potential mode
99f53706b502447e9d0c162c99cd030c3f47e18ba RM: add notebook-style VK macro-porosity update mode
453e4c41714addcc35904d8367f47aa40f90b37 RM: add notebook-style VK local storage solve mode
f0e3e67c378c66c68bae9d24b388f6468b2e8bd RM: add VK micro exchange monitoring and inflow examples
cc3d528e243d5da02584245af7eddi1a56b569a3 RM: add exploratory VK inflow probe cases
f3d4aa32b31280470b8c64ae843cd702a08b12 RM: add exploratory VK microstate outputs to la01 probe
af8d51f6e9891d12c1edd0db4921d73174dae34 RM: retune BEACON la01 stress probe to notebook vdW scale
00e63e90317c65182c8d45a0b9173e08aec4601c RM: make saturation_micro optional in VK mode
ad21d681dfef3d9718eab9c8a18e285d96f0d3 RM: decouple VK swelling from micro saturation
19caa7e437cb5153c9b1fd11c570e4ebef3350d RM: recompose VK total porosity from explicit split
b066340574106025a5b5c02abfa3c6bd34ca06ce RM: add explicit VK micro-porosity state split
ec2efa833c96b4059e4060a4c3bee30e1f1e345f RM: add notebook-style VK swelling from microscale water content
8039c23640b8c43a6e4143f004913837cf7d3a3 RM: add exploratory VK micro-water-content stress gain
4f8176bcb7222ecf98dead9027c9b0724bca9328 RM: add exploratory VK vdW-relaxation stress gain
492cb87aefc974a0b0c75924ad4e0fa2474f5feb RM: add BEACON la01 attractive stress probe input
1fafef1779f63a0e208b0dc10530d78272359d2fa RM: add BEACON stress-field benchmark outputs
f4d8d876bfb92fa65b6dd61b4bc889120ab6d6 RM: harden BEACON 1b as VK reference regression
4511ff6d511aa567c88937466d580affc7f5762 RM: harden BEACON 1c as VK heterogeneous regression
30c5c8ccde97b81230e804eb6d987d76c0415ed RM: harden BEACON la01 as VK reference regression
9df0f8cbe7f1950e0c8db3037acda896906ff5632 RM: add material-specific VK overrides for BEACON smoke cases
c37563de6d2b53b7958388a27319377f4501a384 RM: add BEACON run-only VK smoke cases
21c88bba0dbc0f4786d0a73c2c8b6880d6d678d2 RM: add opt-in attractive vdW convention prototype
324a7f6437966c9016bde0f46cd73599b46dbd9 RM: verify VK coupled tangent by finite difference
8d46a5800e58715f1d802186d7f3cf31a2c72eca RM: add VK branch-transition sensitivity tests
48c35c120e8852af74f666829dd2d87784d83fd5 RM: add VK constant-loading FE regression benchmark
a6524764932d74ff3f9efb26840c9143067718e5 RM: add VK single-point reference path test
94d4c17af5dd368536a04c8744af7c18a116dc0 RM: add opt-in VK local Jacobian diagnostics
21714d8a408f51fe58a083c96e49f31808bba51 RM: add exploratory VK wetting benchmark input
688e81615fe9c878d0de83fd36f6e43735a26f5f RM: make VK own transport porosity and compatibility swelling
b818529ae081d9a53a42096169fb1fc658cb0273 RM: make VK own microscale hydraulic state outputs
d9f1046597119cab3ac4c73699f22705532f2a1 RM: add VK opt-in reference regression test
b573329da62205ce92a371e6474509e41a3266a RM: assemble VK exchange source in non-Jacobian path
804d7b098947f494d5f64e75478d6304bd28e RM: add VK opt-in config guard and smoke ctest
1cd7cdf47406826c0211a764e6f8ab3da79294 RichardsMechanics: add implicit GP n_l solve and chain-rule Jacobian
cc49908bc2f5ead7e9d99807de8bbf74726855e03c RichardsMechanics: add analytical n_l-chain Jacobian for VK exchange
d146fec127ccaa20f83496c7471dd6a12904a RichardsMechanics: add opt-in FD Jacobian for VK exchange
f1565c399887de310c32d624f7c2e8732cb45 RichardsMechanics: implement Phase 2C-F3 vdW exchange and GP n_l state
c41aa7a23bb25745d05185713112b4bea3a422 feat(rm): phase-2c-f2 gate exchange path and use vk pressure tolerance
7adfec36d817ac6e471b9c83e881d9b1b32041 feat(rm): add 2c-f1 vk potential-exchange parser scaffolding
ec8e85d18cd884c91e7789aa7850db09f332 feat(rm): wire phase-2c exchange source via helper compatibility path
0905a8fda7206e98389c935b66002394ecc57b4 feat(rm): add VK potential helpers and phase-2b audit logging
5e6aa3bae40a6387266adde391869844fe63f60 feat(rmpl): add Tuller saturation retention property and parser
74675c21086e2e1f470f096250f285678ccbb049a feat(rm): add reversible phase-1 VK notation adapter audit
5f8e9d38458398f30711270875f5634f4ec94ef feat(rm): add phase-0 transition audit logging
c0fcd4dc8baced54d8ab7ce02953672f153ab8 build(cmake): add local presets for mfront and ci workflows
```

Changed Files

```

CMakePresets.json | 41 +-
MaterialLib/MPL/CreateProperty.cpp | 5 +
.../CreateSaturationTuller.cpp | 88 +
.../CreateSaturationTuller.h | 23 +
.../SaturationTuller.cpp | 141 ++
.../CapillaryPressureSaturation/SaturationTuller.h | 76 +
MaterialLib/MPL/Properties/CreateProperties.h | 1 +
MaterialLib/MPL/Properties/Properties.h | 1 +
PROJECT_CONTEXT.md | 32 +
.../ConstitutiveRelations/ConstitutiveData.h | 6 +-
.../ConstitutiveRelations/MicroLiquidDensity.h | 19 +
.../ConstitutiveRelations/MicroPorosity.h | 18 +
.../ConstitutiveRelations/MicroWaterContent.h | 27 +
.../ConstitutiveRelations/PotentialExchange.h | 164 ++
.../CreateRichardsMechanicsProcess.cpp | 512 +++++
.../RichardsMechanics/LocalAssemblerInterface.h | 38 +
.../PotentialExchangeParameters.h | 158 ++
.../RichardsMechanics/RichardsMechanicsFEM-impl.h | 2343 ++++++
.../RichardsMechanics/RichardsMechanicsFEM.h | 14 +-
.../RichardsMechanicsProcessData.h | 4 +
ProcessLib/RichardsMechanics/Tests.cmake | 150 ++
.../DSM/MicroMacroOverlapTransferBaseline.csv | 6 +
.../DSM/MicroMacroStrainCoupledOverlapBaseline.csv | 6 +
Tests/Data/RichardsMechanics/beacon_1a01.gml | 17 +
.../Data/RichardsMechanics/beacon_1a01_domain.vtu | 24 +
.../beacon_1a01_dsm_micromacro_inflow.prj | 174 ++
.../micromacro_inflow_reference_t_100000.000000.vtu | 35 +
.../beacon_1a01_dsm_micromacro_inflowprobe.prj | 179 ++
.../beacon_1a01_dsm_micromacro_smoke.prj | 159 ++
.../beacon_1a01_dsm_micromacro_stressprobe.prj | 174 ++
.../beacon_1a01_reference_t_1000.000000.vtu | 32 +
Tests/Data/RichardsMechanics/beacon_1b.gml | 17 +
Tests/Data/RichardsMechanics/beacon_1b_domain.vtu | 24 +
.../beacon_1b_dsm_micromacro_smoke.prj | 84 +
.../beacon_1b_reference_t_1000.000000.vtu | 32 +
Tests/Data/RichardsMechanics/beacon_1c.gml | 19 +
Tests/Data/RichardsMechanics/beacon_1c_domain.vtu | 26 +
.../beacon_1c_dsm_micromacro_smoke.prj | 125 ++
.../beacon_1c_reference_t_1000.000000.vtu | 34 +
.../le_porosity_swelling_dsm_micromacro_constbc.xml | 48 +
.../le_porosity_swelling_dsm_micromacro_wetting.xml | 59 +
.../micromacro_constbc_reference_t_1000.000000.vtu | 37 +
Tests/MaterialLib/TestMPLSaturationTuller.cpp | 102 +
.../DSM/MicroMacroSingleIntegrationPoint.cpp | 1637 ++++++
.../RichardsMechanics/PotentialExchange.cpp | 183 ++
scripts/cmake/test/HistoryExtraction.cmake | 51 +
scripts/extract_observables.py | 168 ++
scripts/history_trend_check.py | 120 +
48 files changed, 7311 insertions(+), 122 deletions(-)

```

Full Diff

```

diff --git a/CMakePresets.json b/CMakePresets.json
index 6c5b9220cb..de36b495b4 100644
--- a/CMakePresets.json
+++ b/CMakePresets.json
@@ -57,6 +57,7 @@
     "name": "_petsc",
     "hidden": true,
     "cacheVariables": {
+       "BUILD_SHARED_LIBS": "ON",
       "OGS_USE_PETSC": "ON"
     },
     "environment": {
@@ -173,6 +174,13 @@
       "OGS_TEST_LABEL_EXCLUDE": "-LE omp"
     }
   },
+  {
+    "name": "ci-simplest",
+    "inherits": "release",
+    "cacheVariables": {
+      "OGS_BUILD_UTILS": "OFF"
+    }
+  },
+  {
+    "name": "ci-no-unity",
+    "inherits": [
@@ -202,15 +210,18 @@
       "HDF5_USE_STATIC_LIBRARIES": "ON",
       "OGS_BUILD_HDF5": "ON",
       "OGS_BUILD_WHEEL": "ON",
+      "OGS_BUILD_TESTING": "OFF",
+      "OGS_INSTALL_DEPENDENCIES": "OFF",
       "OGS_USE_PIP": "OFF",
-      "OGS_USE_MFRONT": "ON"
+      "OGS_USE_MFRONT": "ON",
+      "BUILD_SHARED_LIBS": "ON"
     }
   },
+  {
+    "name": "sanitizer",
+    "inherits": "debug-all",
+    "cacheVariables": {
+      "BUILD_SHARED_LIBS": "ON",
+      "OGS_CXX_FLAGS": "-O3;-fno-var-tracking",
+      "OGS_EIGEN_DYNAMIC_SHAPE_MATRICES": "OFF",
+      "ENABLE_ASAN": "ON",
@@ -226,9 +237,30 @@
       "UBSAN_OPTIONS": "print_stacktrace=1",
       "LSAN_OPTIONS": "suppressions=$(sourceDir)/scripts/test/leak_sanitizer.suppressions"
     }
   },
+  {
+    "name": "release-omp-mfront",
+    "inherits": [
+      "release"
+    ],
+    "cacheVariables": {
+      "OGS_USE_MFRONT": "ON",

```



```

+         "{:g}.",
+         characteristic_pore_size);
+     }
+     if (!(surface_tension > 0.0))
+     {
+         OGS_FATAL("SaturationTuller requires surface_tension > 0, got {:g}.",
+             surface_tension);
+     }
+     if (!(pressure_tolerance_ >= 0.0))
+     {
+         OGS_FATAL(
+             "SaturationTuller requires pressure_tolerance >= 0, got {:g}.",
+             pressure_tolerance_);
+     }
+     if (!(coefficient_ > 0.0))
+     {
+         OGS_FATAL("SaturationTuller internal coefficient must be > 0, got {:g}.",
+             coefficient_);
+     }
+ }
+ )
+
+ *PropertyDataType SaturationTuller::value(
+     VariableArray const& variable_array,
+     ParameterLib::SpatialPosition const& /%pos*/, double const /*t*/,
+     double const /*dt*/) const
+ {
+     double const p_cap = variable_array.capillary_pressure;
+     if (p_cap <= pressure_tolerance_)
+     {
+         return S_L_max_;
+     }
+
+     double const e = std::exp(-coefficient_ / (p_cap * p_cap));
+     double const S_eff = 1.0 - e;
+     double const S = S_L_res_ + (S_L_max_ - S_L_res_) * S_eff;
+
+     return std::clamp(S, S_L_res_, S_L_max_);
+ }
+
+ *PropertyDataType SaturationTuller::dValue(
+     VariableArray const& variable_array, Variable const variable,
+     ParameterLib::SpatialPosition const& /%pos*/, double const /*t*/,
+     double const /*dt*/) const
+ {
+     if (variable != Variable::capillary_pressure)
+     {
+         OGS_FATAL(
+             "SaturationTuller::dValue is implemented for derivatives with "
+             "respect to capillary pressure only.");
+     }
+
+     double const p_cap = variable_array.capillary_pressure;
+     if (p_cap <= pressure_tolerance_)
+     {
+         return 0.0;
+     }
+
+     double const e = std::exp(-coefficient_ / (p_cap * p_cap));
+     double const dS_eff_dp_cap = -(2.0 * coefficient_ / (p_cap * p_cap)) * e;
+     return dS_eff_dp_cap * (S_L_max_ - S_L_res_);
+ }
+
+ *PropertyDataType SaturationTuller::d2Value(
+     VariableArray const& variable_array, Variable const variable1,
+     Variable const variable2, ParameterLib::SpatialPosition const& /%pos*/,
+     double const /*t*/, double const /*dt*/) const
+ {
+     (void)variable1;
+     (void)variable2;
+     assert((variable1 == Variable::capillary_pressure) &&
+         (variable2 == Variable::capillary_pressure) &&
+         "SaturationTuller::d2Value is implemented for derivatives with "
+         "respect to capillary pressure only.");
+
+     double const p_cap = variable_array.capillary_pressure;
+     if (p_cap <= pressure_tolerance_)
+     {
+         return 0.0;
+     }
+
+     double const p2 = p_cap * p_cap;
+     double const p4 = p2 * p2;
+     double const p6 = p4 * p2;
+     double const e = std::exp(-coefficient_ / p2);
+     double const d2S_eff_dp_cap2 =
+         2.0 * coefficient_ * e * (3.0 * p2 - 2.0 * coefficient_) / p6;
+     return d2S_eff_dp_cap2 * (S_L_max_ - S_L_res_);
+ }
+ // namespace MaterialPropertyLib
+
+ diff --git a/MaterialLib/MPL/Properties/CapillaryPressureSaturation/SaturationTuller.h b/MaterialLib/MPL/Properties/CapillaryPressureSaturation/SaturationTuller.h
+ new file mode 100644
+ index 0000000000..989788350a
+ --- /dev/null
+ +++ b/MaterialLib/MPL/Properties/CapillaryPressureSaturation/SaturationTuller.h
+ @@ -0,0 +1,76 @@
+ // SPDX-FileCopyrightText: Copyright (c) OpenGeoSys Community (opengeosys.org)
+ // SPDX-License-Identifier: BSD-3-Clause
+
+ #pragma once
+
+ #include "MaterialLib/MPL/Property.h"
+
+ namespace MaterialPropertyLib
+ {
+ class Medium;
+ class Phase;
+ class Component;
+
+ /**
+  * \brief Tuller/Young-Laplace-inspired saturation model used in the DSM
+  * \* micro-macro reference branch
+  * \* macro retention law.
+  *
+  * \* The implemented saturation relation is (for capillary pressure p_c):
+  * \f[
+  * S_L(p_c)=

```



```

+- For long tasks, keep a short `WORKLOG.md` with the current state and next step.
diff --git a/ProcessLib/RichardsMechanics/ConstitutiveRelations/ConstitutiveData.h b/ProcessLib/RichardsMechanics/ConstitutiveRelations/ConstitutiveData.h
index 66ce0f427e..9d23d98a50 100644
--- a/ProcessLib/RichardsMechanics/ConstitutiveRelations/ConstitutiveData.h
+++ b/ProcessLib/RichardsMechanics/ConstitutiveRelations/ConstitutiveData.h
@@ -8,6 +8,9 @@
#include "LiquidDensity.h"
#include "MicroPressure.h"
#include "MicroSaturation.h"
#include "MicroLiquidDensity.h"
#include "MicroPorosity.h"
#include "MicroWaterContent.h"
#include "ProcessLib/ConstitutiveRelations/Base.h"
#include "ProcessLib/ConstitutiveRelations/EffectiveStressData.h"
#include "ProcessLib/ConstitutiveRelations/MechanicalStrainData.h"
@@ -37,7 +40,8 @@ using StatefulData = std::tuple<
    ProcessLib::ThermoRichardsMechanics::SaturationData,
    ProcessLib::ThermoRichardsMechanics::PorosityData,
    ProcessLib::ThermoRichardsMechanics::TransportPorosityData, MicroPressure,
-   MicroSaturation>;
+   MicroSaturation, MicroWaterContent, MicroLiquidDensity, MicroPorosity,
+   MicroExchangeSource>;

template <int DisplacementDim>
using StatefulDataPrev = ProcessLib::ConstitutiveRelations::PrevStateOf<
diff --git a/ProcessLib/RichardsMechanics/ConstitutiveRelations/MicroLiquidDensity.h b/ProcessLib/RichardsMechanics/ConstitutiveRelations/MicroLiquidDensity.h
new file mode 100644
index 0000000000..9dcceff1e6
--- /dev/null
+++ b/ProcessLib/RichardsMechanics/ConstitutiveRelations/MicroLiquidDensity.h
@@ -0,0 +1,19 @@
// SPDX-FileCopyrightText: Copyright (c) OpenGeoSys Community (opengeosys.org)
// SPDX-License-Identifier: BSD-3-Clause
+
+#pragma once
+
+#include <string_view>
+
+#include "BaseLib/StrongType.h"
+
+namespace ProcessLib::RichardsMechanics
+{
+using MicroLiquidDensity =
+    BaseLib::StrongType<double, struct MicroLiquidDensityTag>;
+
+constexpr std::string_view ioName(struct MicroLiquidDensityTag*)
+{
+    return "micro_liquid_density";
+}
+} // namespace ProcessLib::RichardsMechanics
diff --git a/ProcessLib/RichardsMechanics/ConstitutiveRelations/MicroPorosity.h b/ProcessLib/RichardsMechanics/ConstitutiveRelations/MicroPorosity.h
new file mode 100644
index 0000000000..f42471103b
--- /dev/null
+++ b/ProcessLib/RichardsMechanics/ConstitutiveRelations/MicroPorosity.h
@@ -0,0 +1,18 @@
// SPDX-FileCopyrightText: Copyright (c) OpenGeoSys Community (opengeosys.org)
// SPDX-License-Identifier: BSD-3-Clause
+
+#pragma once
+
+#include <string_view>
+
+#include "BaseLib/StrongType.h"
+
+namespace ProcessLib::RichardsMechanics
+{
+using MicroPorosity = BaseLib::StrongType<double, struct MicroPorosityTag>;
+
+constexpr std::string_view ioName(struct MicroPorosityTag*)
+{
+    return "micro_porosity";
+}
+} // namespace ProcessLib::RichardsMechanics
diff --git a/ProcessLib/RichardsMechanics/ConstitutiveRelations/MicroWaterContent.h b/ProcessLib/RichardsMechanics/ConstitutiveRelations/MicroWaterContent.h
new file mode 100644
index 0000000000..9d51b3f32e
--- /dev/null
+++ b/ProcessLib/RichardsMechanics/ConstitutiveRelations/MicroWaterContent.h
@@ -0,0 +1,27 @@
// SPDX-FileCopyrightText: Copyright (c) OpenGeoSys Community (opengeosys.org)
// SPDX-License-Identifier: BSD-3-Clause
+
+#pragma once
+
+#include <string_view>
+
+#include "BaseLib/StrongType.h"
+
+namespace ProcessLib::RichardsMechanics
+{
+using MicroWaterContent =
+    BaseLib::StrongType<double, struct MicroWaterContentTag>;
+
+constexpr std::string_view ioName(struct MicroWaterContentTag*)
+{
+    return "micro_water_content";
+}
+}
+
+using MicroExchangeSource =
+    BaseLib::StrongType<double, struct MicroExchangeSourceTag>;
+
+constexpr std::string_view ioName(struct MicroExchangeSourceTag*)
+{
+    return "micro_exchange_source";
+}
+} // namespace ProcessLib::RichardsMechanics
diff --git a/ProcessLib/RichardsMechanics/ConstitutiveRelations/PotentialExchange.h b/ProcessLib/RichardsMechanics/ConstitutiveRelations/PotentialExchange.h
new file mode 100644
index 0000000000..667849fb6b
--- /dev/null
+++ b/ProcessLib/RichardsMechanics/ConstitutiveRelations/PotentialExchange.h
@@ -0,0 +1,164 @@
// SPDX-FileCopyrightText: Copyright (c) OpenGeoSys Community (opengeosys.org)
// SPDX-License-Identifier: BSD-3-Clause
+
+#pragma once

```

```

+
+#include <cmath>
+
+#include "BaseLib/Error.h"
+
+namespace ProcessLib::RichardsMechanics
+{
+struct YoungLaplaceMacroPotentialData
+{
+    + double mu_LR = 0.0;
+    + double dmu_LR_dplR = 0.0;
+    + double dmu_LR_drho_LR = 0.0;
+    + bool saturated_branch = true;
+};
+
+// DSM dsm_micromacro Phase-2 macro potential helper (Young-Laplace side):
+// mu_LR = 0 for pLR > -ptol
+// mu_LR = pLR / rho_LR otherwise
+inline YoungLaplaceMacroPotentialData computeYoungLaplaceMacroPotential(
+    + double const p_LR, double const rho_LR, double const pressure_tolerance = 0.0)
+{
+    + if (!(rho_LR > 0.0))
+    + {
+        + OGS_FATAL(
+        +     "computeYoungLaplaceMacroPotential requires rho_LR > 0, got {g}.",
+        +     rho_LR);
+    + }
+    + if (!(pressure_tolerance >= 0.0))
+    + {
+        + OGS_FATAL(
+        +     "computeYoungLaplaceMacroPotential requires pressure_tolerance >= "
+        +     "0, got {g}.",
+        +     pressure_tolerance);
+    + }
+
+    + YoungLaplaceMacroPotentialData out;
+    + out.saturated_branch = (p_LR > -pressure_tolerance);
+    + if (out.saturated_branch)
+    + {
+        + return out;
+    + }
+
+    + out.mu_LR = p_LR / rho_LR;
+    + out.dmu_LR_dplR = 1.0 / rho_LR;
+    + out.dmu_LR_drho_LR = -p_LR / (rho_LR * rho_LR);
+    + return out;
+}
+
+struct VanDerWaalsMicroPotentialData
+{
+    + double omega_l = 0.0;
+    + double mu_LR = 0.0;
+
+    + double domega_l_dnl = 0.0;
+    + double domega_l_drho_LR = 0.0;
+    + double domega_l_dns = 0.0;
+    + double domega_l_drho_SR = 0.0;
+
+    + double dmu_LR_dnl = 0.0;
+    + double dmu_LR_drho_LR = 0.0; // exactly zero in the reduced algebraic form.
+    + double dmu_LR_dns = 0.0;
+    + double dmu_LR_drho_SR = 0.0;
+};
+
+// DSM dsm_micromacro microscale vdW potential helper:
+// omega_l = n_l * rho_LR / (nS * rho_SR)
+// mu_LR = (A * rho_LR^3 / (6*pi)) * (5a / omega_l)^3
+//         = (A * 5a^3 / (6*pi)) * (nS^3 * rho_SR^3) / n_l^3
+inline VanDerWaalsMicroPotentialData computeVanDerWaalsMicroPotential(
+    + double const n_l, double const rho_LR, double const nS, double const rho_SR,
+    + double const hamaker_constant, double const specific_surface,
+    + double const potential_sign_factor = 1.0)
+{
+    + if (!(n_l > 0.0))
+    + {
+        + OGS_FATAL("computeVanDerWaalsMicroPotential requires n_l > 0, got {g}.",
+        +     n_l);
+    + }
+    + if (!(rho_LR > 0.0))
+    + {
+        + OGS_FATAL(
+        +     "computeVanDerWaalsMicroPotential requires rho_LR > 0, got {g}.",
+        +     rho_LR);
+    + }
+    + if (!(nS > 0.0))
+    + {
+        + OGS_FATAL("computeVanDerWaalsMicroPotential requires nS > 0, got {g}.",
+        +     nS);
+    + }
+    + if (!(rho_SR > 0.0))
+    + {
+        + OGS_FATAL(
+        +     "computeVanDerWaalsMicroPotential requires rho_SR > 0, got {g}.",
+        +     rho_SR);
+    + }
+    + if (!(hamaker_constant > 0.0))
+    + {
+        + OGS_FATAL(
+        +     "computeVanDerWaalsMicroPotential requires hamaker_constant > 0, "
+        +     "got {g}.",
+        +     hamaker_constant);
+    + }
+    + if (!(specific_surface > 0.0))
+    + {
+        + OGS_FATAL(
+        +     "computeVanDerWaalsMicroPotential requires specific_surface > 0, "
+        +     "got {g}.",
+        +     specific_surface);
+    + }
+
+    + constexpr double pi = 3.141592653589793238462643383279502884;
+
+    + VanDerWaalsMicroPotentialData out;
+
+    + out.omega_l = n_l * rho_LR / (nS * rho_SR);
+
+    + out.domega_l_dnl = rho_LR / (nS * rho_SR);

```



```

+ out.domega_l_drho_LR = n_l / (nS * rho_SR);
+ out.domega_l_dnS = -out.omega_l / nS;
+ out.domega_l_drho_SR = -out.omega_l / rho_SR;
+
+ double const prefactor = hamaker_constant * specific_surface *
+   specific_surface /
+   (6.0 * pi);
+ out.mu_LR = potential_sign_factor * prefactor * (nS * nS * nS) *
+   (rho_SR * rho_SR * rho_SR) / (n_l * n_l * n_l);
+
+ out.dmu_LR_dnl = -3.0 * out.mu_LR / n_l;
+ out.dmu_LR_drho_LR = 0.0;
+ out.dmu_LR_dnS = 3.0 * out.mu_LR / nS;
+ out.dmu_LR_drho_SR = 3.0 * out.mu_LR / rho_SR;
+
+ return out;
+}
+
+struct PotentialDrivenMassExchangeData
+{
+ double rho_l_hat = 0.0;
+ double rho_l_hat = 0.0;
+
+ double drho_l_hat_dmu_LR = 0.0;
+ double drho_l_hat_dmu_LR = 0.0;
+ double drho_l_hat_dalpha_M = 0.0;
+};
+
+// DSM_dsm_micromacro sign convention:
+// rho_l_hat = alpha_M * (mu_LR - mu_LR)
+// rho_l_hat = -rho_l_hat
+inline PotentialDrivenMassExchangeData computePotentialDrivenMassExchange(
+ double const alpha_M, double const mu_LR, double const mu_LR)
+{
+ PotentialDrivenMassExchangeData out;
+ out.rho_l_hat = alpha_M * (mu_LR - mu_LR);
+ out.rho_l_hat = -out.rho_l_hat;
+ out.drho_l_hat_dmu_LR = alpha_M;
+ out.drho_l_hat_dmu_LR = -alpha_M;
+ out.drho_l_hat_dalpha_M = mu_LR - mu_LR;
+ return out;
+}
+} // namespace ProcessLib::RichardsMechanics
diff --git a/ProcessLib/RichardsMechanics/CreateRichardsMechanicsProcess.cpp b/ProcessLib/RichardsMechanics/CreateRichardsMechanicsProcess.cpp
index c8ac7a39a5..c179ba1e4d 100644
--- a/ProcessLib/RichardsMechanics/CreateRichardsMechanicsProcess.cpp
+++ b/ProcessLib/RichardsMechanics/CreateRichardsMechanicsProcess.cpp
@@ -3,6 +3,7 @@

#include "CreateRichardsMechanicsProcess.h"

#include <algorithm>
#include <cassert>

#include "MaterialLib/MPL/CreateMaterialSpatialDistributionMap.h"
@@ -22,6 +23,114 @@ namespace ProcessLib
{
namespace RichardsMechanics
{
namespace
+{
+char const* toString(PotentialExchangeMode const mode)
+{
+ switch (mode)
+ {
+ case PotentialExchangeMode::FullPotential:
+ return "full_potential";
+ }
+ return "unknown";
+}
+
+MicroPotentialConvention parseMicroPotentialConvention(
+ std::string const& convention)
+{
+ if (convention == "positive_reduced")
+ {
+ return MicroPotentialConvention::PositiveReduced;
+ }
+ if (convention == "negative_attractive")
+ {
+ return MicroPotentialConvention::NegativeAttractive;
+ }
+
+ OGS_FATAL(
+ "RichardsMechanics: unsupported potential_exchange "
+ "micro_potential_convention '{}'. Currently supported: "
+ "'positive_reduced', 'negative_attractive'.",
+ convention);
+}
+
+LocalNonlinearSolveMode parseLocalNonlinearSolveMode(
+ std::string const& mode)
+{
+ if (mode == "scalar_exchange")
+ {
+ return LocalNonlinearSolveMode::ScalarExchange;
+ }
+ if (mode == "scalar_microstate_storage_mode")
+ {
+ return LocalNonlinearSolveMode::ScalarReferenceStorage;
+ }
+ if (mode == "scalar_micro_macro_mass_storage_mode")
+ {
+ return LocalNonlinearSolveMode::ScalarReferenceMassStorage;
+ }
+
+ OGS_FATAL(
+ "RichardsMechanics: unsupported potential_exchange "
+ "local_nonlinear_solve_mode '{}'. Currently supported: "
+ "'scalar_exchange', 'scalar_microstate_storage_mode', "
+ "'scalar_micro_macro_mass_storage_mode'.",
+ mode);
+}
+
+MacroPorosityUpdateMode parseMacroPorosityUpdateMode(
+ std::string const& mode)
+{

```

```

+   if (mode == "algebraic_split")
+   {
+       return MacroPorosityUpdateMode::AlgebraicSplit;
+   }
+   if (mode == "additive_macro_porosity_rate_mode")
+   {
+       return MacroPorosityUpdateMode::ReferenceAdditiveRate;
+   }
+
+   OGS_FATAL(
+       "RichardsMechanics: unsupported potential_exchange "
+       "macro_porosity_update_mode '{}'. Currently supported: "
+       "'algebraic_split', 'additive_macro_porosity_rate_mode'.",
+       mode);
+}
+
+*MicroSolidVolumeFractionMode parseMicroSolidVolumeFractionMode(
+  std::string const& mode)
+{
+  if (mode == "reference")
+  {
+      return MicroSolidVolumeFractionMode::Reference;
+  }
+  if (mode == "current_porosity_split")
+  {
+      return MicroSolidVolumeFractionMode::CurrentPorositySplit;
+  }
+
+  OGS_FATAL(
+      "RichardsMechanics: unsupported potential_exchange "
+      "micro_solid_volume_fraction_mode '{}'. Currently supported: "
+      "'reference', 'current_porosity_split'.",
+      mode);
+}
+
+*PotentialExchangeMode parsePotentialExchangeMode(std::string const& mode)
+{
+  if (mode == "full_potential")
+  {
+      return PotentialExchangeMode::FullPotential;
+  }
+
+  OGS_FATAL(
+      "RichardsMechanics: unsupported potential_exchange mode '{}'. "
+      "Currently supported: 'full_potential'.",
+      mode);
+}
+
+// namespace
+
+void checkMPLProperties(
+  std::map<int, std::shared_ptr<MaterialPropertyLib::Medium>> const& media)
+{
+  @ -46,6 +155,371 @@ void checkMPLProperties(
+  }
+}
+
+void validateMicroPorosityAndPotentialExchangeConfiguration(
+  std::map<int, std::shared_ptr<MaterialPropertyLib::Medium>> const& media,
+  std::optional<MicroPorosityParameters> const& micro_porosity_parameters,
+  std::optional<PotentialExchangeParameters> const&
+    potential_exchange_parameters,
+  std::map<int, PotentialExchangeParameters> const&
+    potential_exchange_parameters_by_material)
+{
+  namespace MPL = MaterialPropertyLib;
+
+  bool const micro_porosity_enabled = micro_porosity_parameters.has_value();
+  bool any_saturation_micro = false;
+  bool const any_dsm_exchange_enabled =
+    (potential_exchange_parameters &&
+     potential_exchange_parameters->enabled) ||
+    std::any_of(potential_exchange_parameters_by_material.begin(),
+                potential_exchange_parameters_by_material.end(),
+                [](auto const& item) { return item.second.enabled; });
+
+  for (auto const& [material_id, medium] : media)
+  {
+      bool const has_saturation_micro =
+        medium->hasProperty(MPL::PropertyType::saturation_micro);
+      any_saturation_micro = any_saturation_micro || has_saturation_micro;
+
+      if (has_saturation_micro && !micro_porosity_enabled)
+      {
+          OGS_FATAL(
+              "RichardsMechanics: medium {} defines 'saturation_micro' but "
+              "the process has no <micro_porosity> block. Define "
+              "<micro_porosity> or remove 'saturation_micro'.",
+              material_id);
+      }
+  }
+
+  if (micro_porosity_enabled && !any_saturation_micro && !any_dsm_exchange_enabled)
+  {
+      OGS_FATAL(
+          "RichardsMechanics: <micro_porosity> is configured, but no medium "
+          "defines 'saturation_micro'. Define 'saturation_micro' in at least "
+          "one medium or remove <micro_porosity>.");
+  }
+
+  if (any_dsm_exchange_enabled && !micro_porosity_enabled)
+  {
+      OGS_FATAL(
+          "RichardsMechanics: potential_exchange.enabled=true requires "
+          "a <micro_porosity> process block.");
+  }
+
+  for (auto const& [material_id, potential_exchange_params] :
+    potential_exchange_parameters_by_material)
+  {
+      if (media.find(material_id) == media.end())
+      {
+          OGS_FATAL(
+              "RichardsMechanics: potential_exchange medium override "
+              "references unknown material id {}.",
+              material_id);
+      }
+  }
+}

```

```

+
+ }
+
+}
+
+PotentialExchangeParameters parsePotentialExchangeParameters(
+  BaseLib::ConfigTree const& config,
+  std::optional<PotentialExchangeParameters> const& defaults,
+  std::string const& context)
+{
+  auto const enabled =
+    config.getConfigParameter<bool>("enabled",
+                                     defaults ? defaults->enabled : false);
+
+  auto const mode = parsePotentialExchangeMode(
+    config.getConfigParameter<std::string>(
+      "mode", defaults ? toString(defaults->mode) : "full_potential"));
+
+  auto const pressure_tolerance = config.getConfigParameter<double>{
+    "pressure_tolerance",
+    defaults ? defaults->pressure_tolerance : 0.0;
+  };
+  if (pressure_tolerance < 0.0)
+  {
+    OGS_FATAL(
+      "RichardsMechanics: {} pressure_tolerance must be >= 0, got {g}.",
+      context, pressure_tolerance);
+  }
+
+  auto const micro_potential_convention = parseMicroPotentialConvention(
+    config.getConfigParameter<std::string>(
+      "micro_potential_convention",
+      defaults ? toString(defaults->micro_potential_convention)
+        : "positive_reduced"));
+
+  auto const local_nonlinear_solve_mode = parseLocalNonlinearSolveMode(
+    config.getConfigParameter<std::string>(
+      "local_nonlinear_solve_mode",
+      defaults ? toString(defaults->local_nonlinear_solve_mode)
+        : "scalar_exchange"));
+
+  auto const macro_porosity_update_mode = parseMacroPorosityUpdateMode(
+    config.getConfigParameter<std::string>(
+      "macro_porosity_update_mode",
+      defaults ? toString(defaults->macro_porosity_update_mode)
+        : "algebraic_split"));
+
+  auto const micro_solid_volume_fraction_mode =
+    parseMicroSolidVolumeFractionMode(
+      config.getConfigParameter<std::string>(
+        "micro_solid_volume_fraction_mode",
+        defaults
+          ? toString(defaults->micro_solid_volume_fraction_mode)
+          : "reference"));
+
+  auto get_positive_required_or_default =
+    [&](char const* const key, double const fallback)
+    {
+      auto const value = config.getConfigParameterOptional<double>(key);
+      double const selected = value ? *value : fallback;
+      if (!selected > 0.0)
+      {
+        OGS_FATAL(
+          "RichardsMechanics: {} {} must be > 0, got {g}.", context,
+          key, selected);
+      }
+      return selected;
+    };
+
+  auto get_positive_optional_or_default =
+    [&](char const* const key, std::optional<double> const fallback)
+    -> std::optional<double>
+    {
+      auto const value = config.getConfigParameterOptional<double>(key);
+      std::optional<double> selected = value ? std::optional<double>{*value}
+        : fallback;
+      if (selected && !(*selected > 0.0))
+      {
+        OGS_FATAL(
+          "RichardsMechanics: {} {} must be > 0 if provided, got {g}.",
+          context, key, *selected);
+      }
+      return selected;
+    };
+
+  double const default_hamaker =
+    defaults ? defaults->hamaker_constant : 0.0;
+  double const default_surface =
+    defaults ? defaults->specific_surface : 0.0;
+  double const default_rho_sr =
+    defaults ? defaults->micro_solid_density_reference : 0.0;
+  double const default_ns =
+    defaults ? defaults->micro_solid_volume_fraction_reference : 0.0;
+  double const default_rho_l0 =
+    defaults ? defaults->micro_liquid_density_reference : 0.0;
+  double const default_a_rho =
+    defaults ? defaults->micro_liquid_density_a : 0.0;
+  double const default_b_rho =
+    defaults ? defaults->micro_liquid_density_b : 0.0;
+
+  double hamaker_constant = 0.0;
+  double specific_surface = 0.0;
+  double micro_solid_density_reference = 0.0;
+  double micro_solid_volume_fraction_reference = 0.0;
+  double micro_liquid_density_reference = 0.0;
+  double micro_liquid_density_a = 0.0;
+  double micro_liquid_density_b = 0.0;
+  bool const uses_micro_liquid_density_eos =
+    local_nonlinear_solve_mode ==
+    LocalNonlinearSolveMode::ScalarReferenceMassStorage;
+
+  if (enabled)
+  {
+    hamaker_constant =
+      get_positive_required_or_default("hamaker_constant",
+                                     default_hamaker);
+    specific_surface =
+      get_positive_required_or_default("specific_surface",
+                                     default_surface);
+    micro_solid_density_reference = get_positive_required_or_default(
+      "micro_solid_density_reference", default_rho_sr);
+    micro_solid_volume_fraction_reference =
+      get_positive_required_or_default(

```

```

+         "micro_solid_volume_fraction_reference", default_ns);
+     if (uses_micro_liquid_density_eos)
+     {
+         micro_liquid_density_reference =
+         get_positive_required_or_default(
+             "micro_liquid_density_reference", default_rho_l0);
+         micro_liquid_density_a = get_positive_required_or_default(
+             "micro_liquid_density_a", default_a_rho);
+         micro_liquid_density_b = get_positive_required_or_default(
+             "micro_liquid_density_b", default_b_rho);
+     }
+     else
+     {
+         micro_liquid_density_reference =
+         get_positive_optional_or_default(
+             "micro_liquid_density_reference",
+             defaults ? std::optional<double>{
+                 defaults->micro_liquid_density_reference)
+                 : std::nullopt)
+             .value_or(0.0);
+         micro_liquid_density_a = get_positive_optional_or_default(
+             "micro_liquid_density_a",
+             defaults
+                 ? std::optional<double>{
+                     defaults->micro_liquid_density_a)
+                 : std::nullopt)
+             .value_or(0.0);
+         micro_liquid_density_b = get_positive_optional_or_default(
+             "micro_liquid_density_b",
+             defaults
+                 ? std::optional<double>{
+                     defaults->micro_liquid_density_b)
+                 : std::nullopt)
+             .value_or(0.0);
+     }
+ }
+ }
+ else
+ {
+     hamaker_constant = get_positive_optional_or_default(
+         "hamaker_constant",
+         defaults ? std::optional<double>{
+             defaults->hamaker_constant}
+             : std::nullopt)
+         .value_or(0.0);
+     specific_surface = get_positive_optional_or_default(
+         "specific_surface",
+         defaults ? std::optional<double>{
+             defaults->specific_surface}
+             : std::nullopt)
+         .value_or(0.0);
+     micro_solid_density_reference =
+     get_positive_optional_or_default(
+         "micro_solid_density_reference",
+         defaults ? std::optional<double>{
+             defaults->micro_solid_density_reference)
+             : std::nullopt)
+         .value_or(0.0);
+     micro_solid_volume_fraction_reference =
+     get_positive_optional_or_default(
+         "micro_solid_volume_fraction_reference",
+         defaults ? std::optional<double>{
+             defaults->micro_solid_volume_fraction_reference)
+             : std::nullopt)
+         .value_or(0.0);
+     micro_liquid_density_reference =
+     get_positive_optional_or_default(
+         "micro_liquid_density_reference",
+         defaults ? std::optional<double>{
+             defaults->micro_liquid_density_reference)
+             : std::nullopt)
+         .value_or(0.0);
+     micro_liquid_density_a =
+     get_positive_optional_or_default(
+         "micro_liquid_density_a",
+         defaults
+             ? std::optional<double>{defaults->micro_liquid_density_a}
+             : std::nullopt)
+         .value_or(0.0);
+     micro_liquid_density_b =
+     get_positive_optional_or_default(
+         "micro_liquid_density_b",
+         defaults
+             ? std::optional<double>{defaults->micro_liquid_density_b}
+             : std::nullopt)
+         .value_or(0.0);
+ }
+ }
+
+ auto const initial_micro_water_content = get_positive_optional_or_default(
+     "initial_micro_water_content",
+     defaults ? defaults->initial_micro_water_content : std::nullopt);
+
+ auto const use_fd_jacobian_for_exchange = config.getConfigParameter<bool>(
+     "fd_jacobian_for_exchange",
+     defaults ? defaults->use_fd_jacobian_for_exchange : false);
+
+ auto const fd_jacobian_perturbation = config.getConfigParameter<double>(
+     "fd_jacobian_perturbation",
+     defaults ? defaults->fd_jacobian_perturbation : 1e-8);
+ if (!(fd_jacobian_perturbation > 0.0))
+ {
+     OGS_FATAL(
+         "RichardsMechanics: {} fd_jacobian_perturbation must be > 0, got {g}.",
+         context, fd_jacobian_perturbation);
+ }
+
+ auto const check_local_jacobian = config.getConfigParameter<bool>(
+     "check_local_jacobian",
+     defaults ? defaults->check_local_jacobian : false);
+
+ auto const local_jacobian_perturbation = config.getConfigParameter<double>(
+     "local_jacobian_perturbation",
+     defaults ? defaults->local_jacobian_perturbation : 1e-8);
+ if (!(local_jacobian_perturbation > 0.0))
+ {
+     OGS_FATAL(
+         "RichardsMechanics: {} local_jacobian_perturbation must be > 0, got {g}.",
+         context, local_jacobian_perturbation);
+ }

```

```

+ }
+
+ auto const local_jacobian_relative_tolerance =
+   config.getConfigParameter<double>{
+     "local_jacobian_relative_tolerance",
+     defaults ? defaults->local_jacobian_relative_tolerance : 1e-3;
+   }
+   if (!(local_jacobian_relative_tolerance >= 0.0))
+   {
+     OGS_FATAL(
+       "RichardsMechanics: {} local_jacobian_relative_tolerance must be >= 0, got {g}.",
+       context, local_jacobian_relative_tolerance);
+   }
+
+ auto const vdw_relaxation_stress_gain = config.getConfigParameter<double>{
+   "vdw_relaxation_stress_gain",
+   defaults ? defaults->vdw_relaxation_stress_gain : 0.0;
+ }
+   if (!(vdw_relaxation_stress_gain >= 0.0))
+   {
+     OGS_FATAL(
+       "RichardsMechanics: {} vdw_relaxation_stress_gain must be >= 0, got {g}.",
+       context, vdw_relaxation_stress_gain);
+   }
+
+ auto const micro_water_content_stress_gain =
+   config.getConfigParameter<double>{
+     "micro_water_content_stress_gain",
+     defaults ? defaults->micro_water_content_stress_gain : 0.0;
+   }
+   if (!(micro_water_content_stress_gain >= 0.0))
+   {
+     OGS_FATAL(
+       "RichardsMechanics: {} micro_water_content_stress_gain must be >= 0, got {g}.",
+       context, micro_water_content_stress_gain);
+   }
+
+ auto const micro_water_content_swelling_slope =
+   config.getConfigParameter<double>{
+     "micro_water_content_swelling_slope",
+     defaults ? defaults->micro_water_content_swelling_slope : 0.0;
+   }
+   if (!(micro_water_content_swelling_slope >= 0.0))
+   {
+     OGS_FATAL(
+       "RichardsMechanics: {} micro_water_content_swelling_slope must be >= 0, got {g}.",
+       context, micro_water_content_swelling_slope);
+   }
+
+ return PotentialExchangeParameters{
+   enabled,
+   mode,
+   pressure_tolerance,
+   hamaker_constant,
+   specific_surface,
+   micro_solid_density_reference,
+   micro_solid_volume_fraction_reference,
+   micro_liquid_density_reference,
+   micro_liquid_density_a,
+   micro_liquid_density_b,
+   micro_potential_convention,
+   local_nonlinear_solve_mode,
+   macro_porosity_update_mode,
+   micro_solid_volume_fraction_mode,
+   initial_micro_water_content,
+   use_fd_jacobian_for_exchange,
+   fd_jacobian_perturbation,
+   check_local_jacobian,
+   local_jacobian_perturbation,
+   local_jacobian_relative_tolerance,
+   vdw_relaxation_stress_gain,
+   micro_water_content_stress_gain,
+   micro_water_content_swelling_slope;
+};
+
+ template <int DisplacementDim>
+ std::unique_ptr<Process> createRichardsMechanicsProcess(
+   std::string const& name,
+   @ @ -178,6 +652,42 @ @ std::unique_ptr<Process> createRichardsMechanicsProcess(
+     "mass_exchange_coefficient");
+   )
+
+ std::optional<PotentialExchangeParameters> potential_exchange_parameters;
+ std::map<int, PotentialExchangeParameters>
+   potential_exchange_parameters_by_material;
+ if (auto const potential_exchange_config =
+   //! \ogs_file_param{prj__processes__process__RICHARDS_MECHANICS__potential_exchange}
+   config.getConfigSubtreeOptional("potential_exchange"))
+ {
+   potential_exchange_parameters = parsePotentialExchangeParameters(
+     *potential_exchange_config, std::nullopt,
+     "potential_exchange");
+
+   for (auto medium_config :
+     potential_exchange_config->getConfigSubtreeList("medium"))
+   {
+     int const material_id = medium_config.getConfigAttribute<int>("id");
+     if (!potential_exchange_parameters_by_material
+       .emplace(material_id,
+         parsePotentialExchangeParameters(
+           medium_config,
+           potential_exchange_parameters,
+           fmt::format(
+             "potential_exchange medium id {}",
+             material_id)))
+       .second)
+     {
+       OGS_FATAL(
+         "RichardsMechanics: duplicate potential_exchange medium override for material id {}.",
+         material_id);
+     }
+   }
+ }
+
+ validateMicroPorosityAndPotentialExchangeConfiguration(
+   media, micro_porosity_parameters, potential_exchange_parameters,
+   potential_exchange_parameters_by_material);
+
+ auto const mass_lumping =
+   //! \ogs_file_param{prj__processes__process__RICHARDS_MECHANICS__mass_lumping}
+   config.getConfigParameter<bool>("mass_lumping", false);

```



```

+
+enum class MicroSolidVolumeFractionMode
+{
+    Reference,
+    CurrentPorositySplit
+};
+
+inline constexpr char const* toString(
+    MicroPotentialConvention const convention)
+{
+    switch (convention)
+    {
+        case MicroPotentialConvention::PositiveReduced:
+            return "positive_reduced";
+        case MicroPotentialConvention::NegativeAttractive:
+            return "negative_attractive";
+    }
+    return "unknown";
+}
+
+inline constexpr double microPotentialSignFactor(
+    MicroPotentialConvention const convention)
+{
+    return convention == MicroPotentialConvention::NegativeAttractive ? -1.0
+        : 1.0;
+}
+
+inline constexpr char const* toString(LocalsNonlinearSolveMode const mode)
+{
+    switch (mode)
+    {
+        case LocalsNonlinearSolveMode::ScalarExchange:
+            return "scalar_exchange";
+        case LocalsNonlinearSolveMode::ScalarReferenceStorage:
+            return "scalar_microstate_storage_mode";
+        case LocalsNonlinearSolveMode::ScalarReferenceMassStorage:
+            return "scalar_micro_macro_mass_storage_mode";
+    }
+    return "unknown";
+}
+
+inline constexpr char const* toString(MacroPorosityUpdateMode const mode)
+{
+    switch (mode)
+    {
+        case MacroPorosityUpdateMode::AlgebraicSplit:
+            return "algebraic_split";
+        case MacroPorosityUpdateMode::ReferenceAdditiveRate:
+            return "additive_macro_porosity_rate_mode";
+    }
+    return "unknown";
+}
+
+inline constexpr char const* toString(
+    MicroSolidVolumeFractionMode const mode)
+{
+    switch (mode)
+    {
+        case MicroSolidVolumeFractionMode::Reference:
+            return "reference";
+        case MicroSolidVolumeFractionMode::CurrentPorositySplit:
+            return "current_porosity_split";
+    }
+    return "unknown";
+}
+
+struct PotentialExchangeParameters
+{
+    bool enabled = false;
+    PotentialExchangeMode mode = PotentialExchangeMode::FullPotential;
+
+    // Young-Laplace macro potential branch tolerance.
+    double pressure_tolerance = 0.0;
+
+    // vdW microscale potential parameters / reference state constants.
+    double hamaker_constant = 0.0;
+    double specific_surface = 0.0;
+    double micro_solid_density_reference = 0.0; // rho_SR
+    double micro_solid_volume_fraction_reference = 0.0; // n_S
+    double micro_liquid_density_reference = 0.0; // rho_L0
+    double micro_liquid_density_a = 0.0; // a_rho
+    double micro_liquid_density_b = 0.0; // b_rho
+    MicroPotentialConvention micro_potential_convention =
+        MicroPotentialConvention::PositiveReduced;
+    LocalsNonlinearSolveMode locals_nonlinear_solve_mode =
+        LocalsNonlinearSolveMode::ScalarExchange;
+    MacroPorosityUpdateMode macro_porosity_update_mode =
+        MacroPorosityUpdateMode::AlgebraicSplit;
+    MicroSolidVolumeFractionMode micro_solid_volume_fraction_mode =
+        MicroSolidVolumeFractionMode::Reference;
+
+    // Optional GP-local n_l initialization (future full 2C path).
+    std::optional<double> initial_micro_water_content;
+
+    // Optional Jacobian approximation for DSM exchange contribution only.
+    // If true, drho_L_hat/dp_L is computed by finite difference in the local
+    // helper path.
+    bool use_fd_jacobian_for_exchange = false;
+    double fd_jacobian_perturbation = 1e-8;
+
+    // Optional diagnostic self-check for the local implicit n_l chain rule.
+    // If enabled, the code compares analytic and FD local derivatives once.
+    bool check_locals_jacobian = false;
+    double locals_jacobian_perturbation = 1e-8;
+    double locals_jacobian_relative_tolerance = 1e-3;
+
+    // Optional DSM-only mechanical gain on relaxation of the vdW-derived
+    // compatibility pressure p_L_m = -rho_LR * mu_LR. Zero preserves the
+    // current committed behavior.
+    double vdW_relaxation_stress_gain = 0.0;
+
+    // Optional DSM-only mechanical gain on positive microscale water-content
+    // increments. This is intended for the dsm_micromacro-consistent fully saturated
+    // microscale interpretation, where swelling is driven by n_l growth rather
+    // than by a change in saturation.
+    double micro_water_content_stress_gain = 0.0;
+}

```

```

+ // Optional DSM-only dsm_micromacro-style reversible swelling-strain slope driven
+ // by signed microscale water-content increments Delta n_l. If enabled, the
+ // DSM branch uses Delta eps_sw = slope * Delta n_l and converts that
+ // isotropic swelling-strain increment into a stress increment via the
+ // elastic stiffness, instead of using the legacy saturation-driven
+ // swelling_stress_rate path.
+ double micro_water_content_swelling_slope = 0.0;
+};

+ // namespace ProcessLib::RichardsMechanics
diff --git a/ProcessLib/RichardsMechanics/RichardsMechanicsFEM-impl.h b/ProcessLib/RichardsMechanics/RichardsMechanicsFEM-impl.h
index 99fcc0b837..a4dbd4ae3 100644
--- a/ProcessLib/RichardsMechanics/RichardsMechanicsFEM-impl.h
+++ b/ProcessLib/RichardsMechanics/RichardsMechanicsFEM-impl.h
@@ -3,11 +3,16 @@

#pragma once

#include <algorithm>
#include <cmath>
#include <Eigen/Lib>
#include <cassert>
#include <mutex>

#include "BaseLib/Logging.h"
#include "ComputeMicroPorosity.h"
#include "ConstitutiveRelations/ConstitutiveModels.h"
#include "ConstitutiveRelations/PotentialExchange.h"
#include "IntegrationPointData.h"
#include "MaterialLib/MPL/Medium.h"
#include "MaterialLib/MPL/Utils/FormEigenTensor.h"
@@ -23,6 +28,1717 @@ namespace ProcessLib
{
    namespace RichardsMechanics
    {
        inline bool isPotentialExchangeEnabled(
            PotentialExchangeParameters const& const potential_exchange_parameters)
        {
            return potential_exchange_parameters &&
                potential_exchange_parameters->enabled &&
                potential_exchange_parameters->mode ==
                    PotentialExchangeMode::FullPotential;
        }

        inline bool isPotentialExchangeEnabled(
            std::optional<PotentialExchangeParameters> const&
                potential_exchange_parameters)
        {
            return isPotentialExchangeEnabled(
                potential_exchange_parameters ? &*potential_exchange_parameters
                : nullptr);
        }

        inline double getPotentialPressureTolerance(
            PotentialExchangeParameters const& const potential_exchange_parameters)
        {
            if (!isPotentialExchangeEnabled(potential_exchange_parameters))
            {
                return 0.0;
            }
            return potential_exchange_parameters->pressure_tolerance;
        }

        inline double getPotentialPressureTolerance(
            std::optional<PotentialExchangeParameters> const&
                potential_exchange_parameters)
        {
            return getPotentialPressureTolerance(
                potential_exchange_parameters ? &*potential_exchange_parameters
                : nullptr);
        }

        struct PotentialExchangeUpdateData
        {
            YoungLaplaceMacroPotentialData macro_potential;
            PotentialDrivenMassExchangeData exchange;

            double alpha_M_effective = 0.0;
            double mu_LR_active = 0.0;
            double mu_LR_exchange_input = 0.0;
            bool use_macro_potential_for_active_exchange = false;
            bool use_vdw_micro_potential_for_active_exchange = false;
            bool use_fd_jacobian_for_direct_macro_derivative = false;
            double fd_jacobian_perturbation = 0.0;

            // Direct macro derivative (with density dependence through rho_LR), while
            // keeping the microscale state lagged.
            double drho_L_hat_dpl_direct = 0.0;
        };

        inline PotentialExchangeUpdateData computePotentialExchangeUpdate(
            double const alpha_bar, double const mu, double const p_L_ip,
            double const p_L_m, double const rho_LR, double const beta_LR,
            double const pressure_tolerance = 0.0,
            bool const use_macro_potential_for_active_exchange = false,
            bool const use_vdw_micro_potential_for_active_exchange = false,
            double const mu_LR_vdw = 0.0,
            double const dmu_LR_vdw_drho_LR = 0.0,
            bool const use_custom_dmu_LR_vdw_dpl = false,
            double const dmu_LR_vdw_dpl = 0.0,
            bool const use_fd_jacobian_for_direct_macro_derivative = false,
            double const fd_jacobian_perturbation = 1e-8)
        {
            if (!(mu > 0.0))
            {
                OGS_FATAL(
                    "computePotentialExchangeUpdate requires mu > 0, got {g}.",
                    mu);
            }

            PotentialExchangeUpdateData out;

            // Keep the exchange coefficient scaling in mass-density units.
            out.alpha_M_effective = alpha_bar * rho_LR / mu;

            out.macro_potential =
                computeYoungLaplaceMacroPotential(p_L_ip, rho_LR, pressure_tolerance);

```



```

+ out.use_macro_potential_for_active_exchange =
+   use_macro_potential_for_active_exchange;
+ out.use_vdw_micro_potential_for_active_exchange =
+   use_vdw_micro_potential_for_active_exchange;
+ out.use_fd_jacobian_for_direct_macro_derivative =
+   use_fd_jacobian_for_direct_macro_derivative;
+ out.fd_jacobian_perturbation = fd_jacobian_perturbation;
+
+ out.mu_LR_exchange_input = use_vdw_micro_potential_for_active_exchange
+   ? mu_LR_vdw
+   : p_L_m / rho_LR;
+ out.mu_LR_active = use_macro_potential_for_active_exchange
+   ? out.macro_potential.mu_LR
+   : p_L_ip / rho_LR;
+
+ out.exchange = computePotentialDrivenMassExchange(
+   out.alpha_M_effective, out.mu_LR_active, out.mu_LR_exchange_input);
+
+ // rho_LR depends on liquid pressure in RM through beta_LR = (1/rho) drho/dp.
+ double const drho_LR_dpl = rho_LR * beta_LR;
+
+ if (use_fd_jacobian_for_direct_macro_derivative)
+ {
+   auto const compute_rho_L_hat = [G](double const p_L_ip_eval,
+                                     double const rho_LR_eval)
+   {
+     auto const macro_potential_eval = computeYoungLaplaceMacroPotential(
+       p_L_ip_eval, rho_LR_eval, pressure_tolerance);
+     double const alpha_M_effective_eval =
+       alpha_bar * rho_LR_eval / mu;
+     double const mu_LR_active_eval = use_macro_potential_for_active_exchange
+       ? macro_potential_eval.mu_LR
+       : p_L_ip_eval / rho_LR_eval;
+     double const mu_LR_active_eval = use_vdw_micro_potential_for_active_exchange
+       ? mu_LR_vdw
+       : p_L_m / rho_LR_eval;
+     auto const exchange_eval = computePotentialDrivenMassExchange(
+       alpha_M_effective_eval, mu_LR_active_eval, mu_LR_active_eval);
+     return -exchange_eval.rho_L_hat;
+   };
+
+   double const h =
+     fd_jacobian_perturbation * std::max(1.0, std::abs(p_L_ip));
+   if (!!(h > 0.0) || !std::isfinite(h))
+   {
+     OGS_FATAL(
+       "computePotentialExchangeUpdate requires finite h > 0 for FD Jacobian, got {ig} (from fd_jacobian_perturbation={ig}, p_L_ip={ig}).",
+       h, fd_jacobian_perturbation, p_L_ip);
+   }
+
+   constexpr double rho_floor = 1e-16;
+   double const rho_plus = std::max(rho_floor, rho_LR + drho_LR_dpl * h);
+   double const rho_minus = rho_LR - drho_LR_dpl * h;
+   double const rho_L_hat_plus = compute_rho_L_hat(p_L_ip + h, rho_plus);
+   if (rho_minus > rho_floor)
+   {
+     double const rho_L_hat_minus =
+       compute_rho_L_hat(p_L_ip - h, rho_minus);
+     out.drho_L_hat_dpl_direct =
+       (rho_L_hat_plus - rho_L_hat_minus) / (2.0 * h);
+   }
+   else
+   {
+     double const rho_L_hat = -out.exchange.rho_L_hat;
+     out.drho_L_hat_dpl_direct = (rho_L_hat_plus - rho_L_hat) / h;
+   }
+
+   return out;
+ }
+
+ // alpha_M_effective = alpha_bar * rho_LR / mu (mu dependence is lagged).
+ double const dalpha_M_effective_dpl = alpha_bar / mu * drho_LR_dpl;
+
+ double const dmu_LR_dpl = use_macro_potential_for_active_exchange
+   ? out.macro_potential.dmu_LR_dpl +
+     out.macro_potential.dmu_LR_drho_LR *
+     drho_LR_dpl
+   : 1.0 / rho_LR -
+     p_L_ip / (rho_LR * rho_LR) * drho_LR_dpl;
+
+ double const dmu_LR_exchange_input_dpl =
+   use_vdw_micro_potential_for_active_exchange
+   ? (use_custom_dmu_LR_vdw_dpl
+     ? dmu_LR_vdw_dpl
+     : dmu_LR_vdw_drho_LR * drho_LR_dpl)
+   : -p_L_m / (rho_LR * rho_LR) * drho_LR_dpl;
+
+ double const drho_L_hat_dpl_direct =
+   out.exchange.drho_L_hat_dalphi_M * dalphi_M_effective_dpl +
+   out.exchange.drho_L_hat_dmu_LR * dmu_LR_dpl +
+   out.exchange.drho_L_hat_dmu_LR * dmu_LR_exchange_input_dpl;
+
+ out.drho_L_hat_dpl_direct = -drho_L_hat_dpl_direct;
+ return out;
+}
+
+struct ImplicitMicroWaterContentUpdateData
+{
+  double n_l = 0.0;
+  VanDerWaalsMicroPotentialData micro_potential;
+  PotentialDrivenMassExchangeData exchange;
+  bool converged = true;
+};
+
+struct CompatibilityMicroHydraulicOutputData
+{
+  double p_L_m = 0.0;
+  double S_L_m = 0.0;
+  double n_l_ref = 0.0;
+  VanDerWaalsMicroPotentialData micro_potential;
+};
+
+inline double microPotentialSignFactorFromParameters(
+  PotentialExchangeParameters const& potential_exchange_params)
+{
+  return microPotentialSignFactor(potential_exchange_params.micro_potential_convention);
+}

```

```

+
+inline CompatibilityMicroHydraulicOutputData
+computeCompatibilityMicroHydraulicOutput(
+    double const n_l, double const rho_LR,
+    PotentialExchangeParameters const& potential_exchange_params)
+{
+    double const n_l_safe = std::max(1e-16, n_l);
+    double const n_l_ref = std::max(
+        1e-16, potential_exchange_params.initial_micro_water_content.value_or(
+            potential_exchange_params.micro_solid_volume_fraction_reference));
+
+    auto const micro_potential = computeVanDerWaalsMicroPotential(
+        n_l_safe, rho_LR, potential_exchange_params.micro_solid_volume_fraction_reference,
+        potential_exchange_params.micro_solid_density_reference, potential_exchange_params.hamaker_constant,
+        potential_exchange_params.specific_surface,
+        microPotentialSignFactorFromParameters(potential_exchange_params));
+
+    return {
+        .p_L_m = -rho_LR * micro_potential.mu_LR,
+        .S_L_m = n_l_safe / n_l_ref,
+        .n_L_ref = n_l_ref,
+        .micro_potential = micro_potential,
+    };
+}
+
+struct TransportPorosityUpdateData
+{
+    double phi_M = 0.0;
+    double phi_M_prev = 0.0;
+    double phi_m = 0.0;
+    double phi_m_prev = 0.0;
+};
+
+struct PotentialExchangeLocalSolveContext
+{
+    double phi = 1.0;
+    double phi_M_prev = 0.0;
+    double phi_m_prev = 0.0;
+    double volumetric_strain = 0.0;
+    double volumetric_strain_prev = 0.0;
+};
+
+inline double boundedMicroWaterContentCeiling(
+    PotentialExchangeLocalSolveContext const& local_context,
+    double const n_L_floor)
+{
+    constexpr double porosity_upper = 1.0 - 1e-12;
+    auto const compute_total_porosity_bound = [&]()
+    {
+        double const phi_prev_sum = std::clamp(
+            std::max(0.0, local_context.phi_M_prev) +
+            std::max(0.0, local_context.phi_m_prev),
+            0.0, porosity_upper);
+        if (std::isfinite(local_context.phi))
+        {
+            return std::clamp(std::max(0.0, local_context.phi), 0.0,
+                porosity_upper);
+        }
+
+        double const delta_eps_v =
+            local_context.volumetric_strain - local_context.volumetric_strain_prev;
+        double const denominator = 1.0 + delta_eps_v;
+        if (std::isfinite(denominator) && std::abs(denominator) > 1e-12)
+        {
+            double const phi_from_kinematics =
+                (phi_prev_sum + delta_eps_v) / denominator;
+            if (std::isfinite(phi_from_kinematics))
+            {
+                return std::clamp(phi_from_kinematics, 0.0, porosity_upper);
+            }
+        }
+
+        return phi_prev_sum;
+    };
+
+    return std::max(n_L_floor, compute_total_porosity_bound());
+}
+
+inline TransportPorosityUpdateData computeTransportPorosityUpdate(
+    double const phi, double const phi_M_prev, double const phi_m_prev,
+    double const n_l, double const volumetric_strain,
+    double const volumetric_strain_prev,
+    MacroPorosityUpdateMode const macro_porosity_update_mode)
+{
+    constexpr double porosity_upper = 1.0 - 1e-12;
+    double const phi_prev_sum = std::clamp(
+        std::max(0.0, phi_M_prev) + std::max(0.0, phi_m_prev), 0.0,
+        porosity_upper);
+    double const delta_eps_v = volumetric_strain - volumetric_strain_prev;
+    double const denominator = 1.0 + delta_eps_v;
+    double phi_safe = phi_prev_sum;
+    if (std::isfinite(phi))
+    {
+        phi_safe = std::clamp(std::max(0.0, phi), 0.0, porosity_upper);
+    }
+    else if (std::isfinite(denominator) && std::abs(denominator) > 1e-12)
+    {
+        double const phi_from_kinematics =
+            (phi_prev_sum + delta_eps_v) / denominator;
+        if (std::isfinite(phi_from_kinematics))
+        {
+            phi_safe = std::clamp(phi_from_kinematics, 0.0, porosity_upper);
+        }
+    }
+    double const phi_M_prev_safe = std::min(std::max(0.0, phi_M_prev), phi_safe);
+    double const phi_m_prev_safe =
+        std::min(std::max(0.0, phi_m_prev), std::max(0.0, phi_safe - phi_M_prev_safe));
+
+    if (macro_porosity_update_mode ==
+        MacroPorosityUpdateMode::AlgebraicSplit)
+    {
+        double const phi_m = std::clamp(n_l, 0.0, phi_safe);
+        return {
+            .phi_M = phi_safe - phi_m,
+            .phi_M_prev = phi_M_prev_safe,
+            .phi_m = phi_m,
+            .phi_m_prev = phi_m_prev_safe,
+        };
+    }
+}

```

```

+     };
+ }
+
+ double const phi_m = std::clamp(n_l, 0.0, phi_safe);
+
+ if (!(std::isfinite(denominator) && std::abs(denominator) > 1e-12))
+ {
+     static std::once_flag once;
+     std::call_once(once, []
+     {
+         WARN(
+             "[RM Phase6I] additive_macro_porosity_rate_mode porosity update encountered a near-singular denominator and fell back to algebraic_split at least once.");
+         });
+     return {
+         .phi_M = std::max(0.0, phi_safe - phi_m),
+         .phi_M_prev = phi_M_prev_safe,
+         .phi_m = phi_m,
+         .phi_m_prev = phi_m_prev_safe,
+     };
+ }
+
+ double const phi_M_candidate =
+     (phi_M_prev_safe + (1.0 - phi_m) * delta_eps_v -
+      (phi_m - phi_m_prev_safe)) /
+     denominator;
+ double const phi_M = std::clamp(
+     phi_M_candidate, 0.0, std::max(0.0, phi_safe - phi_m));
+
+ return {
+     .phi_M = phi_M,
+     .phi_M_prev = phi_M_prev_safe,
+     .phi_m = phi_m,
+     .phi_m_prev = phi_m_prev_safe,
+ };
+ }
+
+ inline double computeActiveMicroSolidVolumeFraction(
+     double const n_l, PotentialExchangeLocalSolveContext const& local_context,
+     PotentialExchangeParameters const& potential_exchange_params)
+ {
+     if (potential_exchange_params.micro_solid_volume_fraction_mode ==
+         MicroSolidVolumeFractionMode::Reference)
+     {
+         return std::max(1e-16, potential_exchange_params.micro_solid_volume_fraction_reference);
+     }
+
+     auto const split = computeTransportPorosityUpdate(
+         local_context.phi, local_context.phi_M_prev, local_context.phi_m_prev,
+         n_l, local_context.volumetric_strain,
+         local_context.volumetric_strain_prev,
+         potential_exchange_params.macro_porosity_update_mode);
+     return std::max(1e-16, 1.0 - split.phi_M - split.phi_m);
+ }
+
+ inline double computePreviousMicroSolidVolumeFraction(
+     PotentialExchangeLocalSolveContext const& local_context,
+     PotentialExchangeParameters const& potential_exchange_params)
+ {
+     if (potential_exchange_params.micro_solid_volume_fraction_mode ==
+         MicroSolidVolumeFractionMode::Reference)
+     {
+         return std::max(1e-16, potential_exchange_params.micro_solid_volume_fraction_reference);
+     }
+
+     double const total_prev_porosity = std::clamp(
+         std::max(0.0, local_context.phi_M_prev) +
+         std::max(0.0, local_context.phi_m_prev),
+         0.0, 1.0 - 1e-12);
+     return std::max(1e-16, 1.0 - total_prev_porosity);
+ }
+
+ struct ReducedMicroLiquidDensityData
+ {
+     double rho_LR = 0.0;
+     double omega_l = 0.0;
+     double drho_LR_dnl = 0.0;
+     double drho_l_dnl = 0.0;
+ };
+
+ inline ReducedMicroLiquidDensityData computeReducedMicroLiquidDensity(
+     double const n_l, double const rho_LR, double const n5,
+     PotentialExchangeParameters const& potential_exchange_params)
+ {
+     double const n_l_safe = std::max(1e-16, n_l);
+     double const n5_safe = std::max(1e-16, n5);
+     double const rho_SR = std::max(1e-16, potential_exchange_params.micro_solid_density_reference);
+     double const rho_l0 = std::max(1e-16, potential_exchange_params.micro_liquid_density_reference);
+     double const a_rho = std::max(1e-16, potential_exchange_params.micro_liquid_density_a);
+     double const b_rho = std::max(1e-16, potential_exchange_params.micro_liquid_density_b);
+     double const denominator = n5_safe * rho_SR;
+
+     auto const eval_rhs = [&](double const rho_LR)
+     {
+         double const omega_l =
+             std::max(1e-16, n_l_safe * rho_LR / denominator);
+         double const exp_term =
+             std::exp(-a_rho * std::pow(omega_l, b_rho));
+         return std::pair(omega_l, rho_l0 * exp_term + rho_LR);
+     };
+
+     double rho_LR = rho_LR +
+         rho_l0 *
+         std::exp(-a_rho *
+             std::pow(std::max(1e-16, n_l_safe * rho_LR /
+                 denominator),
+                 b_rho));
+
+     constexpr int max_iterations = 30;
+     constexpr double tolerance = 1e-14;
+     bool converged = false;
+
+     for (int iter = 0; iter < max_iterations; ++iter)
+     {
+         auto const [omega_l, rhs] = eval_rhs(rho_LR);
+         double const residual = rho_LR - rhs;
+         if (std::abs(residual) <=
+             tolerance * std::max(1.0, std::abs(rho_LR)))
+         {

```

```

+         converged = true;
+         break;
+     }
+
+     double const common =
+         (rhs - rho_LR) * a_rho * b_rho *
+         std::pow(omega_L, b_rho - 1.0);
+     double const jacobian =
+         1.0 + common * (n_l_safe / denominator);
+     if (!std::isfinite(jacobian) && std::abs(jacobian) > 1e-20)
+     {
+         break;
+     }
+
+     double const rho_candidate =
+         std::max(1e-16, rho_LR - residual / jacobian);
+     if (std::abs(rho_candidate - rho_LR) <=
+         tolerance * std::max(1.0, std::abs(rho_LR)))
+     {
+         rho_LR = rho_candidate;
+         converged = true;
+         break;
+     }
+     rho_LR = rho_candidate;
+ }
+
+ if (!converged)
+ {
+     static std::once_flag once;
+     std::call_once(once, []
+     {
+         WARN(
+             "IRM Phase6K] reduced microscale liquid-density EOS did not converge at least once; using the last Newton iterate.");
+     });
+ }
+
+ auto const [omega_L, rhs] = eval_rhs(rho_LR);
+ (void)rhs;
+ double const common =
+     (rho_LR - rho_LR) * a_rho * b_rho *
+     std::pow(omega_L, b_rho - 1.0);
+ double const dg_drho =
+     1.0 + common * (n_l_safe / denominator);
+ double const dg_dn = common * (rho_LR / denominator);
+ double const drho_LR_dnl =
+     (std::isfinite(dg_drho) && std::abs(dg_drho) > 1e-20)
+     ? -dg_dn / dg_drho
+     : 0.0;
+
+ return {
+     .rho_LR = rho_LR,
+     .omega_L = omega_L,
+     .drho_LR_dnl = drho_LR_dnl,
+     .drho_L_dnl = rho_LR + n_l_safe * drho_LR_dnl,
+ };
+}
+
+inline ReducedMicroLiquidDensityData computeActiveMicroLiquidDensity(
+    double const n_l, double const rho_LR,
+    PotentialExchangeLocalSolveContext const& local_context,
+    PotentialExchangeParameters const& potential_exchange_params)
+{
+    double const active_nS =
+        computeActiveMicroSolidVolumeFraction(n_l, local_context, potential_exchange_params);
+    return computeReducedMicroLiquidDensity(n_l, rho_LR, active_nS, potential_exchange_params);
+}
+
+inline ReducedMicroLiquidDensityData computePreviousMicroLiquidDensity(
+    double const n_l_prev, double const rho_LR,
+    PotentialExchangeLocalSolveContext const& local_context,
+    PotentialExchangeParameters const& potential_exchange_params)
+{
+    double const previous_nS =
+        computePreviousMicroSolidVolumeFraction(local_context, potential_exchange_params);
+    return computeReducedMicroLiquidDensity(n_l_prev, rho_LR, previous_nS,
+        potential_exchange_params);
+}
+
+struct MicroMacroMassStorageCoupledSolveData
+{
+    double n_l = 0.0;
+    double rho_LR = 0.0;
+    double phi_m = 0.0;
+    double phi_M = 0.0;
+    double p_L_m = 0.0;
+    double S_L_m = 0.0;
+    VanDerWaalsMicroPotentialData micro_potential;
+    PotentialDrivenMassExchangeData exchange;
+    bool converged = true;
+};
+
+inline MicroMacroMassStorageCoupledSolveData
+solveReferenceMassStoragePredictorState(
+    double const n_l_prev, double const rho_L_prev, double const rho_LR_prev,
+    double const dt, double const rho_LR, double const alpha_bar,
+    double const mu, YoungLaplaceMacroPotentialData const& macro_potential,
+    PotentialExchangeLocalSolveContext const& local_context,
+    PotentialExchangeParameters const& potential_exchange_params)
+{
+    constexpr double n_l_floor = 1e-16;
+    constexpr double rho_floor = 1e-16;
+    double const dt_safe = std::isfinite(dt) && dt > 0.0 ? dt : 0.0;
+    double const alpha_M_effective = alpha_bar * rho_LR / mu;
+    double const volumetric_strain_rate =
+        dt_safe > 0.0
+        ? (local_context.volumetric_strain -
+            local_context.volumetric_strain_prev) /
+            dt_safe
+        : 0.0;
+    double const n_l_ceiling =
+        boundedMicroWaterContentCeiling(local_context, n_l_floor);
+
+    auto evaluate = [&](double const n_l)
+    {
+        double const active_nS = computeActiveMicroSolidVolumeFraction(
+            n_l, local_context, potential_exchange_params);
+        auto const micro_liquid_density = computeReducedMicroLiquidDensity(

```

```

+     n_l, rho_LR, active_n5, potential_exchange_params);
+ auto const micro_potential = computeVanDerWaalsMicroPotential(
+     n_l, micro_liquid_density.rho_LR, active_n5,
+     potential_exchange_params.micro_solid_density_reference, potential_exchange_params.hamaker_constant,
+     potential_exchange_params.specific_surface,
+     microPotentialSignFactorFromParameters(potential_exchange_params));
+ double const mu_LR_active = micro_potential.mu_LR;
+ double const mu_LR_active = micro_potential.mu_LR;
+ auto const exchange = computePotentialDrivenMassExchange(
+     alpha_M_effective, mu_LR_active, mu_LR_active);
+ double const rho_l = n_l * micro_liquid_density.rho_LR;
+ double const residual = rho_l - rho_l_prev -
+     dt_safe * exchange.rho_l_hat -
+     dt_safe * rho_l * volumetric_strain_rate;
+ return std::tuple(residual, micro_potential, exchange,
+     micro_liquid_density);
+ };
+
+ MicroMacroMassStorageCoupledSolveData out;
+ if (dt_safe <= 0.0)
+ {
+     out.n_l = std::clamp(n_l_prev, n_l_floor, n_l_ceiling);
+     out.rho_LR = std::max(rho_floor, rho_LR_prev);
+     auto const [residual, micro_potential, exchange, micro_density] =
+         evaluate(out.n_l);
+     (void)residual;
+     (void)micro_density;
+     out.micro_potential = micro_potential;
+     out.exchange = exchange;
+     return out;
+ }
+
+ double n_l = std::clamp(n_l_prev, n_l_floor, n_l_ceiling);
+ constexpr int max_iterations = 40;
+ constexpr double residual_tolerance = 1e-14;
+ constexpr double increment_tolerance = 1e-14;
+
+ for (int iter = 0; iter < max_iterations; ++iter)
+ {
+     auto const [residual, micro_potential, exchange, micro_density] =
+         evaluate(n_l);
+     if (std::abs(residual) <=
+         residual_tolerance * std::max(1.0, std::abs(rho_l_prev)))
+     {
+         out.n_l = n_l;
+         out.rho_LR = micro_density.rho_LR;
+         out.micro_potential = micro_potential;
+         out.exchange = exchange;
+         return out;
+     }
+
+     double const drho_l_hat_dn_l =
+         exchange.drho_l_hat_dmu_LR * micro_potential.dmu_LR_dn_l;
+     double const jacobian = micro_density.drho_l_dn_l -
+         dt_safe * drho_l_hat_dn_l -
+         dt_safe * micro_density.drho_l_dn_l *
+             volumetric_strain_rate;
+     if (!std::isfinite(jacobian) && std::abs(jacobian) > 1e-20)
+     {
+         break;
+     }
+
+     double delta_n_l = -residual / jacobian;
+     double n_l_candidate =
+         std::clamp(n_l + delta_n_l, n_l_floor, n_l_ceiling);
+     auto const [candidate_residual_initial, candidate_micro_potential,
+         candidate_exchange, candidate_micro_density] =
+         evaluate(n_l_candidate);
+     double candidate_residual = candidate_residual_initial;
+     int backtracking_steps = 0;
+     while (std::abs(candidate_residual) > std::abs(residual) &&
+         backtracking_steps < 12)
+     {
+         delta_n_l *= 0.5;
+         n_l_candidate =
+             std::clamp(n_l + delta_n_l, n_l_floor, n_l_ceiling);
+         auto const [retry_residual, retry_micro_potential,
+             retry_exchange, retry_micro_density] =
+             evaluate(n_l_candidate);
+         (void)retry_micro_potential;
+         (void)retry_exchange;
+         (void)retry_micro_density;
+         candidate_residual = retry_residual;
+         ++backtracking_steps;
+     }
+
+     if (std::abs(n_l_candidate - n_l) <=
+         increment_tolerance * std::max(1.0, std::abs(n_l)))
+     {
+         out.n_l = n_l_candidate;
+         auto const [final_residual, final_micro_potential, final_exchange,
+             final_micro_density] =
+             evaluate(n_l_candidate);
+         (void)final_residual;
+         out.rho_LR = final_micro_density.rho_LR;
+         out.micro_potential = final_micro_potential;
+         out.exchange = final_exchange;
+         out.converged = true;
+         return out;
+     }
+
+     n_l = n_l_candidate;
+     out.n_l = n_l;
+     out.rho_LR = micro_density.rho_LR;
+     out.micro_potential = micro_potential;
+     out.exchange = exchange;
+ }
+
+ auto const [residual, micro_potential, exchange, micro_density] =
+     evaluate(n_l);
+ (void)residual;
+ out.n_l = n_l;
+ out.rho_LR = micro_density.rho_LR;
+ out.micro_potential = micro_potential;
+ out.exchange = exchange;
+ out.converged = false;
+ return out;

```

```

*}
+
+inline MicroMacroMassStorageCoupledSolveData
+solveReferenceMassStorageCoupledState(
+ double const n_l_prev, double const rho_l_prev, double const rho_LR_prev,
+ double const dt, double const rho_LR, double const alpha_bar,
+ double const mu, YoungLaplaceMacroPotentialData const& macro_potential,
+ PotentialExchangeLocalSolveContext const& local_context,
+ PotentialExchangeParameters const& potential_exchange_params)
+{
+ constexpr double n_l_floor = 1e-16;
+ constexpr double rho_floor = 1e-16;
+ double const dt_safe = std::isfinite(dt) && dt > 0.0 ? dt : 0.0;
+ double const alpha_M_effective = alpha_bar * rho_LR / mu;
+ double const volumetric_strain_rate =
+ dt_safe > 0.0
+ ? (local_context.volumetric_strain -
+ local_context.volumetric_strain_prev) /
+ dt_safe
+ : 0.0;
+ double const n_l_ceiling =
+ boundedMicroWaterContentCeiling(local_context, n_l_floor);
+
+ auto evaluate = [&](double const n_l, double const rho_LR)
+ {
+ double const active_nS = computeActiveMicroSolidVolumeFraction(
+ n_l, local_context, potential_exchange_params);
+ auto const micro_potential = computeVanDerWaalsMicroPotential(
+ n_l, rho_LR, active_nS, potential_exchange_params.micro_solid_density_reference,
+ potential_exchange_params.hamaker_constant, potential_exchange_params.specific_surface,
+ microPotentialSignFactorFromParameters(potential_exchange_params));
+ double const mu_LR_active = macro_potential.mu_LR;
+ double const mu_LR_active = micro_potential.mu_LR;
+ auto const exchange = computePotentialDrivenMassExchange(
+ alpha_M_effective, mu_LR_active, mu_LR_active);
+ double const rho_l = n_l * rho_LR;
+ double const mass_residual = rho_l - rho_l_prev -
+ dt_safe * exchange.rho_hat -
+ dt_safe * rho_l * volumetric_strain_rate;
+ auto const density = computeReducedMicroLiquidDensity(
+ n_l, rho_LR, active_nS, potential_exchange_params);
+ double const density_residual = rho_LR - density.rho_LR;
+ return std::tuple(mass_residual, density_residual, micro_potential,
+ exchange);
+ };
+
+ auto const predictor = solveReferenceMassStoragePredictorState(
+ n_l_prev, rho_l_prev, rho_LR_prev, dt, rho_LR, alpha_bar, mu,
+ macro_potential, local_context, potential_exchange_params);
+ if (!predictor.converged)
+ {
+ return predictor;
+ }
+
+ MicroMacroMassStorageCoupledSolveData out = predictor;
+ if (dt_safe <= 0.0)
+ {
+ return out;
+ }
+
+ double n_l = std::clamp(n_l_prev, n_l_floor, n_l_ceiling);
+ double rho_LR = std::max(rho_floor, rho_LR_prev);
+ constexpr int max_iterations = 60;
+ constexpr double residual_tolerance = 1e-10;
+ constexpr double increment_tolerance = 1e-10;
+
+ for (int iter = 0; iter < max_iterations; ++iter)
+ {
+ auto const [mass_residual, density_residual, micro_potential, exchange] =
+ evaluate(n_l, rho_LR);
+
+ double const residual_norm =
+ std::abs(mass_residual) / std::max(1.0, std::abs(rho_l_prev)) +
+ std::abs(density_residual) / std::max(1.0, std::abs(rho_LR));
+ if (residual_norm <= residual_tolerance)
+ {
+ {
+ out.n_l = n_l;
+ out.rho_LR = rho_LR;
+ out.micro_potential = micro_potential;
+ out.exchange = exchange;
+ out.converged = true;
+ return out;
+ }
+
+ double const h_n = 1e-8 * std::max(1.0, std::abs(n_l));
+ double const h_rho = 1e-8 * std::max(1.0, std::abs(rho_LR));
+ auto const [r1_n_plus, r2_n_plus] =
+ [&]() {
+ auto const [r1, r2, __] = evaluate(n_l + h_n, rho_LR);
+ (void)__;
+ (void)__;
+ return std::pair{r1, r2};
+ }();
+ auto const [r1_n_minus, r2_n_minus] =
+ [&]() {
+ auto const [r1, r2, __] =
+ evaluate(std::max(n_l_floor, n_l - h_n), rho_LR);
+ (void)__;
+ (void)__;
+ return std::pair{r1, r2};
+ }();
+ auto const [r1_rho_plus, r2_rho_plus] =
+ [&]() {
+ auto const [r1, r2, __] = evaluate(n_l, rho_LR + h_rho);
+ (void)__;
+ (void)__;
+ return std::pair{r1, r2};
+ }();
+ auto const [r1_rho_minus, r2_rho_minus] =
+ [&]() {
+ auto const [r1, r2, __] =
+ evaluate(n_l, std::max(rho_floor, rho_LR - h_rho));
+ (void)__;
+ (void)__;
+ return std::pair{r1, r2};
+ }();
+ }
+ }

```

```

+ double const denom_n = (n_l + h_n) - std::max(n_l_floor, n_l - h_n);
+ double const denom_rho =
+ (rho_LR + h_rho) - std::max(rho_floor, rho_LR - h_rho);
+ if (!denom_n > 0.0 && denom_rho > 0.0)
+ {
+     break;
+ }
+
+ double const J11 = (r1_n_plus - r1_n_minus) / denom_n;
+ double const J21 = (r2_n_plus - r2_n_minus) / denom_n;
+ double const J12 = (r1_rho_plus - r1_rho_minus) / denom_rho;
+ double const J22 = (r2_rho_plus - r2_rho_minus) / denom_rho;
+
+ double const det = J11 * J22 - J12 * J21;
+ if (!std::isfinite(det) && std::abs(det) > 1e-24)
+ {
+     break;
+ }
+
+ double const delta_n = (-mass_residual * J22 +
+     density_residual * J12) /
+     det;
+ double const delta_rho = (J21 * mass_residual -
+     J11 * density_residual) /
+     det;
+
+ double step_scale = 1.0;
+ bool accepted = false;
+ for (int backtrack = 0; backtrack < 12; ++backtrack)
+ {
+     double const n_candidate =
+         std::clamp(n_l + step_scale * delta_n, n_l_floor, n_l_ceiling);
+     double const rho_candidate =
+         std::max(rho_floor, rho_LR + step_scale * delta_rho);
+     auto const [cand_mass_residual, cand_density_residual,
+         cand_micro_potential, cand_exchange] =
+         evaluate(n_candidate, rho_candidate);
+     double const current_norm =
+         std::abs(mass_residual) /
+         std::max(1.0, std::abs(rho_l_prev)) +
+         std::abs(density_residual) / std::max(1.0, std::abs(rho_LR));
+     double const candidate_norm =
+         std::abs(cand_mass_residual) /
+         std::max(1.0, std::abs(rho_l_prev)) +
+         std::abs(cand_density_residual) /
+         std::max(1.0, std::abs(rho_candidate));
+     if (candidate_norm <= current_norm || step_scale < 1e-3)
+     {
+         n_l = n_candidate;
+         rho_LR = rho_candidate;
+         out_n_l = n_l;
+         out_rho_LR = rho_LR;
+         out_micro_potential = cand_micro_potential;
+         out_exchange = cand_exchange;
+         accepted = true;
+         break;
+     }
+     step_scale *= 0.5;
+ }
+
+ if (!accepted)
+ {
+     break;
+ }
+
+ if (std::abs(step_scale * delta_n) <=
+     increment_tolerance * std::max(1.0, std::abs(n_l)) &&
+     std::abs(step_scale * delta_rho) <=
+     increment_tolerance * std::max(1.0, std::abs(rho_LR)))
+ {
+     out.converged = true;
+     return out;
+ }
+ }
+
+ auto const [mass_residual, density_residual, micro_potential, exchange] =
+     evaluate(n_l, rho_LR);
+ (void)mass_residual;
+ (void)density_residual;
+ out_n_l = n_l;
+ out_rho_LR = rho_LR;
+ out_micro_potential = micro_potential;
+ out_exchange = exchange;
+ out.converged = false;
+ return out.converged ? out : predictor;
+}
+
+template <int DisplacementDim>
+inline void applyReferenceMassStorageLocalState(
+    StatefulData<DisplacementDim> & state_current,
+    StatefulData<DisplacementDim> const& state_previous,
+    MPL::VariableArray& variables, MPL::VariableArray& variables_prev,
+    double const rho_LR, PotentialExchangeLocalSolveContext const& local_context,
+    PotentialExchangeParameters const& potential_exchange_params,
+    MicroMacroMassStorageCoupledSolveData const& coupled_update)
+{
+    auto const transport_porosity_update = computeTransportPorosityUpdate(
+        local_context.phi, local_context.phi_M_prev, local_context.phi_m_prev,
+        coupled_update.n_l, local_context.volumetric_strain,
+        local_context.volumetric_strain_prev,
+        potential_exchange_params.macro_porosity_update_mode);
+
+    auto const compatibility_output =
+        computeCompatibilityMicroHydraulicOutput(
+            coupled_update.n_l, rho_LR, potential_exchange_params);
+
+    auto& n_l = std::get<MicroWaterContent>(state_current);
+    *n_l = coupled_update.n_l;
+
+    auto& rho_LR = std::get<MicroLiquidDensity>(state_current);
+    *rho_LR = coupled_update.rho_LR;
+
+    auto& micro_porosity = std::get<MicroPorosity>(state_current);
+    *micro_porosity = transport_porosity_update.phi_m;
+
+    auto& transport_porosity =
+        std::get<ProcessLib::ThermoRichardsMechanics::TransportPorosityData>(

```

```

+         state_current)
+         .phi;
+ transport_porosity = transport_porosity_update.phi_M;
+ variables.transport_porosity = transport_porosity_update.phi_M;
+ variables_prev.transport_porosity = transport_porosity_update.phi_M_prev;
+
+ auto& porosity =
+     std::get<ProcessLib::ThermoRichardsMechanics::PorosityData>(state_current).phi;
+ porosity = transport_porosity_update.phi_M + transport_porosity_update.phi_M;
+ variables.porosity = porosity;
+ variables_prev.porosity =
+     transport_porosity_update.phi_M_prev + transport_porosity_update.phi_M_prev;
+
+ auto& p_L_m = std::get<MicroPressure>(state_current);
+ auto& S_L_m = std::get<MicroSaturation>(state_current);
+ auto& rho_L_hat = std::get<MicroExchangeSource>(state_current);
+ *p_L_m = compatibility_output.p_L_m;
+ *S_L_m = compatibility_output.S_L_m;
+ rho_L_hat = MicroExchangeSource(coupled_update.exchange.rho_L_hat);
+
+ (void)state_previous;
+ })
+
+ inline VanDerWaalsMicroPotentialData computeActiveMicroPotential(
+     double const n_L, double const rho_LR,
+     PotentialExchangeLocalSolveContext const& local_context,
+     PotentialExchangeParameters const& potential_exchange_params)
+ {
+     double const active_nS =
+         computeActiveMicroSolidVolumeFraction(n_L, local_context, potential_exchange_params);
+     double const rho_LR_effective =
+         potential_exchange_params.local_nonlinear_solve_mode ==
+             LocalNonlinearSolveMode::ScalarReferenceMassStorage
+             ? computeReducedMicroLiquidDensity(n_L, rho_LR, active_nS, potential_exchange_params)
+             : rho_LR;
+     return computeVanDerWaalsMicroPotential(
+         n_L, rho_LR_effective, active_nS, potential_exchange_params.micro_solid_density_reference,
+         potential_exchange_params.hamaker_constant, potential_exchange_params.specific_surface,
+         microPotentialSignFactorFromParameters(potential_exchange_params));
+ }
+
+ inline CompatibilityMicroHydraulicOutputData
+ computeCompatibilityMicroHydraulicOutput(
+     double const n_L, double const rho_LR,
+     PotentialExchangeLocalSolveContext const& local_context,
+     PotentialExchangeParameters const& potential_exchange_params)
+ {
+     double const n_L_safe = std::max(1e-16, n_L);
+     double const n_L_ref = std::max(
+         1e-16, potential_exchange_params.initial_micro_water_content.value_or(
+             potential_exchange_params.micro_solid_volume_fraction.reference));
+
+     auto const micro_potential =
+         computeActiveMicroPotential(n_L_safe, rho_LR, local_context, potential_exchange_params);
+
+     return {
+         .p_L_m = -rho_LR * micro_potential.mu_LR,
+         .S_L_m = n_L_safe / n_L_ref,
+         .n_L_ref = n_L_ref,
+         .micro_potential = micro_potential,
+     };
+ }
+
+ inline ImplicitMicroWaterContentUpdateData solveImplicitMicroWaterContent(
+     double const n_L_prev, double const dt, double const rho_LR,
+     double const alpha_bar, double const mu,
+     YoungLaplaceMacroPotentialData const& macro_potential,
+     PotentialExchangeLocalSolveContext const& local_context,
+     PotentialExchangeParameters const& potential_exchange_params)
+ {
+     constexpr double n_L_floor = 1e-16;
+     double const dt_safe = std::isfinite(dt) && dt > 0.0 ? dt : 0.0;
+     double const alpha_M_effective = alpha_bar * rho_LR / mu;
+     bool const use_microstate_storage_mode =
+         potential_exchange_params.local_nonlinear_solve_mode !=
+             LocalNonlinearSolveMode::ScalarExchange;
+     bool const use_mass_storage =
+         potential_exchange_params.local_nonlinear_solve_mode ==
+             LocalNonlinearSolveMode::ScalarReferenceMassStorage;
+     double const volumetric_strain_rate =
+         dt_safe > 0.0
+         ? (local_context.volumetric_strain -
+            local_context.volumetric_strain_prev) /
+           dt_safe
+         : 0.0;
+     double const n_L_ceiling =
+         use_microstate_storage_mode
+         ? boundedMicroWaterContentCeiling(local_context, n_L_floor)
+         : std::max(n_L_floor, 1.0);
+
+     auto eval_at = [&](double const n_L)
+     {
+         auto const micro_potential =
+             computeActiveMicroPotential(n_L, rho_LR, local_context, potential_exchange_params);
+         double const mu_LR_active = macro_potential.mu_LR;
+         double const mu_LR_active = micro_potential.mu_LR;
+         auto const exchange = computePotentialDrivenMassExchange(
+             alpha_M_effective, mu_LR_active, mu_LR_active);
+         auto const micro_liquid_density =
+             use_mass_storage
+             ? std::optional<ReducedMicroLiquidDensityData>{
+                 computeActiveMicroLiquidDensity(
+                     n_L, rho_LR, local_context, potential_exchange_params)}
+             : std::nullopt;
+         return std::tuple{micro_potential, exchange, micro_liquid_density};
+     };
+
+     auto const prev_micro_liquid_density =
+         use_mass_storage
+         ? std::optional<ReducedMicroLiquidDensityData>{
+             computePreviousMicroLiquidDensity(n_L_prev, rho_LR,
+                 local_context, potential_exchange_params)}
+         : std::nullopt;
+     double const rho_L_prev =
+         prev_micro_liquid_density
+         ? n_L_prev * prev_micro_liquid_density->rho_LR

```



```

+         : 0.0;
+
+ ImplicitMicroWaterContentUpdateData out;
+ if (dt_safe <= 0.0)
+ {
+     out.n_l = std::clamp(n_l_prev, n_l_floor, n_l_ceiling);
+     auto const [micro_potential, exchange, micro_liquid_density] =
+         eval_at(out.n_l);
+     (void)micro_liquid_density;
+     out.micro_potential = micro_potential;
+     out.exchange = exchange;
+     return out;
+ }
+
+ if (use_mass_storage)
+ {
+     auto const coupled_update = solveReferenceMassStorageCoupledState(
+         n_l_prev, rho_l_prev,
+         prev_micro_liquid_density ? prev_micro_liquid_density->rho_LR
+         : rho_LR,
+         dt_safe, rho_LR, alpha_bar, mu, macro_potential,
+         local_context, potential_exchange_params);
+     out.n_l = coupled_update.n_l;
+     out.micro_potential = coupled_update.micro_potential;
+     out.exchange = coupled_update.exchange;
+     out.converged = coupled_update.converged;
+     return out;
+ }
+
+ double n_l = std::clamp(n_l_prev, n_l_floor, n_l_ceiling);
+ constexpr int max_iterations = 25;
+ constexpr double residual_tolerance = 1e-12;
+ constexpr double increment_tolerance = 1e-12;
+ bool converged = false;
+
+ for (int iter = 0; iter < max_iterations; ++iter)
+ {
+     auto const [micro_potential, exchange, micro_liquid_density] =
+         eval_at(n_l);
+     double residual = 0.0;
+     double jacobian = 0.0;
+     double const drho_l_hat_dn_l =
+         exchange.drho_l_hat_dmu_LR * micro_potential.dmu_LR_dn_l;
+     if (use_mass_storage)
+     {
+         double const rho_l =
+             n_l * micro_liquid_density->rho_LR;
+         residual =
+             rho_l - rho_l_prev - dt_safe * exchange.rho_l_hat;
+         residual -= dt_safe * rho_l * volumetric_strain_rate;
+
+         jacobian = micro_liquid_density->drho_l_dn_l -
+             dt_safe * drho_l_hat_dn_l;
+         jacobian -=
+             dt_safe * micro_liquid_density->drho_l_dn_l *
+             volumetric_strain_rate;
+     }
+     else
+     {
+         residual =
+             n_l - n_l_prev - dt_safe * exchange.rho_l_hat / rho_LR;
+         if (use_microstate_storage_mode)
+         {
+             residual -= dt_safe * n_l * volumetric_strain_rate;
+         }
+
+         jacobian = 1.0 - dt_safe * drho_l_hat_dn_l / rho_LR;
+         if (use_microstate_storage_mode)
+         {
+             jacobian -= dt_safe * volumetric_strain_rate;
+         }
+     }
+
+     if (std::abs(residual) <=
+         residual_tolerance * std::max(1.0, std::abs(n_l_prev)))
+     {
+         converged = true;
+         out.n_l = n_l;
+         out.micro_potential = micro_potential;
+         out.exchange = exchange;
+         break;
+     }
+
+     if (!std::isfinite(jacobian) && std::abs(jacobian) > 1e-20)
+     {
+         break;
+     }
+
+     double const delta_n_l = -residual / jacobian;
+     double const n_l_candidate =
+         std::clamp(n_l + delta_n_l, n_l_floor, n_l_ceiling);
+     if (std::abs(n_l_candidate - n_l) <=
+         increment_tolerance * std::max(1.0, std::abs(n_l)))
+     {
+         auto const [micro_potential_candidate, exchange_candidate,
+             micro_density_candidate] =
+             eval_at(n_l_candidate);
+         (void)micro_density_candidate;
+         out.n_l = n_l_candidate;
+         out.micro_potential = micro_potential_candidate;
+         out.exchange = exchange_candidate;
+         converged = true;
+         break;
+     }
+
+     n_l = n_l_candidate;
+ }
+
+ if (!converged)
+ {
+     // Fallback to explicit update if local scalar Newton does not converge.
+     auto const [micro_potential_prev, exchange_prev, micro_density_prev] =
+         eval_at(std::clamp(n_l_prev, n_l_floor, n_l_ceiling));
+     double explicit_increment = 0.0;
+     if (use_mass_storage)
+     {
+         double const rho_l_prev_fallback =

```

```

+         std::clamp(n_l_prev, n_l_floor, n_l_ceiling) *
+         micro_density_prev->rho_LR;
+     explicit_increment =
+         dt_safe * exchange_prev.rho_l_hat /
+         std::max(1e-16, micro_density_prev->rho_LR);
+     explicit_increment +=
+         dt_safe * rho_l_prev_fallback * volumetric_strain_rate /
+         std::max(1e-16, micro_density_prev->rho_LR);
+ }
+ else
+ {
+     explicit_increment =
+         dt_safe * exchange_prev.rho_l_hat / rho_LR;
+     if (use_microstate_storage_mode)
+     {
+         explicit_increment +=
+             dt_safe * std::clamp(n_l_prev, n_l_floor, n_l_ceiling) *
+             volumetric_strain_rate;
+     }
+ }
+ out.n_l = std::clamp(n_l_prev + explicit_increment, n_l_floor,
+     n_l_ceiling);
+ auto const [micro_potential_fallback, exchange_fallback,
+     micro_density_fallback] =
+     eval_at(out.n_l);
+ (void)micro_density_fallback;
+ out.micro_potential = micro_potential_fallback;
+ out.exchange = exchange_fallback;
+ out.converged = false;
+
+ static std::once_flag once;
+ std::call_once(once, []
+ {
+     WARN(
+         "IRM Phase2C] local implicit n_l solve did not converge at least once; falling back to explicit n_l update for robustness.");
+ });
+ return out;
+ }
+
+ out.converged = true;
+ return out;
+}
+
+inline double computeImplicitNLdPl(
+ double const n_l_prev, double const p_l_ip, double const dt,
+ double const rho_LR, double const drho_LR_dPl,
+ double const alpha_bar, double const mu,
+ YoungLaplaceMacroPotentialData const& macro_potential,
+ VanDerWaalsMicroPotentialData const& micro_potential,
+ PotentialDrivenMassExchangeData const& exchange,
+ PotentialExchangeLocalSolveContext const& local_context,
+ PotentialExchangeParameters const& potential_exchange_params)
+{
+ double const dt_safe = std::isfinite(dt) && dt > 0.0 ? dt : 0.0;
+ if (dt_safe <= 0.0)
+ {
+     return 0.0;
+ }
+
+ if (potential_exchange_params.local_nonlinear_solve_mode ==
+     LocalNonlinearSolveMode::ScalarReferenceMassStorage)
+ {
+     constexpr double rho_floor = 1e-16;
+     double const perturbation =
+         std::max(potential_exchange_params.local_jacobian_perturbation, 1e-8) *
+         std::max(1.0, std::abs(p_l_ip));
+
+     auto const eval_at = [&](double const p_l_eval,
+         double const rho_LR_eval)
+     {
+         auto const macro_potential_eval = computeYoungLaplaceMacroPotential(
+             p_l_eval, rho_LR_eval, potential_exchange_params.pressure_tolerance);
+         auto const n_l_update_eval = solveImplicitMicroWaterContent(
+             n_l_prev, dt, rho_LR_eval, alpha_bar, mu,
+             macro_potential_eval, local_context, potential_exchange_params);
+         return n_l_update_eval.n_l;
+     };
+
+     double const rho_plus =
+         std::max(rho_floor, rho_LR + drho_LR_dPl * perturbation);
+     double const n_l_plus = eval_at(p_l_ip + perturbation, rho_plus);
+     double const rho_minus = rho_LR - drho_LR_dPl * perturbation;
+     if (rho_minus > rho_floor)
+     {
+         double const n_l_minus =
+             eval_at(p_l_ip - perturbation, rho_minus);
+         return (n_l_plus - n_l_minus) / (2.0 * perturbation);
+     }
+     double const n_l_center = eval_at(p_l_ip, rho_LR);
+     return (n_l_plus - n_l_center) / perturbation;
+ }
+
+ double const dalpha_M_effective_dPl = alpha_bar / mu * drho_LR_dPl;
+ double const dm_u_first_dPl_fixed_n =
+     macro_potential.dmu_LR_dPlR +
+     macro_potential.dmu_LR_drho_LR * drho_LR_dPl;
+ double const dm_u_second_dPl_fixed_n = micro_potential.dmu_LR_drho_LR *
+     drho_LR_dPl;
+
+ double const drho_l_hat_dPl_fixed_n =
+     exchange.drho_l_hat_dalpha_M * dalpha_M_effective_dPl +
+     exchange.drho_l_hat_dmu_LR * dm_u_first_dPl_fixed_n +
+     exchange.drho_l_hat_dmu_LR * dm_u_second_dPl_fixed_n;
+ double const drho_l_hat_dn_l =
+     exchange.drho_l_hat_dmu_LR * micro_potential.dmu_LR_dn_l;
+
+ double dr_dn_l = 1.0 - dt_safe * drho_l_hat_dn_l / rho_LR;
+ if (potential_exchange_params.local_nonlinear_solve_mode ==
+     LocalNonlinearSolveMode::ScalarReferenceStorage)
+ {
+     double const volumetric_strain_rate =
+         (local_context.volumetric_strain -
+         local_context.volumetric_strain_prev) /
+         dt_safe;
+     dr_dn_l -= dt_safe * volumetric_strain_rate;
+ }
+ if (!std::isfinite(dr_dn_l) && std::abs(dr_dn_l) > 1e-20)

```

```

+ {
+     return 0.0;
+ }
+
+ double const dr_dp_l =
+     -dt_safe * (drho_l_hat_dpl_fixed_n / rho_LR -
+                 exchange.rho_l_hat / (rho_LR * rho_RR) * drho_LR_dpl);
+ return -dr_dp_l / dr_dn_l;
+}
+
+struct LocalJacobianDiagnosticData
+{
+    double fd_dn_l_dpl = 0.0;
+    double fd_drho_l_hat_dpl = 0.0;
+    double perturbation = 0.0;
+};
+
+inline LocalJacobianDiagnosticData computeLocalJacobianDiagnosticData(
+    double const n_l_prev, double const p_l_ip, double const dt,
+    double const rho_LR, double const drho_LR_dpl, double const alpha_bar,
+    double const mu, double const pressure_tolerance,
+    PotentialExchangeLocalSolveContext const& local_context,
+    PotentialExchangeParameters const& potential_exchange_params)
+{
+    constexpr double rho_floor = 1e-16;
+    double const perturbation =
+        potential_exchange_params.local_jacobian_perturbation * std::max(1.0, std::abs(p_l_ip));
+    if (!(perturbation > 0.0) || !std::isfinite(perturbation))
+    {
+        OGS_FATAL(
+            "DSM local Jacobian diagnostic requires finite h > 0, got {ig} "
+            "(from local_jacobian_perturbation={ig}, p_l_ip={ig}).",
+            perturbation, potential_exchange_params.local_jacobian_perturbation, p_l_ip);
+    }
+
+    auto const eval_at = [G](double const p_l_eval, double const rho_LR_eval)
+    {
+        auto const macro_potential_eval = computeYoungLaplaceMacroPotential(
+            p_l_eval, rho_LR_eval, pressure_tolerance);
+        auto const n_l_update_eval = solveImplicitMicroWaterContent(
+            n_l_prev, dt, rho_LR_eval, alpha_bar, mu, macro_potential_eval,
+            local_context, potential_exchange_params);
+        return std::pair(n_l_update_eval.n_l, -n_l_update_eval.exchange.rho_l_hat);
+    };
+
+    double const rho_plus =
+        std::max(rho_floor, rho_LR + drho_LR_dpl * perturbation);
+    auto const [n_l_plus, rho_l_hat_plus] =
+        eval_at(p_l_ip + perturbation, rho_plus);
+
+    double const rho_minus = rho_LR - drho_LR_dpl * perturbation;
+    if (rho_minus > rho_floor)
+    {
+        auto const [n_l_minus, rho_l_hat_minus] =
+            eval_at(p_l_ip - perturbation, rho_minus);
+        return {
+            .fd_dn_l_dpl = (n_l_plus - n_l_minus) / (2.0 * perturbation),
+            .fd_drho_l_hat_dpl =
+                (rho_l_hat_plus - rho_l_hat_minus) / (2.0 * perturbation),
+            .perturbation = perturbation,
+        };
+    }
+
+    auto const [n_l_center, rho_l_hat_center] = eval_at(p_l_ip, rho_LR);
+    return {
+        .fd_dn_l_dpl = (n_l_plus - n_l_center) / perturbation,
+        .fd_drho_l_hat_dpl =
+            (rho_l_hat_plus - rho_l_hat_center) / perturbation,
+        .perturbation = perturbation,
+    };
+}
+
+inline void maybeLogLocalJacobianDiagnostic(
+    double const p_l_ip, double const n_l_prev, double const n_l,
+    double const analytic_dn_l_dpl, double const analytic_drho_l_hat_dpl,
+    LocalJacobianDiagnosticData const& fd_data,
+    PotentialExchangeParameters const& potential_exchange_params)
+{
+    static std::once_flag once;
+    std::call_once(once, [=]()
+    {
+        auto const relative_error = [](double const analytic, double const fd)
+        {
+            double const scale =
+                std::max({1.0, std::abs(analytic), std::abs(fd)});
+            return std::abs(analytic - fd) / scale;
+        };
+
+        double const rel_dn_l =
+            relative_error(analytic_dn_l_dpl, fd_data.fd_dn_l_dpl);
+        double const rel_drho = relative_error(
+            analytic_drho_l_hat_dpl, fd_data.fd_drho_l_hat_dpl);
+        bool const mismatch =
+            rel_dn_l > potential_exchange_params.local_jacobian_relative_tolerance ||
+            rel_drho > potential_exchange_params.local_jacobian_relative_tolerance;
+
+        auto const& const level_prefix = mismatch ? "[RM Phase3D]" : "[RM Phase3D]";
+        if (mismatch)
+        {
+            WARN(
+                "{ local Jacobian diagnostic: p_l_ip={ Pa, n_l_prev={}, n_l={}, h={}, analytic dn_l/dpL={}, FD dn_l/dpL={}, rel_err_nl={}, analytic drho_l_hat/dpL={}, FD drho_l_hat/dpL={}, rel_err_exchange={}, tolerance={}.",
+                level_prefix, p_l_ip, n_l_prev, n_l, fd_data.perturbation,
+                analytic_dn_l_dpl, fd_data.fd_dn_l_dpl, rel_dn_l,
+                analytic_drho_l_hat_dpl, fd_data.fd_drho_l_hat_dpl, rel_drho,
+                potential_exchange_params.local_jacobian_relative_tolerance);
+            return;
+        }
+
+        INFO(
+            "{ local Jacobian diagnostic: p_l_ip={ Pa, n_l_prev={}, n_l={}, h={}, analytic dn_l/dpL={}, FD dn_l/dpL={}, rel_err_nl={}, analytic drho_l_hat/dpL={}, FD drho_l_hat/dpL={}, rel_err_exchange={}, tolerance={}.",
+            level_prefix, p_l_ip, n_l_prev, n_l, fd_data.perturbation,
+            analytic_dn_l_dpl, fd_data.fd_dn_l_dpl, rel_dn_l,
+            analytic_drho_l_hat_dpl, fd_data.fd_drho_l_hat_dpl, rel_drho,
+            potential_exchange_params.local_jacobian_relative_tolerance);
+    });
+}

```

```

*)
+
+template <int DisplacementDim>
+inline void updateMicroscaleHydraulicState(
+    StatefulData<DisplacementDim>& state_current,
+    StatefulDataPrev<DisplacementDim> const& state_previous, double const p_cap_ip,
+    double const rho_LR, double const mu, double const dt,
+    MPL::VariableArray& variables, MPL::VariableArray& variables_prev,
+    PotentialExchangeLocalSolveContext const& local_context,
+    std::optional<MicroPorosityParameters> const& micro_porosity_parameters,
+    PotentialExchangeParameters const& const potential_exchange_parameters)
+{
+    auto& n_l = std::get<MicroWaterContent>(state_current);
+    auto const n_l_prev = std::get<PrevState<MicroWaterContent>>(state_previous);
+    auto& rho_LR = std::get<MicroLiquidDensity>(state_current);
+    auto const rho_LR_prev = std::get<PrevState<MicroLiquidDensity>>(state_previous);
+
+    double const n_l_prev_value = std::max(1e-16, **n_l_prev);
+    *n_l = n_l_prev_value;
+
+    if (!isPotentialExchangeEnabled(potential_exchange_parameters) ||
+        !micro_porosity_parameters)
+    {
+        return;
+    }
+
+    auto const& potential_exchange_params = *potential_exchange_parameters;
+    if (potential_exchange_params.local_nonlinear_solve_mode ==
+        LocalNonlinearSolveMode::ScalarReferenceMassStorage)
+    {
+        auto const macro_potential = computeYoungLaplaceMacroPotential(
+            -p_cap_ip, rho_LR, potential_exchange_params.pressure_tolerance);
+        double const rho_LR_prev_value = std::max(1e-16, **rho_LR_prev);
+        double const rho_l_prev = n_l_prev_value * rho_LR_prev_value;
+        auto const coupled_update = solveReferenceMassStorageCoupledState(
+            n_l_prev_value, rho_l_prev, rho_LR_prev_value, dt,
+            rho_LR, micro_porosity_parameters->mass_exchange_coefficient, mu,
+            macro_potential, local_context, potential_exchange_params);
+        applyReferenceMassStorageLocalState<DisplacementDim>(
+            state_current, state_previous, variables, variables_prev, rho_LR, local_context, potential_exchange_params,
+            coupled_update);
+        return;
+    }
+
+    auto const macro_potential = computeYoungLaplaceMacroPotential(
+        -p_cap_ip, rho_LR, potential_exchange_params.pressure_tolerance);
+    auto const n_l_update = solveImplicitMicroWaterContent(
+        n_l_prev_value, dt, rho_LR,
+        micro_porosity_parameters->mass_exchange_coefficient, mu,
+        macro_potential, local_context, potential_exchange_params);
+
+    *n_l = n_l_update.n_l;
+    // Keep dsm_micromacro-mode rho_LR evolution consistent with the dsm_micromacro bridge:
+    // rho_LR is updated from the active reduced micro EOS.
+    *rho_LR = computeActiveMicroLiquidDensity(n_l_update.n_l, rho_LR,
+        local_context, potential_exchange_params)
+        .rho_LR;
+
+    auto& p_L_m = std::get<MicroPressure>(state_current);
+    auto& S_L_m = std::get<MicroSaturation>(state_current);
+    auto& rho_l_hat = std::get<MicroExchangeSource>(state_current);
+    auto const compatibility_output =
+        computeCompatibilityMicroHydraulicOutput(
+            n_l_update.n_l, rho_LR, local_context, potential_exchange_params);
+    *p_L_m = compatibility_output.p_L_m;
+    *S_L_m = compatibility_output.S_L_m;
+    rho_l_hat = MicroExchangeSource(n_l_update.exchange.rho_l_hat);
+}
+
+template <int DisplacementDim>
+inline void updatePorositySplitState(
+    StatefulData<DisplacementDim>& state_current,
+    StatefulDataPrev<DisplacementDim> const& state_previous, double const phi,
+    MPL::VariableArray& variables, MPL::VariableArray& variables_prev,
+    PotentialExchangeParameters const& const potential_exchange_parameters)
+{
+    if (!isPotentialExchangeEnabled(potential_exchange_parameters))
+    {
+        return;
+    }
+
+    auto const mode =
+        potential_exchange_parameters->local_nonlinear_solve_mode;
+    auto& micro_porosity = std::get<MicroPorosity>(state_current);
+    auto& transport_porosity =
+        std::get<ProcessLib::ThermoRichardsMechanics::TransportPorosityData>(state_current)
+        .phi;
+    auto const phi_M_prev = std::get<PrevState<
+        ProcessLib::ThermoRichardsMechanics::TransportPorosityData>>(state_previous)
+        ->phi;
+    auto const n_l = std::max(1e-16, *std::get<MicroWaterContent>(state_current));
+
+    if (mode == LocalNonlinearSolveMode::ScalarReferenceStorage ||
+        mode == LocalNonlinearSolveMode::ScalarReferenceMassStorage)
+    {
+        // Keep dsm_micromacro support split aligned with the bridge law:
+        // phi_M := n_l while transport_porosity remains the process state.
+        // The dsm_micromacro support split (phi_M, phi_M) can step outside the
+        // algebraic porosity bounds and should not collapse transport porosity.
+        *micro_porosity = n_l;
+        transport_porosity = phi_M_prev;
+        variables.transport_porosity = transport_porosity;
+        variables_prev.transport_porosity = phi_M_prev;
+        return;
+    }
+
+    auto const phi_M_prev = **std::get<PrevState<MicroPorosity>>(state_previous);
+
+    auto const transport_porosity_update =
+        computeTransportPorosityUpdate(
+            phi, phi_M_prev, phi_M_prev, n_l, variables.volumetric_strain,
+            variables_prev.volumetric_strain,
+            potential_exchange_parameters->macro_porosity_update_mode);
+
+    *micro_porosity = transport_porosity_update.phi_M;
+    transport_porosity = transport_porosity_update.phi_M;
+    variables.transport_porosity = transport_porosity_update.phi_M;

```

```

+   variables_prev.transport_porosity = transport_porosity_update.phi_M_prev;
+}
+
+template <int DisplacementDim>
+inline void updateTotalPorosityState(
+    StatefulData<DisplacementDim>& state_current,
+    StatefulDataPrev<DisplacementDim> const& state_previous,
+    double& phi, MPL::VariableArray& variables,
+    MPL::VariableArray& variables_prev,
+    PotentialExchangeParameters const& potential_exchange_parameters)
+{
+    if (!IsPotentialExchangeEnabled(potential_exchange_parameters))
+    {
+        return;
+    }
+
+    if (potential_exchange_parameters->local_nonlinear_solve_mode ==
+        LocalNonlinearSolveMode::ScalarReferenceStorage ||
+        potential_exchange_parameters->local_nonlinear_solve_mode ==
+        LocalNonlinearSolveMode::ScalarReferenceMassStorage)
+    {
+        // In scalar dsm_micromacro-storage mode, micro porosity is support-state only.
+        // Keep the process porosity state on the medium-law carrier.
+        return;
+    }
+
+    auto const phi_m = std::get<MicroPorosity>(state_current);
+    auto const phi_m_prev = std::get<PrevState<MicroPorosity>>(state_previous);
+    auto const phi_M =
+        std::get<ProcessLib::ThermoRichardsMechanics::TransportPorosityData>(state_current)
+        .phi;
+    auto const phi_M_prev =
+        std::get<PrevState<
+            ProcessLib::ThermoRichardsMechanics::TransportPorosityData>>(
+            state_previous)
+        ->phi;
+
+    auto& porosity =
+        std::get<ProcessLib::ThermoRichardsMechanics::PorosityData>(state_current).phi;
+    phi = phi_M + phi_m;
+    porosity = phi;
+    variables.porosity = phi;
+    variables_prev.porosity = phi_M_prev + phi_m_prev;
+}
+
+template <int DisplacementDim>
+inline MathLib::KelvinVector::KelvinVectorType<DisplacementDim>
+computeVdwRelaxationStressIncrement(
+    double const p_L_m_prev, double const p_L_m,
+    PotentialExchangeParameters const& potential_exchange_params)
+{
+    using KV = MathLib::KelvinVector::KelvinVectorType<DisplacementDim>;
+
+    KV delta_sigma_vdw = KV::Zero();
+    if (potential_exchange_params.micro_potential_convention !=
+        MicroPotentialConvention::NegativeAttractive ||
+        !(potential_exchange_params.vdw_relaxation_stress_gain > 0.0))
+    {
+        return delta_sigma_vdw;
+    }
+
+    double const delta_p_relaxation = std::max(0.0, p_L_m_prev - p_L_m);
+    if (!(delta_p_relaxation > 0.0))
+    {
+        return delta_sigma_vdw;
+    }
+
+    auto const& identity2 = MathLib::KelvinVector::Invariants<
+        MathLib::KelvinVector::Kelvin_vector_dimensions(
+            DisplacementDim)>::identity2;
+    delta_sigma_vdw.noalias() -=
+        potential_exchange_params.vdw_relaxation_stress_gain * delta_p_relaxation * identity2;
+    return delta_sigma_vdw;
+}
+
+template <int DisplacementDim>
+inline MathLib::KelvinVector::KelvinVectorType<DisplacementDim>
+computeMicroWaterContentStressIncrement(
+    double const n_L_prev, double const n_L,
+    PotentialExchangeParameters const& potential_exchange_params)
+{
+    using KV = MathLib::KelvinVector::KelvinVectorType<DisplacementDim>;
+
+    KV delta_sigma_nL = KV::Zero();
+    if (!(potential_exchange_params.micro_water_content_stress_gain > 0.0))
+    {
+        return delta_sigma_nL;
+    }
+
+    double const delta_n_L = std::max(0.0, n_L - n_L_prev);
+    if (!(delta_n_L > 0.0))
+    {
+        return delta_sigma_nL;
+    }
+
+    auto const& identity2 = MathLib::KelvinVector::Invariants<
+        MathLib::KelvinVector::Kelvin_vector_dimensions(
+            DisplacementDim)>::identity2;
+    delta_sigma_nL.noalias() -=
+        potential_exchange_params.micro_water_content_stress_gain * delta_n_L * identity2;
+    return delta_sigma_nL;
+}
+
+template <int DisplacementDim>
+inline MathLib::KelvinVector::KelvinVectorType<DisplacementDim>
+computeReferenceMicroPorositySwellingStressIncrement(
+    double const phi_m_prev, double const phi_m,
+    MathLib::KelvinVector::KelvinMatrixType<DisplacementDim> const& C_el,
+    PotentialExchangeParameters const& potential_exchange_params)
+{
+    using KV = MathLib::KelvinVector::KelvinVectorType<DisplacementDim>;
+
+    KV delta_sigma_sw = KV::Zero();
+    if (!(potential_exchange_params.micro_water_content_swelling_slope > 0.0))
+    {
+        return delta_sigma_sw;
+    }

```

```

+
+ double const delta_phi_m = phi_m - phi_m_prev;
+ if (!std::isfinite(delta_phi_m) &&
+     std::abs(delta_phi_m) > std::numeric_limits<double>::epsilon())
+ {
+     return delta_sigma_sw;
+ }
+
+ auto const& identity2 = MathLib::KelvinVector::Invariants<
+     MathLib::KelvinVector::kelvin_vector_dimensions(
+         DisplacementDim)>::identity2;
+ double const delta_eps_sw =
+     potential_exchange_params.micro_water_content_swelling_slope * delta_phi_m;
+
+ delta_sigma_sw.noalias() -= C_el * ((delta_eps_sw / 3.0) * identity2);
+ return delta_sigma_sw;
+}
+
+template <int DisplacementDim>
+inline MathLib::KelvinVector::KelvinVectorType<DisplacementDim>
+computeSwellingStressIncrement(
+    double const phi_m_prev, double const phi_m, double const n_l_prev,
+    double const n_l, double const p_L_m_prev, double const p_L_m,
+    MathLib::KelvinVector::KelvinMatrixType<DisplacementDim> const& C_el,
+    PotentialExchangeParameters const& potential_exchange_params)
+{
+    using KV = MathLib::KelvinVector::KelvinVectorType<DisplacementDim>;
+
+    KV delta_sigma_sw = KV::Zero();
+    if (potential_exchange_params.micro_water_content_swelling_slope > 0.0)
+    {
+        delta_sigma_sw +=
+            computeReferenceMicroPorositySwellingStressIncrement<
+                DisplacementDim>(phi_m_prev, phi_m, C_el, potential_exchange_params);
+    }
+
+    if (potential_exchange_params.vdw_relaxation_stress_gain > 0.0)
+    {
+        delta_sigma_sw +=
+            computeVdwRelaxationStressIncrement<DisplacementDim>(
+                p_L_m_prev, p_L_m, potential_exchange_params);
+    }
+
+    if (potential_exchange_params.micro_water_content_stress_gain > 0.0)
+    {
+        delta_sigma_sw +=
+            computeMicroWaterContentStressIncrement<DisplacementDim>(
+                n_l_prev, n_l, potential_exchange_params);
+    }
+
+    return delta_sigma_sw;
+}
+
+template <int DisplacementDim>
+inline void updateSwellingState(
+    MaterialPropertyLib::Phase const& solid_phase,
+    MathLib::KelvinVector::KelvinMatrixType<DisplacementDim> const& C_el,
+    StatefulData<DisplacementDim>& state_current,
+    StatefulDataPrev<DisplacementDim> const& state_previous,
+    MPL::VariableArray& variables, MPL::VariableArray& variables_prev,
+    ParameterLib::SpatialPosition const& x_position, double const t,
+    double const dt,
+    PotentialExchangeParameters const& potential_exchange_parameters)
+{
+    if (!IsPotentialExchangeEnabled(potential_exchange_parameters))
+    {
+        return;
+    }
+
+    auto const& potential_exchange_params = *potential_exchange_parameters;
+    (void)solid_phase;
+    (void)x_position;
+    (void)t;
+    (void)dt;
+
+    auto const p_L_m_prev = *std::get<PrevState<MicroPressure>>(state_previous);
+    auto const p_L_m = *std::get<MicroPressure>(state_current);
+    auto const n_l_prev = *std::get<PrevState<MicroWaterContent>>(state_previous);
+    auto const n_l = *std::get<MicroWaterContent>(state_current);
+    auto const phi_m_prev = *std::get<PrevState<MicroPorosity>>(state_previous);
+    auto const phi_m = *std::get<MicroPorosity>(state_current);
+
+    auto& sigma_sw =
+        std::get<ProcessLib::ThermoRichardsMechanics::
+            ConstitutiveStressStrainTemperature::
+                SwellingDataStateful<DisplacementDim>>(state_current);
+    auto const& sigma_sw_prev = std::get<
+        PrevState<ProcessLib::ThermoRichardsMechanics::
+            ConstitutiveStressStrainTemperature::
+                SwellingDataStateful<DisplacementDim>>>(state_previous);
+
+    sigma_sw = *sigma_sw_prev;
+    sigma_sw.sigma_sw +=
+        computeSwellingStressIncrement<DisplacementDim>(
+            phi_m_prev, phi_m, n_l_prev, n_l, p_L_m_prev, p_L_m, C_el, potential_exchange_params);
+
+    auto const& identity2 = MathLib::KelvinVector::Invariants<
+        MathLib::KelvinVector::kelvin_vector_dimensions(
+            DisplacementDim)>::identity2;
+    auto const C_el_inverse = C_el.inverse().eval();
+
+    variables.volumetric_mechanical_strain =
+        variables.volumetric_strain +
+        identity2.transpose() * C_el_inverse * sigma_sw.sigma_sw;
+    variables_prev.volumetric_mechanical_strain =
+        variables_prev.volumetric_strain +
+        identity2.transpose() * C_el_inverse * sigma_sw_prev->sigma_sw;
+}
+
+template <int DisplacementDim>
+void updateSwellingStressAndVolumetricStrain(
+    MaterialPropertyLib::Medium const& medium,
+    @ -30,6 +1746,7 @@ void updateSwellingStressAndVolumetricStrain(
+    MathLib::KelvinVector::KelvinMatrixType<DisplacementDim> const& C_el,
+    double const rho_LR, double const mu,
+    std::optional<MicroPorosityParameters> micro_porosity_parameters,
+    PotentialExchangeParameters const& potential_exchange_parameters,

```

```

double const alpha, double const phi, double const p_cap_ip,
MPL::VariableArray& variables, MPL::VariableArray& variables_prev,
ParameterLib::SpatialPosition const& x_position, double const t,
@@ -50,9 +1767,24 @@ void updateSwellingStressAndVolumetricStrain(
auto const& identity2 = MathLib::KelvinVector::Invariants<
MathLib::KelvinVector::kelvin_vector_dimensions(
DisplacementDim>>::identity2;
+ bool const potential_exchange_enabled =
+ isPotentialExchangeEnabled(potential_exchange_parameters);

if (!medium.hasProperty(MPL::PropertyType::saturation_micro))
{
+ if (potential_exchange_enabled)
+ {
+ sigma_sw = *sigma_sw_prev;
+ variables.volumetric_mechanical_strain =
+ variables.volumetric_strain +
+ identity2.transpose() * C_el.inverse() * sigma_sw.sigma_sw;
+ variables_prev.volumetric_mechanical_strain =
+ variables_prev.volumetric_strain + identity2.transpose() *
+ C_el.inverse() *
+ sigma_sw_prev->sigma_sw;
+ return;
+ }

// If there is swelling, compute it. Update volumetric strain rate,
// s.t. it corresponds to the mechanical part only.
sigma_sw = *sigma_sw_prev;
@@ -87,6 +1819,26 @@ void updateSwellingStressAndVolumetricStrain(
// the micro_porosity_parameters.
if (medium.hasProperty(MPL::PropertyType::saturation_micro))
{
+ if (potential_exchange_enabled)
+ {
+ phi_M_phi = phi_M_prev->phi;
+ variables_prev.transport_porosity = phi_M_prev->phi;
+ variables.transport_porosity = phi_M_phi;
+
+ *p_L_m = *p_L_m_prev;
+ *S_L_m = *S_L_m_prev;
+ sigma_sw = *sigma_sw_prev;
+
+ variables.volumetric_mechanical_strain =
+ variables.volumetric_strain +
+ identity2.transpose() * C_el.inverse() * sigma_sw.sigma_sw;
+ variables_prev.volumetric_mechanical_strain =
+ variables_prev.volumetric_strain +
+ identity2.transpose() * C_el.inverse() *
+ sigma_sw_prev->sigma_sw;
+ return;
+ }

double const phi_m_prev = phi_prev->phi - phi_M_prev->phi;

auto const [delta_phi_m, delta_e_sw, delta_p_L_m, delta_sigma_sw] =
@@ -309,12 +2061,125 @@ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
*S_L_m_prev = S_L_m;
}

+ {
+ auto& n_l = std::get<MicroWaterContent>(this->current_states_[ip]);
+ auto& n_l_prev =
+ std::get<PrevState<MicroWaterContent>>(this->prev_states_[ip]);
+ auto& rho_LR =
+ std::get<MicroLiquidDensity>(this->current_states_[ip]);
+ auto& rho_LR_prev =
+ std::get<PrevState<MicroLiquidDensity>>(this->prev_states_[ip]);
+ auto& phi_m = std::get<MicroPorosity>(this->current_states_[ip]);
+ auto& phi_m_prev =
+ std::get<PrevState<MicroPorosity>>(this->prev_states_[ip]);
+
+ // Default fallback keeps state positive for vdW algebra.
+ double n_l_initial = 1e-6;
+ double rho_LR_initial = 1.0;
+ if (medium->hasProperty(MPL::PropertyType::saturation_micro))
+ {
+ auto const S_L_m_init =
+ *std::get<MicroSaturation>(this->current_states_[ip]);
+ n_l_initial = std::max(1e-12, S_L_m_init);
+ }
+
+ auto const& const potential_exchange_params_ptr =
+ this->getPotentialExchangeParameters();
+
+ if (isPotentialExchangeEnabled(potential_exchange_params_ptr))
+ {
+ auto const porosity =
+ std::get<ProcessLib::ThermoRichardsMechanics::PorosityData>(
+ this->current_states_[ip])
+ .phi;
+ auto const transport_porosity =
+ std::get<ProcessLib::ThermoRichardsMechanics::
+ TransportPorosityData>(this->current_states_[ip])
+ .phi;
+ n_l_initial = std::max(1e-12, porosity - transport_porosity);
+
+ n_l_initial =
+ potential_exchange_params_ptr->initial_micro_water_content
+ .value_or(n_l_initial);
+ rho_LR_initial = std::max(
+ 1e-16,
+ potential_exchange_params_ptr
+ ->micro_liquid_density_reference);
+ }
+
+ *n_l = n_l_initial;
+ **n_l_prev = n_l_initial;
+ *rho_LR = rho_LR_initial;
+ **rho_LR_prev = rho_LR_initial;
+ *phi_m = n_l_initial;
+ **phi_m_prev = n_l_initial;
+
+ if (isPotentialExchangeEnabled(potential_exchange_params_ptr))
+ {
+ auto const porosity =
+ std::get<ProcessLib::ThermoRichardsMechanics::PorosityData>(
+ this->current_states_[ip])

```

```

+         .phi;
+         auto const transport_porosity_init =
+         std::get<ProcessLib::ThermoRichardsMechanics::
+             TransportPorosityData>(this->current_states_[ip])
+         .phi;
+         auto const rho_LR_initial =
+         medium->phase("AqueousLiquid")
+         .property(MPL::PropertyType::density)
+         .template value<double>(variables, x_position, t, dt);
+         auto const compatibility_output =
+         computeCompatibilityMicroHydraulicOutput(
+             n_l_initial, rho_LR_initial,
+             { .phi = porosity,
+               .phi_M_prev = transport_porosity_init,
+               .phi_m_prev = n_l_initial,
+               .volumetric_strain = 0.0,
+               .volumetric_strain_prev = 0.0 },
+             *potential_exchange_params_ptr);
+         auto& p_L_m =
+         std::get<MicroPressure>(this->current_states_[ip]);
+         auto& p_L_m_prev =
+         std::get<PrevState<MicroPressure>>(this->prev_states_[ip]);
+         auto& S_L_m =
+         std::get<MicroSaturation>(this->current_states_[ip]);
+         auto& S_L_m_prev =
+         std::get<PrevState<MicroSaturation>>(
+             this->prev_states_[ip]);
+         *p_L_m = compatibility_output.p_L_m;
+         **p_L_m_prev = compatibility_output.p_L_m;
+         *S_L_m = compatibility_output.S_L_m;
+         **S_L_m_prev = compatibility_output.S_L_m;
+
+         auto& transport_porosity =
+         std::get<ProcessLib::ThermoRichardsMechanics::
+             TransportPorosityData>(
+             this->current_states_[ip])
+         .phi;
+         auto& transport_porosity_prev = std::get<PrevState<
+             ProcessLib::ThermoRichardsMechanics::TransportPorosityData>>(
+             this->prev_states_[ip]);
+         auto const transport_porosity_update =
+         computeTransportPorosityUpdate(
+             porosity, transport_porosity, n_l_initial, n_l_initial,
+             /*volumetric_strain=*/0.0,
+             /*volumetric_strain_prev=*/0.0,
+             potential_exchange_params_ptr
+             ->macro_porosity_update_mode);
+         *phi_m = transport_porosity_update.phi_m;
+         **phi_m_prev = transport_porosity_update.phi_m_prev;
+         transport_porosity = transport_porosity_update.phi_m;
+         transport_porosity_prev = phi =
+             transport_porosity_update.phi_M_prev;
+     }
+ }
+
+ // Set eps_m_prev from potentially non-zero eps and sigma_sw from
+ // restart.
+ auto& SD = this->current_states_[ip];
+ auto& state_current = this->current_states_[ip];
+ variables.stress =
+     std::get<ProcessLib::ConstitutiveRelations::EffectiveStressData>(
+         DisplacementDim>(SD)
+         DisplacementDim>(state_current)
+         .sigma_eff;
+
+ auto const& N_u = ip_data_[ip].N_u;
+ @ -352,10 +2217,11 @ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
+     this->prev_states_[ip])
+     ->eps_m;
+
+ bool const swelling_stress_active =
+     solid_phase.hasProperty(MPL::PropertyType::swelling_stress_rate) ||
+     isPotentialExchangeEnabled(this->getPotentialExchangeParameters());
+ eps_m_prev.noalias() =
+     - solid_phase.hasProperty(MPL::PropertyType::swelling_stress_rate)
+     - ? eps + C_el.inverse() * sigma_sw
+     - : eps;
+     swelling_stress_active ? eps + C_el.inverse() * sigma_sw : eps;
+ }
+ }
+
+ @ -468,10 +2334,10 @ void RichardsMechanicsLocalAssembler<
+     auto const alpha =
+     medium->property(MPL::PropertyType::biot_coefficient)
+     .template value<double>(variables, x_position, t, dt);
+     - auto& SD = this->current_states_[ip];
+     + auto& state_current = this->current_states_[ip];
+     variables.stress =
+     - std::get<ProcessLib::ConstitutiveRelations::EffectiveStressData>(
+     - DisplacementDim>(SD)
+     + DisplacementDim>(state_current)
+     - .sigma_eff;
+
+     // Set mechanical strain temporary to compute tangent stiffness.
+     variables.mechanical_strain
+     @ -489,7 +2355,6 @ void RichardsMechanicsLocalAssembler<
+         liquid_phase.property(MPL::PropertyType::density)
+         .template value<double>(variables, x_position, t, dt);
+         variables.density = rho_LR;
+
+     -
+
+     auto const& b = this->process_data_.specific_body_force;
+
+     S_L = medium->property(MPL::PropertyType::saturation)
+     @ -669,32 +2534,35 @ void RichardsMechanicsLocalAssembler<
+         MechanicalStrainData<DisplacementDim>>(
+             this->current_states_[ip])
+             .eps_m;
+     - eps_m.noalias() =
+     - solid_phase.hasProperty(MPL::PropertyType::swelling_stress_rate)
+     - ? eps.eps + C_el.inverse() * sigma_sw
+     - : eps.eps;
+     + bool const swelling_stress_active =
+     + solid_phase.hasProperty(MPL::PropertyType::swelling_stress_rate) ||
+     + isPotentialExchangeEnabled(
+     +     this->getPotentialExchangeParameters());
+     + eps_m.noalias() = swelling_stress_active
+     + ? eps.eps + C_el.inverse() * sigma_sw
+     + : eps.eps;

```



```

variables.mechanical_strain.emplace<
    MathLib::KelvinVector::KelvinVectorType<DisplacementDim>>(
        eps_m);
}

{
-   auto& SD = this->current_states_[ip];
-   auto const& SD_prev = this->prev_states_[ip];
+   auto& state_current = this->current_states_[ip];
+   auto const& state_previous = this->prev_states_[ip];
+   auto& sigma_eff =
        std::get<ProcessLib::ConstitutiveRelations::EffectiveStressData<
-           DisplacementDim>>(SD);
+           DisplacementDim>>(state_current);
+   auto const& sigma_eff_prev =
        std::get<PrevState<ProcessLib::ConstitutiveRelations::
            EffectiveStressData<DisplacementDim>>>(
-           SD_prev);
+           state_previous);
+   auto const& eps_m =
        std::get<ProcessLib::ConstitutiveRelations::
-           MechanicalStrainData<DisplacementDim>>(SD);
+           MechanicalStrainData<DisplacementDim>>(state_current);
+   auto& eps_m_prev =
        std::get<PrevState<ProcessLib::ConstitutiveRelations::
            MechanicalStrainData<DisplacementDim>>>(
-           SD_prev);
+           state_previous);

        auto const C = ip_data_[ip].updateConstitutiveRelation(
            variables, t, x_position, dt, temperature, sigma_eff,
@@ -749,6 +2617,67 @@ void RichardsMechanicsLocalAssembler<
    rhs.template segment<pressure_size>(pressure_index).noalias() +=
        dNdx_p.transpose() * rho_LR * rho_K_over_mu * b * w;

+   auto const* const potential_exchange_params_ptr =
+       this->getPotentialExchangeParameters();
+   bool const potential_exchange_enabled =
+       isPotentialExchangeEnabled(potential_exchange_params_ptr);
+   if ((medium->hasProperty(MPL::PropertyType::saturation_micro) ||
+       potential_exchange_enabled) &&
+       this->process_data._micro_porosity_parameters)
+   {
+       double const alpha_bar =
+           this->process_data._micro_porosity_parameters
+               ->mass_exchange_coefficient;
+       auto const p_L_m =
+           *std::get<MicroPressure>(this->current_states_[ip]);
+       double const p_L_ip = -p_cap_ip;
+       double const pressure_tolerance =
+           getPotentialPressureTolerance(
+               potential_exchange_params_ptr);
+
+       bool use_vdw_micro_potential_for_active_exchange = false;
+       double mu_LR_vdw = 0.0;
+       double dmu_LR_vdw_drho_LR = 0.0;
+
+       if (potential_exchange_enabled)
+       {
+           auto const n_L =
+               std::max(1e-16,
+                   *std::get<MicroWaterContent>(
+                       this->current_states_[ip]));
+           auto const transport_porosity_prev =
+               std::get<PrevState<ProcessLib::ThermoRichardsMechanics::
+                   TransportPorosityData>>(
+                   this->prev_states_[ip])
+                   ->phi;
+           auto const n_L_prev = *std::get<PrevState<MicroWaterContent>>(
+               this->prev_states_[ip]);
+           PotentialExchangeLocalSolveContext const local_solve_context{
+               .phi = phi,
+               .phi_M_prev = transport_porosity_prev,
+               .phi_m_prev = n_L_prev,
+               .volumetric_strain = variables.volumetric_strain,
+               .volumetric_strain_prev = variables_prev.volumetric_strain;
+           auto const micro_potential = computeActiveMicroPotential(
+               n_L, rho_LR, local_solve_context,
+               *potential_exchange_params_ptr);
+           use_vdw_micro_potential_for_active_exchange = true;
+           mu_LR_vdw = micro_potential.mu_LR;
+           dmu_LR_vdw_drho_LR = micro_potential.dmu_LR_drho_LR;
+       }

+       auto const potential_exchange_result = computePotentialExchangeUpdate(
+           alpha_bar, mu, p_L_ip, p_L_m, rho_LR, beta_LR,
+           pressure_tolerance, potential_exchange_enabled,
+           use_vdw_micro_potential_for_active_exchange, mu_LR_vdw,
+           dmu_LR_vdw_drho_LR,
+           /*use_custom_dmu_LR_vdw_dpl=*/false, /*dmu_LR_vdw_dpl=*/0.0,
+           /*use_fd_jacobian_for_direct_macro_derivative=*/false,
+           /*fd_jacobian_perturbation=*/1e-8);
+       rhs.template segment<pressure_size>(pressure_index).noalias() +=
+           N_p.transpose() * potential_exchange_result.exchange.rho_L_hat * w;
+   }

    //
    // displacement equation, pressure part
    //
@@ -767,9 +2696,9 @@ void RichardsMechanicsLocalAssembler<
    if (this->process_data._apply_mass_lumping)
    {
-       auto Mpp = M.template block<pressure_size, pressure_size>(
+       auto pressure_mass_block_diag = M.template block<pressure_size, pressure_size>(
            pressure_index, pressure_index);
-       Mpp = Mpp.colwise().sum().eval().asDiagonal();
+       pressure_mass_block_diag = pressure_mass_block_diag.colwise().sum().eval().asDiagonal();
    }
}

@@ -786,10 +2715,12 @@ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
    MPL::VariableArray& variables, MPL::VariableArray& variables_prev,
    MPL::Medium const* const medium, TemperatureData const T_data,
    CapillaryPressureData<DisplacementDim> const& p_cap_data,
-    ConstitutiveData<DisplacementDim>& CD,
-    StatefulData<DisplacementDim>& SD,

```

```

-     StatefulDataPrev<DisplacementDim> const& SD_prev,
+     ConstitutiveData<DisplacementDim>& constitutive_data,
+     StatefulData<DisplacementDim>& state_current,
+     StatefulDataPrev<DisplacementDim> const& state_previous,
+     std::optional<MicroPorosityParameters> const& micro_porosity_parameters,
+     PotentialExchangeParameters const& const
+     potential_exchange_parameters,
+     MaterialLib::Solids::MechanicsBase<DisplacementDim> const&
+     solid_material,
+     ProcessLib::ThermoRichardsMechanics::MaterialStateData<DisplacementDim>&
@@ -806,22 +2737,22 @@ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
double const p_cap_ip = p_cap_data.p_cap;
double const p_cap_prev_ip = p_cap_data.p_cap_prev;

- auto const& eps = std::get<StrainData<DisplacementDim>>>(SD);
+ auto const& eps = std::get<StrainData<DisplacementDim>>>(state_current);
auto& S_L =
-     std::get<ProcessLib::ThermoRichardsMechanics::SaturationData>(SD).S_L;
+     std::get<ProcessLib::ThermoRichardsMechanics::SaturationData>(state_current).S_L;
auto const S_L_prev =
std::get<
PrevState<ProcessLib::ThermoRichardsMechanics::SaturationData>>(<
SD_prev)
state_previous)
->S_L;
auto const alpha =
medium->property(MPL::PropertyType::biot_coefficient)
.template value<double>(variables, x_position, t, dt);
- *std::get<ProcessLib::ThermoRichardsMechanics::BiotData>(CD) = alpha;
+ *std::get<ProcessLib::ThermoRichardsMechanics::BiotData>(constitutive_data) = alpha;

variables.stress =
std::get<ProcessLib::ConstitutiveRelations::EffectiveStressData>(<
DisplacementDim>>>(SD)
DisplacementDim>>>(state_current)
.sigma_eff;
// Set mechanical strain temporary to compute tangent stiffness.
variables.mechanical_strain
@@ -834,14 +2765,14 @@ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
auto const beta_SR =
(1 - alpha) / solid_material.getBulkModulus(t, x_position, &C_e1);
variables.grain_compressibility = beta_SR;
- std::get<ProcessLib::ThermoRichardsMechanics::SolidCompressibilityData>(CD)
+ std::get<ProcessLib::ThermoRichardsMechanics::SolidCompressibilityData>(constitutive_data)
.beta_SR = beta_SR;

auto const rho_LR =
liquid_phase.property(MPL::PropertyType::density)
.template value<double>(variables, x_position, t, dt);
variables.density = rho_LR;
- *std::get<LiquidDensity>(CD) = rho_LR;
+ *std::get<LiquidDensity>(constitutive_data) = rho_LR;

S_L = medium->property(MPL::PropertyType::saturation)
.template value<double>(variables, x_position, t, dt);
@@ -854,7 +2785,7 @@ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
.template dValue<double>(variables,
MPL::Variable::capillary_pressure,
x_position, t, dt);
- std::get<ProcessLib::ThermoRichardsMechanics::SaturationDataDeriv>(CD)
+ std::get<ProcessLib::ThermoRichardsMechanics::SaturationDataDeriv>(constitutive_data)
.dS_L_dp_cap = dS_L_dp_cap;
// secant derivative from time discretization for storage
// use tangent, if secant is not available
@@ -862,7 +2793,7 @@ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
(p_cap_ip == p_cap_prev_ip)
? dS_L_dp_cap
: (S_L - S_L_prev) / (p_cap_ip - p_cap_prev_ip);
- std::get<SaturationSecantDerivative>(CD).DeltaS_L_Deltap_cap =
+ std::get<SaturationSecantDerivative>(constitutive_data).DeltaS_L_Deltap_cap =
DeltaS_L_Deltap_cap;

auto const chi = [medium, x_position, t, dt](double const S_L)
@@ -873,17 +2804,17 @@ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
.template value<double>(vs, x_position, t, dt);
});
double const chi_S_L = chi(S_L);
- std::get<ProcessLib::ThermoRichardsMechanics::BishopsData>(CD).chi_S_L =
+ std::get<ProcessLib::ThermoRichardsMechanics::BishopsData>(constitutive_data).chi_S_L =
chi_S_L;
double const chi_S_L_prev = chi(S_L_prev);
- std::get<PrevState<ProcessLib::ThermoRichardsMechanics::BishopsData>>>(CD)
+ std::get<PrevState<ProcessLib::ThermoRichardsMechanics::BishopsData>>>(constitutive_data)
->chi_S_L = chi_S_L_prev;

auto const dchi_dS_L =
medium->property(MPL::PropertyType::bishops_effective_stress)
.template dValue<double>(<
variables, MPL::Variable::liquid_saturation, x_position, t, dt);
- std::get<ProcessLib::ThermoRichardsMechanics::BishopsData>(CD).dchi_dS_L =
+ std::get<ProcessLib::ThermoRichardsMechanics::BishopsData>(constitutive_data).dchi_dS_L =
dchi_dS_L;

double const p_FR = -chi_S_L * p_cap_ip;
@@ -895,15 +2826,15 @@ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
// T000 (CL) changed that, using eps_prev for the moment, not B * u_prev
// variables_prev.volumetric_strain = Invariants::trace(B * u_prev);
variables_prev.volumetric_strain = Invariants::trace(
-     std::get<PrevState<StrainData<DisplacementDim>>>(SD_prev)->eps);
+     std::get<PrevState<StrainData<DisplacementDim>>>(state_previous)->eps);

auto& phi =
-     std::get<ProcessLib::ThermoRichardsMechanics::PorosityData>(SD).phi;
+     std::get<ProcessLib::ThermoRichardsMechanics::PorosityData>(state_current).phi;
{ // Porosity update
auto const phi_prev =
std::get<
PrevState<ProcessLib::ThermoRichardsMechanics::PorosityData>>(<
SD_prev)
state_previous)
->phi;
variables_prev.porosity = phi_prev;
phi = medium->property(MPL::PropertyType::porosity)
@@ -911,7 +2842,7 @@ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
t, dt);
variables.porosity = phi;
}

```

```

- std::get<ProcessLib::ThermoRichardsMechanics::PorosityData>(CD).phi = phi;
+ std::get<ProcessLib::ThermoRichardsMechanics::PorosityData>(constitutive_data).phi = phi;

if (alpha < phi)
{
@@ -927,7 +2858,7 @@ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,

auto const mu = liquid_phase.property(MPL::PropertyType::viscosity)
    .template value<double>(variables, x_position, t, dt);
- *std::get<ProcessLib::ThermoRichardsMechanics::LiquidViscosityData>(CD) =
+ *std::get<ProcessLib::ThermoRichardsMechanics::LiquidViscosityData>(constitutive_data) =
mu;

{
@@ -935,44 +2866,72 @@ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
auto& sigma_sw =
std::get<ProcessLib::ThermoRichardsMechanics::
ConstitutiveStressStrainTemperature::
SwellingDataStateful<DisplacementDim>>(SD);
+ SwellingDataStateful<DisplacementDim>>(state_current);
+
auto const& sigma_sw_prev =
std::get<PrevState<ProcessLib::ThermoRichardsMechanics::
ConstitutiveStressStrainTemperature::
SwellingDataStateful<DisplacementDim>>(
- SD_prev);
+ state_previous);
+
auto const transport_porosity_prev = std::get<PrevState<
ProcessLib::ThermoRichardsMechanics::TransportPorosityData>>(
- SD_prev);
+ state_previous);
+
auto const phi_prev = std::get<
PrevState<ProcessLib::ThermoRichardsMechanics::PorosityData>>(
- SD_prev);
+ state_previous);
+
auto& transport_porosity = std::get<
ProcessLib::ThermoRichardsMechanics::TransportPorosityData>(SD);
- auto& p_L_m = std::get<MicroPressure>(SD);
- auto const p_L_m_prev = std::get<PrevState<MicroPressure>>(SD_prev);
- auto& S_L_m = std::get<MicroSaturation>(SD);
- auto const S_L_m_prev = std::get<PrevState<MicroSaturation>>(SD_prev);
+ ProcessLib::ThermoRichardsMechanics::TransportPorosityData>(state_current);
+ auto& p_L_m = std::get<MicroPressure>(state_current);
+ auto const p_L_m_prev = std::get<PrevState<MicroPressure>>(state_previous);
+ auto& S_L_m = std::get<MicroSaturation>(state_current);
+ auto const S_L_m_prev = std::get<PrevState<MicroSaturation>>(state_previous);
+
updateSwellingStressAndVolumetricStrain<DisplacementDim>({
medium, solid_phase, C_el, rho_LR, mu, micro_porosity_parameters,
- alpha, phi, p_cap_ip, variables, variables_prev, x_position, t, dt,
- sigma_sw, sigma_sw_prev, transport_porosity_prev, phi_prev,
- transport_porosity, p_L_m_prev, S_L_m_prev, p_L_m, S_L_m);
+ potential_exchange_parameters, alpha, phi, p_cap_ip, variables,
+ variables_prev, x_position, t, dt, sigma_sw, sigma_sw_prev,
+ transport_porosity_prev, phi_prev, transport_porosity, p_L_m_prev,
+ S_L_m_prev, p_L_m, S_L_m);
}

+ auto const transport_porosity_prev_value = std::get<PrevState<
+ ProcessLib::ThermoRichardsMechanics::TransportPorosityData>>(state_previous)
+ ->phi;
+
+ auto const n_L_prev_value =
+ *std::get<PrevState<MicroWaterContent>>(state_previous);
+
+ updateMicroscaleHydraulicState<DisplacementDim>({
+ state_current, state_previous, p_cap_ip, rho_LR, mu, dt, variables, variables_prev,
+ {phi = phi,
+ .phi_M_prev = transport_porosity_prev_value,
+ .phi_m_prev = n_L_prev_value,
+ .volumetric_strain = variables.volumetric_strain,
+ .volumetric_strain_prev = variables_prev.volumetric_strain},
+ micro_porosity_parameters, potential_exchange_parameters);
+ updatePorositySplitState<DisplacementDim>({
+ state_current, state_previous, phi, variables, variables_prev,
+ potential_exchange_parameters);
+ updateTotalPorosityState<DisplacementDim>({
+ state_current, state_previous, phi, variables, variables_prev,
+ potential_exchange_parameters);
+ std::get<ProcessLib::ThermoRichardsMechanics::PorosityData>(constitutive_data).phi =
+ std::get<ProcessLib::ThermoRichardsMechanics::PorosityData>(state_current).phi;
+ updateSwellingState<DisplacementDim>({
+ solid_phase, C_el, state_current, state_previous, variables, variables_prev, x_position,
+ t, dt, potential_exchange_parameters);
+
if (medium->hasProperty(MPL::PropertyType::transport_porosity))
{
- if (!medium->hasProperty(MPL::PropertyType::saturation_micro))
+ if (!medium->hasProperty(MPL::PropertyType::saturation_micro) &&
+ !isPotentialExchangeEnabled(potential_exchange_parameters))
{
auto& transport_porosity =
std::get<
ProcessLib::ThermoRichardsMechanics::TransportPorosityData>(
- SD)
+ state_current)
+ .phi;
+
auto const transport_porosity_prev = std::get<PrevState<
ProcessLib::ThermoRichardsMechanics::TransportPorosityData>>(
- SD_prev)
+ state_previous)
+ ->phi;
+
variables_prev.transport_porosity = transport_porosity_prev;
}

@@ -994,7 +2953,7 @@ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
// T000 mechanical constitutive relation will be evaluated afterwards
auto const sigma_total =
(std::get<ProcessLib::ConstitutiveRelations::EffectiveStressData<
- DisplacementDim>>(SD)
+ DisplacementDim>>(state_current)
+ .sigma_eff +
alpha * p_FR * identity2)
    .eval();
@@ -1017,11 +2976,11 @@ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,

std::get<
ProcessLib::ThermoRichardsMechanics::PermeabilityData<DisplacementDim>>(
- CD)
+ constitutive_data)

```

```

.k_rel = k_rel;
std::get<
    ProcessLib::ThermoRichardsMechanics::PermeabilityData<DisplacementDim>>(
        CD)
+    constitutive_data)
.kl = K_intrinsic;

//
@@ -1032,17 +2991,19 @@ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
    auto& sigma_sw =
        std::get<ProcessLib::ThermoRichardsMechanics::
            ConstitutiveStressStrainTemperature::
                SwellingDataStateful<DisplacementDim>>(SD)
+                SwellingDataStateful<DisplacementDim>>(state_current)
        .sigma_sw;

    auto& eps_m =
        std::get<ProcessLib::ConstitutiveRelations::MechanicalStrainData<
            DisplacementDim>>(SD)
+            DisplacementDim>>(state_current)
        .eps_m;
+    bool const swelling_stress_active =
+        solid_phase.hasProperty(MPL::PropertyType::swelling_stress_rate) ||
+        isPotentialExchangeEnabled(potential_exchange_parameters);
    eps_m.noalias() =
        solid_phase.hasProperty(MPL::PropertyType::swelling_stress_rate)
        ? eps.eps + C_el.inverse() * sigma_sw
        : eps.eps;
+    swelling_stress_active ? eps.eps + C_el.inverse() * sigma_sw
+        : eps.eps;
    variables.mechanical_strain
        .emplace<MathLib::KelvinVectorType<DisplacementDim>>(
            eps_m);
@@ -1051,31 +3012,31 @@ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
{
    auto& sigma_eff =
        std::get<ProcessLib::ConstitutiveRelations::EffectiveStressData<
            DisplacementDim>>(SD);
+            DisplacementDim>>(state_current);
    auto const& sigma_eff_prev =
        std::get<PrevState<ProcessLib::ConstitutiveRelations::
            EffectiveStressData<DisplacementDim>>>(
                SD_prev);
+                state_previous);
    auto const& eps_m =
        std::get<ProcessLib::ConstitutiveRelations::MechanicalStrainData<
            DisplacementDim>>(SD);
+            DisplacementDim>>(state_current);
    auto& eps_m_prev =
        std::get<PrevState<ProcessLib::ConstitutiveRelations::
            MechanicalStrainData<DisplacementDim>>>(
                SD_prev);
+                state_previous);

    auto C = ip_data.updateConstitutiveRelation(
        variables, t, x_position, dt, temperature, sigma_eff,
        sigma_eff_prev, eps_m, eps_m_prev, solid_material,
        material_state_data.material_state_variables);

    *std::get<StiffnessTensor<DisplacementDim>>(CD) = std::move(C);
+    *std::get<StiffnessTensor<DisplacementDim>>(constitutive_data) = std::move(C);
}

// p_SR
variables.solid_grain_pressure =
    p_FR - std::get<ProcessLib::ConstitutiveRelations::EffectiveStressData<
        DisplacementDim>>(SD)
+        DisplacementDim>>(state_current)
        .sigma_eff.dot(identity2) /
        (3 * (1 - phi));
    auto const rho_SR =
@@ -1083,7 +3044,7 @@ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
    .template value<double>(variables, x_position, t, dt);

    double const rho = rho_SR * (1 - phi) + S_L * phi * rho_LR;
    *std::get<Density>(CD) = rho;
+    *std::get<Density>(constitutive_data) = rho;
}

template <typename ShapeFunctionDisplacement, typename ShapeFunctionPressure,
@@ -1156,9 +3117,9 @@ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
    this->integration_method_.getNumberOfPoints();
    for (unsigned ip = 0; ip < n_integration_points; ip++)
    {
        ConstitutiveData<DisplacementDim> CD;
        auto& SD = this->current_states[ip];
        auto const& SD_prev = this->prev_states[ip];
+        ConstitutiveData<DisplacementDim> constitutive_data;
        auto& state_current = this->current_states[ip];
        auto const& state_previous = this->prev_states[ip];
        [[maybe_unused]] auto models = createConstitutiveModels(
            this->process_data_, this->solid_material_);
@@ -1201,7 +3162,7 @@ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
    .template value<double>(variables, x_position, t, dt);
    variables.temperature = temperature;

    std::get<StrainData<DisplacementDim>>(SD).eps.noalias() = B * u;
+    std::get<StrainData<DisplacementDim>>(state_current).eps.noalias() = B * u;

    assembleWithJacobianEvalConstitutiveSetting(
        t, dt, x_position, ip_data[ip], variables, variables_prev, medium,
@@ -1209,11 +3170,11 @@ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
    CapillaryPressureData<DisplacementDim>{
        p_cap_ip, p_cap_prev_ip,
        Eigen::Vector<double, DisplacementDim::Zero()>,
        CD, SD, SD_prev, this->process_data_.micro_porosity_parameters,
+        constitutive_data, state_current, state_previous, this->process_data_.micro_porosity_parameters,
        this->getPotentialExchangeParameters(),
        this->solid_material_, this->material_states[ip]);

    {
        auto const& C = *std::get<StiffnessTensor<DisplacementDim>>(CD);
+        auto const& C = *std::get<StiffnessTensor<DisplacementDim>>(constitutive_data);
        local_Jac
            .template block<displacement_size, displacement_size>(
                displacement_index, displacement_index)

```

```

@@ -1227,7 +3189,7 @@ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
    std::get<ProcessLib::ConstitutiveRelations::EffectiveStressData<
        DisplacementDim>>(this->current_states[ip])
        .sigma_eff;
-    double const rho = *std::get<Density>(CD);
+    double const rho = *std::get<Density>(constitutive_data);
    local_rhs.template segment<displacement_size>(displacement_index)
        .noalias() -= (B.transpose() * sigma_eff -
            N_u_op(N_u).transpose() * rho * b) *
@@ -1239,20 +3201,20 @@ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
    //

    double const alpha =
-    *std::get<ProcessLib::ThermoRichardsMechanics::BiotData>(CD);
+    *std::get<ProcessLib::ThermoRichardsMechanics::BiotData>(constitutive_data);
    double const dS_L_dp_cap =
    std::get<ProcessLib::ThermoRichardsMechanics::SaturationDataDeriv>(
        CD)
+    constitutive_data)
        .dS_L_dp_cap;

    {
        double const chi_S_L =
-        std::get<ProcessLib::ThermoRichardsMechanics::BishopsData>(CD)
+        std::get<ProcessLib::ThermoRichardsMechanics::BishopsData>(constitutive_data)
            .chi_S_L;
        Kp.noalias() +=
            B.transpose() * alpha * chi_S_L * identity2 * N_p * w;
        double const dchi_dS_L =
-        std::get<ProcessLib::ThermoRichardsMechanics::BishopsData>(CD)
+        std::get<ProcessLib::ThermoRichardsMechanics::BishopsData>(constitutive_data)
            .dchi_dS_L;

        local_jac
@@ -1264,8 +3226,8 @@ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
    }

    double const phi =
-    std::get<ProcessLib::ThermoRichardsMechanics::PorosityData>(CD).phi;
-    double const rho_LR = *std::get<LiquidDensity>(CD);
+    std::get<ProcessLib::ThermoRichardsMechanics::PorosityData>(constitutive_data).phi;
+    double const rho_LR = *std::get<LiquidDensity>(constitutive_data);
    local_jac
        .template block<displacement_size, pressure_size>(
            displacement_index, pressure_index)
@@ -1282,6 +3244,6 @@ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
    //      (p_cap_ip - p_cap_prev_ip) * N_p * w;
    // }
    if (!medium->hasProperty(MPL::PropertyType::saturation_micro) &&
+    !isPotentialExchangeEnabled(
+    this->getPotentialExchangeParameters()) &&
        solid_phase.hasProperty(MPL::PropertyType::swelling_stress_rate))
    {
        using DimMatrix = Eigen::Matrix<double, 3, 3>;
@@ -1309,7 +3273,7 @@ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
    if (this->process_data._explicit_hm_coupling_in_unsaturated_zone)
    {
        double const chi_S_L_prev = std::get<PrevState<
-            ProcessLib::ThermoRichardsMechanics::BishopsData>>(CD)
+            ProcessLib::ThermoRichardsMechanics::BishopsData>>(constitutive_data)
            ->chi_S_L;
        Kpu.noalias() += N_p.transpose() * chi_S_L_prev * rho_LR * alpha *
            identity2.transpose() * B * w;
@@ -1326,15 +3290,15 @@ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,

    double const k_rel =
    std::get<ProcessLib::ThermoRichardsMechanics::PermeabilityData<
-        DisplacementDim>>(CD)
+        DisplacementDim>>(constitutive_data)
        .k_rel;
    auto const& K_intrinsic =
    std::get<ProcessLib::ThermoRichardsMechanics::PermeabilityData<
-        DisplacementDim>>(CD)
+        DisplacementDim>>(constitutive_data)
        .Ki;
    double const mu =
        *std::get<ProcessLib::ThermoRichardsMechanics::LiquidViscosityData>(
            CD);
+    constitutive_data);

    GlobalDimMatrixType const rho_Ki_over_mu = K_intrinsic * rho_LR / mu;

@@ -1351,7 +3315,7 @@ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
    double const beta_SR =
    std::get<
        ProcessLib::ThermoRichardsMechanics::SolidCompressibilityData>(
-        CD)
+        constitutive_data)
        .beta_SR;
    double const a0 = (alpha - phi) * beta_SR;
    double const specific_storage_a_p = S_L * (phi * beta_LR + S_L * a0);
@@ -1366,7 +3330,7 @@ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
    N_p.transpose() * rho_LR * specific_storage_a_p * N_p * w;

    double const DeltaS_L_Deltap_cap =
-    std::get<SaturationSecantDerivative>(CD).DeltaS_L_Deltap_cap;
+    std::get<SaturationSecantDerivative>(constitutive_data).DeltaS_L_Deltap_cap;
    storage_p_a_S.noalias() -= N_p.transpose() * rho_LR *
        specific_storage_a_S * DeltaS_L_Deltap_cap *
        N_p * w;

@@ -1428,22 +3384,22 @@ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
    local_rhs.template segment<pressure_size>(pressure_index).noalias() +=
        dNdX_p.transpose() * rho_LR * k_rel * rho_Ki_over_mu * b * w;

-    if (medium->hasProperty(MPL::PropertyType::saturation_micro))
+    auto const& const potential_exchange_params_ptr =
+    this->getPotentialExchangeParameters();
+    bool const potential_exchange_enabled =
+    isPotentialExchangeEnabled(potential_exchange_params_ptr);
+    if ((medium->hasProperty(MPL::PropertyType::saturation_micro) ||
+    potential_exchange_enabled) &&
+    this->process_data._micro_porosity_parameters)
    {
        double const alpha_bar =
            this->process_data._micro_porosity_parameters
            ->mass_exchange_coefficient;
        auto const p_L_m =

```

```

+         *std::get<MicroPressure>(this->current_states_[ip]);
+
+         double const p_L_ip = -p_cap_ip;
+         double const pressure_tolerance =
+             getPotentialPressureTolerance(
+                 potential_exchange_params_ptr);
+
+
+         bool use_vdw_micro_potential_for_active_exchange = false;
+         double mu_LR_vdw = 0.0;
+         double dmu_LR_vdw_drho_LR = 0.0;
+         bool use_custom_dmu_LR_vdw_dpl = false;
+         double dmu_LR_vdw_dpl = 0.0;
+         bool use_fd_jacobian_for_direct_macro_derivative = false;
+         double fd_jacobian_perturbation = 1e-8;
+         if (potential_exchange_enabled)
+         {
+             auto const n_l =
+                 std::max(1e-16,
+                     *std::get<MicroWaterContent>(
+                         this->current_states_[ip]));
+             auto const transport_porosity_prev =
+                 std::get<PrevState<ProcessLib::ThermoRichardsMechanics::
+                     TransportPorosityData>>(
+                     this->prev_states_[ip])
+                     ->phi;
+             auto const n_l_prev = **std::get<PrevState<MicroWaterContent>>(
+                 this->prev_states_[ip]);
+             PotentialExchangeLocalSolveContext const local_solve_context{
+                 .phi = phi,
+                 .phi_M_prev = transport_porosity_prev,
+                 .phi_m_prev = n_l_prev,
+                 .volumetric_strain = variables.volumetric_strain,
+                 .volumetric_strain_prev = variables_prev.volumetric_strain;
+             }
+             auto const micro_potential = computeActiveMicroPotential(
+                 n_l, rho_LR, local_solve_context,
+                 *potential_exchange_params_ptr);
+             use_vdw_micro_potential_for_active_exchange = true;
+             mu_LR_vdw = micro_potential.mu_LR;
+             dmu_LR_vdw_drho_LR = micro_potential.dmu_LR_drho_LR;
+             use_fd_jacobian_for_direct_macro_derivative =
+                 potential_exchange_params_ptr
+                 ->use_fd_jacobian_for_exchange;
+             fd_jacobian_perturbation =
+                 potential_exchange_params_ptr->fd_jacobian_perturbation;
+
+
+             // In analytic mode, include implicit n_l(p_L) chain coupling
+             // in dmu_LR/dp_L for the active exchange Jacobian term.
+             if (!use_fd_jacobian_for_direct_macro_derivative)
+             {
+                 double const drho_LR_dpl = rho_LR * beta_LR;
+                 auto const macro_potential = computeYoungLaplaceMacroPotential(
+                     p_L_ip, rho_LR, pressure_tolerance);
+                 double const alpha_M_effective = alpha_bar * rho_LR / mu;
+                 auto const exchange = computePotentialDrivenMassExchange(
+                     alpha_M_effective, macro_potential.mu_LR,
+                     micro_potential.mu_LR);
+                 double const dn_l_dpl = computeImplicitNLDpl(
+                     n_l_prev, p_L_ip, dt, rho_LR, drho_LR_dpl, alpha_bar, mu,
+                     macro_potential, micro_potential, exchange,
+                     local_solve_context,
+                     *potential_exchange_params_ptr);
+
+                 dmu_LR_vdw_dpl = micro_potential.dmu_LR_dnl * dn_l_dpl +
+                     micro_potential.dmu_LR_drho_LR *
+                     drho_LR_dpl;
+                 use_custom_dmu_LR_vdw_dpl = true;
+
+                 auto const n_l_prev =
+                     std::max(1e-16,
+                         **std::get<PrevState<MicroWaterContent>>(
+                             this->prev_states_[ip]));
+                 if (potential_exchange_params_ptr->check_local_jacobian)
+                 {
+                     auto const fd_data =
+                         computeLocalJacobianDiagnosticData(
+                             n_l_prev, p_L_ip, dt, rho_LR, drho_LR_dpl,
+                             alpha_bar, mu, pressure_tolerance,
+                             local_solve_context,
+                             *potential_exchange_params_ptr);
+                     auto const analytic_drho_L_hat_dpl =
+                         -exchange.drho_L_hat_dalpha_M *
+                         (alpha_bar / mu * drho_LR_dpl) -
+                         exchange.drho_L_hat_dmu_LR *
+                         (macro_potential.dmu_LR_dpl +
+                         macro_potential.dmu_LR_drho_LR *
+                         drho_LR_dpl) -
+                         exchange.drho_L_hat_dmu_LR * dmu_LR_vdw_dpl;
+                     maybeLogLocalJacobianDiagnostic(
+                         p_L_ip, n_l_prev, n_l, dn_l_dpl,
+                         analytic_drho_L_hat_dpl, fd_data,
+                         *potential_exchange_params_ptr);
+                 }
+             }
+         }
+
+         auto const potential_exchange_result = computePotentialExchangeUpdate(
+             alpha_bar, mu, p_L_ip, p_L_m, rho_LR, beta_LR,
+             pressure_tolerance, potential_exchange_enabled,
+             use_vdw_micro_potential_for_active_exchange, mu_LR_vdw,
+             dmu_LR_vdw_drho_LR, use_custom_dmu_LR_vdw_dpl,
+             dmu_LR_vdw_dpl,
+             use_fd_jacobian_for_direct_macro_derivative,
+             fd_jacobian_perturbation);
+
+         // Phase 2C: activate potential-driven exchange source in the macro
+         // balance.
+         local_rhs.template segment<pressure_size>(pressure_index)
+         -
+         .noalias() -=
+         -
+         N_p.transpose() * alpha_bar / mu * (-p_cap_ip - p_L_m) * w;
+         .noalias() += N_p.transpose() * potential_exchange_result.exchange.drho_L_hat *
+         +
+         w;
+
+
+         // Direct macro Jacobian term for the exchange source. In analytic
+         // mode this includes the implicit n_l(p_L) chain contribution.
+         local_Jac
+         .template block<pressure_size, pressure_size>(pressure_index,
+             pressure_index)
+         -
+         .noalias() += N_p.transpose() * alpha_bar / mu * N_p * w;
+         -
+         if (p_cap_ip != p_cap_prev_ip)

```

```

+         .noalias() -= N_p.transpose() *
+         potential_exchange_result.drho_L_hat_dpL_direct * N_p * w;
+
+ // Keep the microscale pressure-state sensitivity lagged via the
+ // secant term only in the placeholder microscale path. In the
+ // vdw+ n_l opt-in path this term is intentionally omitted because
+ // the active microscale potential is no longer p_L_m/rho_LR.
+ if (luse_vdw_micro_potential_for_active_exchange &&
+     p_cap_ip != p_cap_prev_ip)
+ {
+     auto const p_L_m_prev = *std::get<PrevState<MicroPressure>>(
+         this->prev_states_[ip]);
+ @ -1631,10 +3713,10 @ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
+     auto const alpha =
+         medium->property(MPL::PropertyType::biot_coefficient)
+         .template value<double>(variables, x_position, t, dt);
+ -     auto& SD = this->current_states_[ip];
+ +     auto& state_current = this->current_states_[ip];
+     variables.stress =
+         std::get<ProcessLib::ConstitutiveRelations::EffectiveStressData<
+ -         DisplacementDim>>(SD)
+ +         DisplacementDim>>(state_current)
+         .sigma_eff;
+
+     // Set mechanical strain temporary to compute tangent stiffness.
+     variables.mechanical_strain
+ @ -1708,15 +3790,47 @ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
+
+     updateSwellingStressAndVolumetricStrain<DisplacementDim>(
+         *medium, solid_phase, C_el, rho_LR, mu,
+ -         this->process_data->micro_porosity_parameters, alpha, phi,
+ -         p_cap_ip, variables, variables_prev, x_position, t, dt,
+ -         sigma_sw, sigma_sw_prev, transport_porosity_prev, phi_prev,
+ +         this->process_data->micro_porosity_parameters,
+ +         this->getPotentialExchangeParameters(), alpha, phi, p_cap_ip,
+ +         variables, variables_prev, x_position, t, dt, sigma_sw,
+ +         sigma_sw_prev, transport_porosity_prev, phi_prev,
+ +         transport_porosity, p_L_m_prev, S_L_m_prev, p_L_m, S_L_m);
+ }
+
+ auto const transport_porosity_prev_value = std::get<PrevState<
+ ProcessLib::ThermoRichardsMechanics::TransportPorosityData>>(
+     this->prev_states_[ip])
+
+ ->phi;
+
+ auto const n_l_prev_value =
+ *std::get<PrevState<MicroWaterContent>>(
+     this->prev_states_[ip]);
+
+ updateMicroscaleHydraulicState<DisplacementDim>(
+     this->current_states_[ip], this->prev_states_[ip], p_cap_ip,
+     rho_LR, mu, dt, variables, variables_prev,
+     { .phi = phi,
+       .phi_M_prev = transport_porosity_prev_value,
+       .phi_m_prev = n_l_prev_value,
+       .volumetric_strain = variables.volumetric_strain,
+       .volumetric_strain_prev = variables_prev.volumetric_strain,
+       this->process_data->micro_porosity_parameters,
+       this->getPotentialExchangeParameters();
+
+     updatePorositySplitState<DisplacementDim>(
+         this->current_states_[ip], this->prev_states_[ip], phi, variables,
+         variables_prev, this->getPotentialExchangeParameters());
+     updateTotalPorosityState<DisplacementDim>(
+         this->current_states_[ip], this->prev_states_[ip], phi, variables,
+         variables_prev, this->getPotentialExchangeParameters());
+     updateSwellingState<DisplacementDim>(
+         solid_phase, C_el, this->current_states_[ip],
+         this->prev_states_[ip], variables, variables_prev, x_position, t,
+         dt, this->getPotentialExchangeParameters());
+
+ if (medium->hasProperty(MPL::PropertyType::transport_porosity))
+ {
+ -     if (!medium->hasProperty(MPL::PropertyType::saturation_micro))
+ +     if (!medium->hasProperty(MPL::PropertyType::saturation_micro) &&
+ +         !isPotentialExchangeEnabled(
+ +             this->getPotentialExchangeParameters()))
+     {
+         auto& transport_porosity =
+             std::get<ProcessLib::ThermoRichardsMechanics::
+ @ -1783,42 +3897,45 @ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
+ *std::get<DrySolidDensity>(this->output_data_[ip]) = (1 - phi) * rho_SR;
+
+ {
+     auto& SD = this->current_states_[ip];
+ +     auto& state_current = this->current_states_[ip];
+     auto const& sigma_sw =
+         std::get<ProcessLib::ThermoRichardsMechanics::
+         ConstitutiveStressStrainTemperature::
+ -         SwellingDataStateful<DisplacementDim>>(SD)
+ +         SwellingDataStateful<DisplacementDim>>(state_current)
+         .sigma_sw;
+     auto& eps_m =
+         std::get<ProcessLib::ConstitutiveRelations::
+ -         MechanicalStrainData<DisplacementDim>>(SD)
+ +         MechanicalStrainData<DisplacementDim>>(state_current)
+         .eps_m;
+ -     eps_m.noalias() =
+ -         solid_phase.hasProperty(MPL::PropertyType::swelling_stress_rate)
+ -         ? eps + C_el.inverse() * sigma_sw
+ -         : eps;
+ +     bool const swelling_stress_active =
+ +         solid_phase.hasProperty(MPL::PropertyType::swelling_stress_rate) ||
+ +         isPotentialExchangeEnabled(
+ +             this->getPotentialExchangeParameters());
+ +     eps_m.noalias() = swelling_stress_active
+ +         ? eps + C_el.inverse() * sigma_sw
+ +         : eps;
+     variables.mechanical_strain.emplace<
+         MathLib::KelvinVector::KelvinVectorType<DisplacementDim>>(
+             eps_m);
+ }
+
+ {
+     auto& SD = this->current_states_[ip];
+ -     auto const& SD_prev = this->prev_states_[ip];
+ +     auto& state_current = this->current_states_[ip];
+ +     auto const& state_previous = this->prev_states_[ip];
+     auto& sigma_eff =
+         std::get<ProcessLib::ConstitutiveRelations::EffectiveStressData<

```

```

-         DisplacementDim>>(SD);
+         DisplacementDim>>(state_current);
    auto const& sigma_eff_prev =
        std::get<PrevState<ProcessLib::ConstitutiveRelations::
            EffectiveStressData<DisplacementDim>>>(<
-             SD_prev);
+             state_previous);
    auto const& eps_m =
        std::get<ProcessLib::ConstitutiveRelations::
            MechanicalStrainData<DisplacementDim>>>(SD);
+        MechanicalStrainData<DisplacementDim>>(state_current);
    auto const& eps_m_prev =
        std::get<PrevState<ProcessLib::ConstitutiveRelations::
            MechanicalStrainData<DisplacementDim>>>(<
-             SD_prev);
+             state_previous);

    ip_data[ip].updateConstitutiveRelation(
        variables, t, x_position, dt, temperature, sigma_eff,
diff --git a/ProcessLib/RichardsMechanics/RichardsMechanicsFEM.h b/ProcessLib/RichardsMechanics/RichardsMechanicsFEM.h
index c50bb2f436..eeb77c825c 100644
--- a/ProcessLib/RichardsMechanics/RichardsMechanicsFEM.h
+++ b/ProcessLib/RichardsMechanics/RichardsMechanicsFEM.h
@@ -114,7 +114,7 @@ public:

    for (unsigned ip = 0; ip < n_integration_points; ip++)
    {
-        auto& SD = this->current_states_[ip];
+        auto& state_current = this->current_states_[ip];
        auto& ip_data = ip_data_[ip];

        ParameterLib::SpatialPosition const x_position{
@@ -128,7 +128,7 @@ public:
            if (this->process_data._initial_stress.value)
            {
                std::get<ProcessLib::ConstitutiveRelations::EffectiveStressData<
-                 DisplacementDim>>>(SD)
+                 DisplacementDim>>>(state_current)
                .sigma_eff =
                    MathLib::KelvinVector::symmetricTensorToKelvinVector<
                    DisplacementDim>(<
@@ -147,7 +147,7 @@ public:

                this->material_states_[ip].pushBackState();

-                this->prev_states_[ip] = SD;
+                this->prev_states_[ip] = state_current;
            }
        }

@@ -226,10 +226,12 @@ private:
    MPL::VariableArray& variables, MPL::VariableArray& variables_prev,
    MPL::Medium const& medium, TemperatureData const T_data,
    CapillaryPressureData<DisplacementDim> const& p_cap_data,
-    ConstitutiveData<DisplacementDim>& CD,
-    StatefulData<DisplacementDim>& SD,
+    StatefulData<DisplacementDim>& SD,
+    StatefulData<DisplacementDim>& SD_prev,
+    ConstitutiveData<DisplacementDim>& constitutive_data,
+    StatefulData<DisplacementDim>& state_current,
+    StatefulData<DisplacementDim>& state_previous,
    std::optional<MicroPorosityParameters> const& micro_porosity_parameters,
    PotentialExchangeParameters const& const
    potential_exchange_parameters,
    MaterialLib::Solids::MechanicsBase<DisplacementDim> const&
    solid_material,
    ProcessLib::ThermoRichardsMechanics::MaterialStateData<DisplacementDim>&
diff --git a/ProcessLib/RichardsMechanics/RichardsMechanicsProcessData.h b/ProcessLib/RichardsMechanics/RichardsMechanicsProcessData.h
index 192f6b6b88..81c7f055ab 100644
--- a/ProcessLib/RichardsMechanics/RichardsMechanicsProcessData.h
+++ b/ProcessLib/RichardsMechanics/RichardsMechanicsProcessData.h
@@ -11,6 +11,7 @@
@@ -11,6 +11,7 @@
#include "MaterialLib/MPL/MaterialSpatialDistributionMap.h"
#include "ParameterLib/Parameter.h"
#include "ProcessLib/Common/HydroMechanics/InitialStress.h"
+include "PotentialExchangeParameters.h"

namespace MaterialLib
{
@@ -44,6 +45,9 @@ struct RichardsMechanicsProcessData
    Eigen::Matrix<double, DisplacementDim, 1> const specific_body_force;

    std::optional<MicroPorosityParameters> micro_porosity_parameters;
+    std::optional<PotentialExchangeParameters> potential_exchange_parameters;
+    std::map<int, PotentialExchangeParameters>
+    potential_exchange_parameters_by_material;

    bool const apply_mass_lumping;
diff --git a/ProcessLib/RichardsMechanics/Tests.cmake b/ProcessLib/RichardsMechanics/Tests.cmake
index a90e2696f8..663a3c8d02 100644
--- a/ProcessLib/RichardsMechanics/Tests.cmake
+++ b/ProcessLib/RichardsMechanics/Tests.cmake
@@ -34,6 +34,15 @@
@@ -34,6 +34,15 @@
if (NOT OGS_USE_MPI AND OGS_USE_MFRONT)
    OgsTest(PROJECTFILE RichardsMechanics/DoubleStructureBenchmark/double_porosity_swelling_RM.prj RUNTIME 1)
endif()

+AddTest(
+    NAME RichardsMechanics_double_porosity_swelling_dsm_micromacro_constbc_reference
+    PATH RichardsMechanics
+    EXECUTABLE ogs
+    RUNTIME 20
+    EXECUTABLE_ARGS double_porosity_swelling_dsm_micromacro_constbc.xml
+    WRAPPER time
+    TESTER vtkdiff
+    REQUIREMENTS NOT OGS_USE_MPI
+    DIFF_DATA
+    dsm_micromacro_constbc_reference_t_1000.000000.vtu double_porosity_swelling_dsm_micromacro_constbc_t_1000.000000.vtu pressure pressure 1e-16 1e-12
+    dsm_micromacro_constbc_reference_t_1000.000000.vtu double_porosity_swelling_dsm_micromacro_constbc_t_1000.000000.vtu saturation saturation 4e-15 0
+    dsm_micromacro_constbc_reference_t_1000.000000.vtu double_porosity_swelling_dsm_micromacro_constbc_t_1000.000000.vtu porosity porosity 1e-16 0
+    dsm_micromacro_constbc_reference_t_1000.000000.vtu double_porosity_swelling_dsm_micromacro_constbc_t_1000.000000.vtu transport_porosity transport_porosity 1e-16 0
+    dsm_micromacro_constbc_reference_t_1000.000000.vtu double_porosity_swelling_dsm_micromacro_constbc_t_1000.000000.vtu micro_pressure micro_pressure 1e-16 1e-12
+    dsm_micromacro_constbc_reference_t_1000.000000.vtu double_porosity_swelling_dsm_micromacro_constbc_t_1000.000000.vtu micro_saturation micro_saturation 4e-15 0
+    dsm_micromacro_constbc_reference_t_1000.000000.vtu double_porosity_swelling_dsm_micromacro_constbc_t_1000.000000.vtu swelling_stress swelling_stress 5e-14 0
+)
+
+AddTest(
+    NAME RichardsMechanics_beacon_1a01_dsm_micromacro_smoke

```



```

+ PATH RichardsMechanics
+ EXECUTABLE ogs
+ RUNTIME 20
+ EXECUTABLE_ARGS beacon_1a01_dsm_micromacro_smoke.prj
+ WRAPPER time
+ REQUIREMENTS NOT OGS_USE_MPI
+)
+
+AddTest(
+ NAME RichardsMechanics_beacon_1a01_dsm_micromacro_stressprobe
+ PATH RichardsMechanics
+ EXECUTABLE ogs
+ RUNTIME 20
+ EXECUTABLE_ARGS beacon_1a01_dsm_micromacro_stressprobe.prj
+ WRAPPER time
+ REQUIREMENTS NOT OGS_USE_MPI
+)
+
+AddTest(
+ NAME RichardsMechanics_beacon_1a01_dsm_micromacro_inflow
+ PATH RichardsMechanics
+ EXECUTABLE ogs
+ RUNTIME 20
+ EXECUTABLE_ARGS beacon_1a01_dsm_micromacro_inflow.prj
+ WRAPPER time
+ REQUIREMENTS NOT OGS_USE_MPI
+)
+
+AddTest(
+ NAME RichardsMechanics_beacon_1a01_dsm_micromacro_reference
+ PATH RichardsMechanics
+ EXECUTABLE ogs
+ RUNTIME 20
+ EXECUTABLE_ARGS beacon_1a01_dsm_micromacro_smoke.prj
+ WRAPPER time
+ TESTER vtkdiff
+ REQUIREMENTS NOT OGS_USE_MPI
+ DIFF_DATA
+ beacon_1a01_reference_t_1000.000000.vtu beacon_1a01_dsm_micromacro_smoke_t_1000.000000.vtu displacement displacement 1e-16 0
+ beacon_1a01_reference_t_1000.000000.vtu beacon_1a01_dsm_micromacro_smoke_t_1000.000000.vtu pressure pressure 1e-16 1e-12
+ beacon_1a01_reference_t_1000.000000.vtu beacon_1a01_dsm_micromacro_smoke_t_1000.000000.vtu saturation saturation 1e-14 0
+ beacon_1a01_reference_t_1000.000000.vtu beacon_1a01_dsm_micromacro_smoke_t_1000.000000.vtu micro_pressure micro_pressure 1e-16 1e-12
+ beacon_1a01_reference_t_1000.000000.vtu beacon_1a01_dsm_micromacro_smoke_t_1000.000000.vtu micro_saturation micro_saturation 1e-14 0
+ beacon_1a01_reference_t_1000.000000.vtu beacon_1a01_dsm_micromacro_smoke_t_1000.000000.vtu swelling_stress swelling_stress 1e-16 0
+ beacon_1a01_reference_t_1000.000000.vtu beacon_1a01_dsm_micromacro_smoke_t_1000.000000.vtu sigma sigma 1e-16 1e-10
+)
+
+AddTest(
+ NAME RichardsMechanics_beacon_1a01_dsm_micromacro_inflow_reference
+ PATH RichardsMechanics
+ EXECUTABLE ogs
+ RUNTIME 20
+ EXECUTABLE_ARGS beacon_1a01_dsm_micromacro_inflow.prj
+ WRAPPER time
+ TESTER vtkdiff
+ REQUIREMENTS NOT OGS_USE_MPI
+ DIFF_DATA
+ beacon_1a01_dsm_micromacro_inflow_reference_t_100000.000000.vtu beacon_1a01_dsm_micromacro_inflow_t_100000.000000.vtu displacement displacement 1e-16 0
+ beacon_1a01_dsm_micromacro_inflow_reference_t_100000.000000.vtu beacon_1a01_dsm_micromacro_inflow_t_100000.000000.vtu pressure pressure 1e-16 1e-12
+ beacon_1a01_dsm_micromacro_inflow_reference_t_100000.000000.vtu beacon_1a01_dsm_micromacro_inflow_t_100000.000000.vtu saturation saturation 1e-14 0
+ beacon_1a01_dsm_micromacro_inflow_reference_t_100000.000000.vtu beacon_1a01_dsm_micromacro_inflow_t_100000.000000.vtu micro_pressure micro_pressure 1e-16 1e-12
+ beacon_1a01_dsm_micromacro_inflow_reference_t_100000.000000.vtu beacon_1a01_dsm_micromacro_inflow_t_100000.000000.vtu micro_saturation micro_saturation 1e-14 0
+ beacon_1a01_dsm_micromacro_inflow_reference_t_100000.000000.vtu beacon_1a01_dsm_micromacro_inflow_t_100000.000000.vtu micro_water_content micro_water_content 1e-16 0
+ beacon_1a01_dsm_micromacro_inflow_reference_t_100000.000000.vtu beacon_1a01_dsm_micromacro_inflow_t_100000.000000.vtu micro_porosity micro_porosity 1e-16 0
+ beacon_1a01_dsm_micromacro_inflow_reference_t_100000.000000.vtu beacon_1a01_dsm_micromacro_inflow_t_100000.000000.vtu micro_exchange_source micro_exchange_source 1e-16 0
+ beacon_1a01_dsm_micromacro_inflow_reference_t_100000.000000.vtu beacon_1a01_dsm_micromacro_inflow_t_100000.000000.vtu swelling_stress swelling_stress 1e-16 0
+ beacon_1a01_dsm_micromacro_inflow_reference_t_100000.000000.vtu beacon_1a01_dsm_micromacro_inflow_t_100000.000000.vtu sigma sigma 1e-16 1e-10
+)
+
+AddTest(
+ NAME RichardsMechanics_beacon_1b_dsm_micromacro_smoke
+ PATH RichardsMechanics
+ EXECUTABLE ogs
+ RUNTIME 20
+ EXECUTABLE_ARGS beacon_1b_dsm_micromacro_smoke.prj
+ WRAPPER time
+ REQUIREMENTS NOT OGS_USE_MPI
+)
+
+AddTest(
+ NAME RichardsMechanics_beacon_1b_dsm_micromacro_reference
+ PATH RichardsMechanics
+ EXECUTABLE ogs
+ RUNTIME 20
+ EXECUTABLE_ARGS beacon_1b_dsm_micromacro_smoke.prj
+ WRAPPER time
+ TESTER vtkdiff
+ REQUIREMENTS NOT OGS_USE_MPI
+ DIFF_DATA
+ beacon_1b_reference_t_1000.000000.vtu beacon_1b_dsm_micromacro_smoke_t_1000.000000.vtu displacement displacement 1e-16 0
+ beacon_1b_reference_t_1000.000000.vtu beacon_1b_dsm_micromacro_smoke_t_1000.000000.vtu pressure pressure 1e-16 1e-12
+ beacon_1b_reference_t_1000.000000.vtu beacon_1b_dsm_micromacro_smoke_t_1000.000000.vtu saturation saturation 1e-16 0
+ beacon_1b_reference_t_1000.000000.vtu beacon_1b_dsm_micromacro_smoke_t_1000.000000.vtu micro_pressure micro_pressure 1e-16 1e-12
+ beacon_1b_reference_t_1000.000000.vtu beacon_1b_dsm_micromacro_smoke_t_1000.000000.vtu micro_saturation micro_saturation 1e-16 0
+ beacon_1b_reference_t_1000.000000.vtu beacon_1b_dsm_micromacro_smoke_t_1000.000000.vtu swelling_stress swelling_stress 1e-16 0
+ beacon_1b_reference_t_1000.000000.vtu beacon_1b_dsm_micromacro_smoke_t_1000.000000.vtu sigma sigma 1e-16 1e-10
+)
+
+AddTest(
+ NAME RichardsMechanics_beacon_1c_dsm_micromacro_smoke
+ PATH RichardsMechanics
+ EXECUTABLE ogs
+ RUNTIME 20
+ EXECUTABLE_ARGS beacon_1c_dsm_micromacro_smoke.prj
+ WRAPPER time
+ REQUIREMENTS NOT OGS_USE_MPI
+)
+
+AddTest(
+ NAME RichardsMechanics_beacon_1c_dsm_micromacro_reference
+ PATH RichardsMechanics
+ EXECUTABLE ogs
+ RUNTIME 20
+ EXECUTABLE_ARGS beacon_1c_dsm_micromacro_smoke.prj
+ WRAPPER time
+ TESTER vtkdiff
+ REQUIREMENTS NOT OGS_USE_MPI

```

```

+ DIFF_DATA
+ beacon_1c_reference_t_1000.000000.vtu beacon_1c_dsm_micromacro_smoke_t_1000.000000.vtu displacement displacement 1e-16 0
+ beacon_1c_reference_t_1000.000000.vtu beacon_1c_dsm_micromacro_smoke_t_1000.000000.vtu pressure pressure 1e-16 1e-12
+ beacon_1c_reference_t_1000.000000.vtu beacon_1c_dsm_micromacro_smoke_t_1000.000000.vtu saturation saturation 1e-14 0
+ beacon_1c_reference_t_1000.000000.vtu beacon_1c_dsm_micromacro_smoke_t_1000.000000.vtu porosity porosity 1e-16 0
+ beacon_1c_reference_t_1000.000000.vtu beacon_1c_dsm_micromacro_smoke_t_1000.000000.vtu transport_porosity transport_porosity 1e-16 0
+ beacon_1c_reference_t_1000.000000.vtu beacon_1c_dsm_micromacro_smoke_t_1000.000000.vtu micro_pressure micro_pressure 1e-16 1e-12
+ beacon_1c_reference_t_1000.000000.vtu beacon_1c_dsm_micromacro_smoke_t_1000.000000.vtu micro_saturation micro_saturation 1e-14 0
+ beacon_1c_reference_t_1000.000000.vtu beacon_1c_dsm_micromacro_smoke_t_1000.000000.vtu swelling_stress swelling_stress 1e-16 0
+ beacon_1c_reference_t_1000.000000.vtu beacon_1c_dsm_micromacro_smoke_t_1000.000000.vtu sigma sigma 1e-16 1e-10
+)
+
+
+ AddTest(
+   NAME RichardsMechanics_square_1e2_confined_compression_restart
+   PATH RichardsMechanics
diff --git a/Tests/Data/RichardsMechanics/DSMMicroMacroOverlapTransferBaseline.csv b/Tests/Data/RichardsMechanics/DSMMicroMacroOverlapTransferBaseline.csv
new file mode 100644
index 0000000000..7408152749
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/DSMMicroMacroOverlapTransferBaseline.csv
@@ -0,0 +1,6 @@
+*step,"pressure","S_L","mu_LR","n_L","phi_m","phi_M","phi","n_S","n_L","rho_LR","rho_LR","omega_L","mu_LR","rho_L_hat","delta_epsilon_sw","epsilon_sw","sigma_S_xx","sigma_S_yy","sigma_S_zz","sigma_S_xy","sigma_S_yz","sigma_S_xz"
+0,0,1,0,"0.059376781504527311","0.059316781504527311","0.14068321849547272,0.2,0.8,0.14068321849547272","2193.3876431815676",1000,0.06584254728002587,-0.11767337584908294,0.11767337584908294,-0.00406832184954727,-0.00406832184954727,27122145.663648464,27122145.663648464,27122145.663648464,0,0,0
+1,250,1,0,"0.05936274766696872","0.05936274766696872","0.1406372523338313,0.2,0.8,0.1406372523338313","2193.3876306177184",1000,0.065893477998593624,-0.117400234537006,0.117400234537006,0.000004596616244140899,-0.004063725233383129,27091501.555354193,27091501.555354193,27091501.555354193,0,0,0
+2,500,1,0,"0.059374212533624873","0.059374212533624873","0.14062578746637514,0.2,0.8,0.14062578746637514","2193.3876197482889",1000,0.06590620582163789,-0.11733223945503486,0.11733223945503486,0.00000146466656152322,-0.004062578746637514,27083858.310916755,27083858.310916755,27083858.310916755,0,0,0
+3,750,1,0,"0.059379941646881342","0.059379941646881342","0.14062005835311867,0.2,0.8,0.14062005835311867","2193.3876133501935",1000,0.06591256502516539,-0.1172982812791468,0.1172982812791468,5.72911325646963e-7,-0.004062005835311867,27080038.90207911,27080038.90207911,27080038.90207911,0,0,0
+4,1000,1,0,"0.059382805374452955","0.059382805374452955","0.14061719462554706,0.2,0.8,0.14061719462554706","2193.3876099104677",1000,0.06591574369941794,-0.11728131201736554,0.11728131201736554,2.863725716128506e-7,-0.004061719462554706,27078129.750364702,27078129.750364702,27078129.750364702,0,0,0
diff --git a/Tests/Data/RichardsMechanics/DSMMicroMacroStrainCoupledOverlapBaseline.csv b/Tests/Data/RichardsMechanics/DSMMicroMacroStrainCoupledOverlapBaseline.csv
new file mode 100644
index 0000000000..e0ebcc0862
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/DSMMicroMacroStrainCoupledOverlapBaseline.csv
@@ -0,0 +1,6 @@
+*step,"pressure","epsilon_v_total","delta_epsilon_v","S_L","mu_LR","n_L","phi_m","phi_M","phi","n_S","n_L","rho_LR","rho_LR","omega_L","mu_LR","rho_L_hat","delta_epsilon_sw","epsilon_sw","sigma_S_xx","sigma_S_yy","sigma_S_zz","sigma_S_xy","sigma_S_yz","sigma_S_xz"
+0,1000,0.0001,0.0001,1,0,"0.059448758824633521","0.059448758824633521","0.1405512411753665,0.2,0.8,0.1405512411753665","2193.1397976177655",1000,0.0659814974176539,-0.11689140334452633,0.11689140334452633,0.0000005953450180056698,-0.004055124117536649,27700827.450244326,27700827.450244326,27700827.450244326,0,0,0
+1,1000,0.0002,0.0002,1,0,"0.059510602148738246","0.059510602148738246","0.14048939785126177,0.2,0.8,0.14048939785126177","2193.0380881484352",1000,0.06604707346195905,-0.1165273618308234,0.1165273618308234,0.000000184332410472446,-0.004048939785126176,28326265.234174512,28326265.234174512,28326265.234174512,0,0,0
+2,1000,0.0003,0.0003,1,0,"0.059572284544791592","0.059572284544791592","0.14042771545520844,0.2,0.8,0.14042771545520844","2192.9366609925291",1000,0.06611247305534014,-0.11616577184530422,0.11616577184530422,0.000000168239605334681,-0.004042771545520842,28951810.30347228,28951810.30347228,28951810.30347228,0,0,0
+3,1000,0.0004,0.0004,1,0,"0.059633807109963041","0.059633807109963041","0.14036619289003696,0.2,0.8,0.14036619289003696","2192.8355142031387",1000,0.06617769740782722,2.0,-0.11580660750668251,0.11580660750668251,0.000000152256517144815,-0.004036619289003697,29577461.926691316,29577461.926691316,29577461.926691316,0,0,0
+4,1000,0.0005,0.0005,1,0,"0.059695170929877553","0.059695170929877553","0.14030482907012246,0.2,0.8,0.14030482907012246","2192.7346458540233",1000,0.06624274771666014,-0.11544984380503506,0.11544984380503506,0.00000013681991451217,-0.004030482907012246,30203219.380081642,30203219.380081642,30203219.380081642,0,0,0
diff --git a/Tests/Data/RichardsMechanics/beacon_1a01.gml b/Tests/Data/RichardsMechanics/beacon_1a01.gml
new file mode 100644
index 0000000000..e0a5f64b40
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/beacon_1a01.gml
@@ -0,0 +1,17 @@
+<?xml version="1.0" encoding="ISO-8859-1"?>
+<?xml-stYLESHEET type="text/xml" href="OpenGeoSysGLI.xsl"?>
+<OpenGeoSysGLI xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:ogs="http://www.opengeosys.org">
+  <name>beacon_1a01_geometry</name>
+  <points>
+    <point id="0" x="0.0" y="0.0" z="0.0" name="origin"/>
+    <point id="1" x="0.0" y="0.02" z="0.0"/>
+    <point id="2" x="0.025" y="0.0" z="0.0"/>
+    <point id="3" x="0.025" y="0.02" z="0.0"/>
+  </points>
+  <polyLines>
+    <polyline id="0" name="left"><pnt>0</pnt><pnt>1</pnt></polyline>
+    <polyline id="1" name="right"><pnt>2</pnt><pnt>3</pnt></polyline>
+    <polyline id="2" name="bottom"><pnt>0</pnt><pnt>2</pnt></polyline>
+    <polyline id="3" name="top"><pnt>1</pnt><pnt>3</pnt></polyline>
+  </polyLines>
+</OpenGeoSysGLI>
diff --git a/Tests/Data/RichardsMechanics/beacon_1a01_domain.vtu b/Tests/Data/RichardsMechanics/beacon_1a01_domain.vtu
new file mode 100644
index 0000000000..f0b1859060
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/beacon_1a01_domain.vtu
@@ -0,0 +1,24 @@
+<?xml version="1.0"?>
+<VTKFile type="UnstructuredGrid" version="0.1" byte_order="LittleEndian">
+  <UnstructuredGrid>
+    <Piece NumberOfPoints="4" NumberOfCells="1">
+      <PointData>
+      <CellData>
+        <DataArray type="Int32" Name="MaterialIDs" format="ascii">0</DataArray>
+      </CellData>
+      <Points>
+        <DataArray type="Float64" NumberOfComponents="3" format="ascii">
+          0.0 0.0 0.0
+          0.025 0.0 0.0
+          0.025 0.02 0.0
+          0.0 0.02 0.0
+        </DataArray>
+      </Points>
+      <Cells>
+        <DataArray type="Int32" Name="connectivity" format="ascii">0 1 2 3</DataArray>
+        <DataArray type="Int32" Name="offsets" format="ascii">4</DataArray>
+        <DataArray type="UInt8" Name="types" format="ascii">9</DataArray>
+      </Cells>
+    </Piece>
+  </UnstructuredGrid>
+</VTKFile>
diff --git a/Tests/Data/RichardsMechanics/beacon_1a01_dsm_micromacro_inflow.prj b/Tests/Data/RichardsMechanics/beacon_1a01_dsm_micromacro_inflow.prj
new file mode 100644
index 0000000000..93492d3477
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/beacon_1a01_dsm_micromacro_inflow.prj
@@ -0,0 +1,174 @@
+<?xml version="1.0" encoding="ISO-8859-1"?>
+<OpenGeoSysProject>
+  <mesh axially_symmetric="true">beacon_1a01_domain.vtu</mesh>
+  <geometry>beacon_1a01.gml</geometry>
+  <processes>
+    <process>
+      <name>RM</name>

```

```

+ <type>RICHARDS_MECHANICS</type>
+ <integration_order>2</integration_order>
+ <micro_porosity>
+   <mass_exchange_coefficient>1e-13</mass_exchange_coefficient>
+   <nonlinear_solver>
+     <maximum_iterations>100</maximum_iterations>
+     <residuum_tolerance>1e-8</residuum_tolerance>
+     <increment_tolerance>1e-20</increment_tolerance>
+   </nonlinear_solver>
+ </micro_porosity>
+ <!-- Exploratory inflow probe:
+   keep the macro branch on the saturated helper side (mu_LR = 0)
+   by using positive liquid pressure, and make the attractive vdm
+   microscale term larger by lowering the initial micro water
+   content and using literature-scale clay values. -->
+ <potential_exchange>
+   <enabled>true</enabled>
+   <mode>full_potential</mode>
+   <pressure_tolerance>0.0</pressure_tolerance>
+   <hamaker_constant>5.1e-21</hamaker_constant>
+   <specific_surface>523</specific_surface>
+   <micro_solid_density_reference>2650</micro_solid_density_reference>
+   <micro_solid_volume_fraction_reference>0.6</micro_solid_volume_fraction_reference>
+   <initial_micro_water_content>0.01</initial_micro_water_content>
+   <local_nonlinear_solve_mode>scalar_microstate_storage_mode</local_nonlinear_solve_mode>
+   <fd_jacobian_for_exchange>false</fd_jacobian_for_exchange>
+   <micro_potential_convention>negative_attractive</micro_potential_convention>
+   <micro_water_content_swelling_slope>0.1</micro_water_content_swelling_slope>
+ </potential_exchange>
+ <constitutive_relation>
+   <type>LinearElasticIsotropic</type>
+   <youngs_modulus>E</youngs_modulus>
+   <poissons_ratio>nu</poissons_ratio>
+ </constitutive_relation>
+ <process_variables>
+   <pressure>pressure</pressure>
+   <displacement>displacement</displacement>
+ </process_variables>
+ <secondary_variables>
+   <secondary_variable name="sigma"/>
+   <secondary_variable name="swelling_stress"/>
+   <secondary_variable name="epsilon"/>
+   <secondary_variable name="velocity"/>
+   <secondary_variable name="saturation"/>
+   <secondary_variable name="porosity"/>
+   <secondary_variable name="transport_porosity"/>
+   <secondary_variable name="dry_density_solid"/>
+   <secondary_variable name="micro_saturation"/>
+   <secondary_variable name="micro_pressure"/>
+   <secondary_variable name="micro_water_content"/>
+   <secondary_variable name="micro_porosity"/>
+   <secondary_variable name="micro_exchange_source"/>
+ </secondary_variables>
+ <specific_body_force>0 0</specific_body_force>
+ <initial_stress>sigma0</initial_stress>
+ <mass_lumping>true</mass_lumping>
+ </process>
+ </processes>
+ <media>
+   <medium id="0">
+     <phases>
+       <phase>
+         <type>AqueousLiquid</type>
+         <properties>
+           <property<name>viscosity</name><type>Constant</type><value>1e-3</value></property>
+           <property<name>density</name><type>Constant</type><value>1e3</value></property>
+         </properties>
+       </phase>
+       <phase>
+         <type>Solid</type>
+         <properties>
+           <property<name>density</name><type>Constant</type><value>2780</value></property>
+           <property>
+             <name>swelling_stress_rate</name>
+             <type>SaturationDependentSwelling</type>
+             <swelling_pressures>1.3e7 1.3e7 1.3e7</swelling_pressures>
+             <exponents>1 1 1</exponents>
+             <lower_saturation_limit>0</lower_saturation_limit>
+             <upper_saturation_limit>1</upper_saturation_limit>
+           </property>
+         </properties>
+       </phase>
+     </phases>
+     <properties>
+       <property<name>transport_porosity</name><type>TransportPorosityFromMassBalance</type><initial_porosity>phi_tr0</initial_porosity><minimal_porosity>0</minimal_porosity><maximal_porosity>1</maximal_porosity></property>
+       <property<name>biot_coefficient</name><type>Constant</type><value>0.6</value></property>
+       <property<name>permeability</name><type>Constant</type><value>1e-18</value></property>
+       <property<name>porosity</name><type>PorosityFromMassBalance</type><initial_porosity>phi0</initial_porosity><minimal_porosity>0</minimal_porosity><maximal_porosity>1</maximal_porosity></property>
+       <property<name>reference_temperature</name><type>Constant</type><value>293.15</value></property>
+       <property<name>relative_permeability</name><type>Constant</type><value>1</value></property>
+       <property<name>saturation</name><type>SaturationVanGenuchten</type><residual_liquid_saturation>0.0</residual_liquid_saturation><residual_gas_saturation>0</residual_gas_saturation><exponent>0.5</exponent><p_b>1e6</p_b></property>
+       <property<name>saturation_micro</name><type>SaturationVanGenuchten</type><residual_liquid_saturation>0</residual_liquid_saturation><residual_gas_saturation>0</residual_gas_saturation><exponent>0.5</exponent><p_b>1e7</p_b></property>
+       <property<name>bishops_effective_stress</name><type>BishopsPowerLaw</type><exponent>1</exponent></property>
+     </properties>
+   </medium>
+ </media>
+ <time_loop>
+   <processes>
+     <process ref="RM">
+       <nonlinear_solver>nonlinear_solver</nonlinear_solver>
+       <convergence_criterion>
+         <norm_type>NORM2</norm_type>
+         <type>PerComponentDeltaX</type>
+         <abstols>5e-8 1e-13 1e-13</abstols>
+       </convergence_criterion>
+       <time_discretization><type>BackwardEuler</type></time_discretization>
+       <time_stepping>
+         <type>IterationNumberBasedTimeStepping</type>
+         <t_initial>0.0</t_initial>
+         <t_end>1e5</t_end>
+         <initial_dt>10</initial_dt>
+         <minimum_dt>0.1</minimum_dt>
+         <maximum_dt>2e3</maximum_dt>

```

```

+         <number_iterations>1 2 3 4 5 6 7 8 9 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25</number_iterations>
+         <multiplier>1.225 1.2 1.175 1.15 1.125 1.1 1.075 1.05 1.025 0.975 0.95 0.925 0.9 0.875 0.85 0.825 0.8 0.775 0.75 0.725 0.7 0.675 0.65 0.625</multiplier>
+         </time_stepping>
+     </process>
+ </processes>
+ <output>
+     <type>VTK</type>
+     <prefix>beacon_1a01_dsm_micromacro_inflow</prefix>
+     <suffix>_t(:time)</suffix>
+     <fixed_output_times>1e3 1e4 5e4 1e5</fixed_output_times>
+     <variables><variable>pressure</variable><variable>displacement</variable><variable>saturation</variable><variable>micro_pressure</variable><variable>micro_saturation</variable><variable>micro_water_content</variable><variable>micro_porosity</variable><variable>micro_exchange_source</variable><variable>swelling_stress</variable><variable>sigma</variable></variables>
+ </output>
+ </time_loop>
+ <parameters>
+     <parameter><name>sigma</name><type>Function</type><expression>0</expression><expression>0</expression><expression>0</expression><expression>0</expression></parameter>
+     <parameter><name>E</name><type>Constant</type><value>50e6</value></parameter>
+     <parameter><name>nu</name><type>Constant</type><value>0.2</value></parameter>
+     <parameter><name>phi0</name><type>Constant</type><value>0.4</value></parameter>
+     <parameter><name>phi_tr0</name><type>Constant</type><value>0.3</value></parameter>
+     <parameter><name>displacement0</name><type>Constant</type><values>0 0</values></parameter>
+     <parameter><name>pressure_ic</name><type>Constant</type><values>2e3</values></parameter>
+     <parameter><name>dirichlet0</name><type>Constant</type><value>0</value></parameter>
+     <parameter><name>top_pressure</name><type>Constant</type><value>2e3</value></parameter>
+ </parameters>
+ <process_variables>
+     <process_variable>
+         <name>displacement</name>
+         <components>2</components>
+         <order>1</order>
+         <initial_condition>displacement0</initial_condition>
+         <boundary_conditions>
+             <boundary_condition><geometrical_set>beacon_1a01_geometry</geometrical_set><geometry>left</geometry><type>Dirichlet</type><component>0</component><parameter>dirichlet0</parameter></boundary_condition>
+             <boundary_condition><geometrical_set>beacon_1a01_geometry</geometrical_set><geometry>right</geometry><type>Dirichlet</type><component>0</component><parameter>dirichlet0</parameter></boundary_condition>
+             <boundary_condition><geometrical_set>beacon_1a01_geometry</geometrical_set><geometry>bottom</geometry><type>Dirichlet</type><component>1</component><parameter>dirichlet0</parameter></boundary_condition>
+             <boundary_condition><geometrical_set>beacon_1a01_geometry</geometrical_set><geometry>top</geometry><type>Dirichlet</type><component>1</component><parameter>dirichlet0</parameter></boundary_condition>
+         </boundary_conditions>
+     </process_variable>
+     <process_variable>
+         <name>pressure</name>
+         <components>1</components>
+         <order>1</order>
+         <initial_condition>pressure_ic</initial_condition>
+         <boundary_conditions>
+             <boundary_condition><geometrical_set>beacon_1a01_geometry</geometrical_set><geometry>top</geometry><type>Dirichlet</type><component>0</component><parameter>top_pressure</parameter></boundary_condition>
+         </boundary_conditions>
+     </process_variable>
+ </process_variables>
+ <nonlinear_solvers>
+     <nonlinear_solver><name>nonlinear_solver</name><type>Newton</type><max_iter>30</max_iter><linear_solver>general_linear_solver</linear_solver></nonlinear_solver>
+ </nonlinear_solvers>
+ <linear_solvers>
+     <linear_solver><name>general_linear_solver</name><eigen>csolver_type>SparseLU</solver_type><scaling>true</scaling></eigen></linear_solver>
+ </linear_solvers>
+ </OpenGeoSysProject>
diff --git a/Tests/Data/RichardsMechanics/beacon_1a01_dsm_micromacro_inflow_reference_t_100000.000000.vtu b/Tests/Data/RichardsMechanics/beacon_1a01_dsm_micromacro_inflow_reference_t_100000.000000.vtu
new file mode 100644
index 0000000000..43ca2acd92
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/beacon_1a01_dsm_micromacro_inflow_reference_t_100000.000000.vtu
@@ -0,0 +1,35 @@
<?xml version="1.0"?>
<VTKFile type="UnstructuredGrid" version="1.0" byte_order="LittleEndian" header_types="UInt64" compressor="vtkZLibDataCompressor">
+ <UnstructuredGrid>
+     <FieldData>
+         <DataArray type="Int8" Name="OGS_VERSION" NumberOfTuples="19" format="appended" RangeMin="45" RangeMax="103" offset="0" />
+     </FieldData>
+     <Piece NumberOfPoints="4" NumberOfCells="1" >
+         <PointData>
+             <DataArray type="Float64" Name="displacement" NumberOfComponents="2" format="appended" RangeMin="0" RangeMax="0" offset="80" />
+             <DataArray type="Float64" Name="micro_exchange_source" format="appended" RangeMin="1.083244732e-05" RangeMax="1.0836045685e-05" offset="140" />
+             <DataArray type="Float64" Name="micro_porosity" format="appended" RangeMin="0.011281449057" RangeMax="0.011282698091" offset="224" />
+             <DataArray type="Float64" Name="micro_pressure" format="appended" RangeMin="108324.4732" RangeMax="108360.45685" offset="308" />
+             <DataArray type="Float64" Name="micro_saturation" format="appended" RangeMin="1.1281449057" RangeMax="1.1282698091" offset="392" />
+             <DataArray type="Float64" Name="micro_water_content" format="appended" RangeMin="0.011281449057" RangeMax="0.011282698091" offset="476" />
+             <DataArray type="Float64" Name="pressure" format="appended" RangeMin="-170.58939149" RangeMax="2000" offset="560" />
+             <DataArray type="Float64" Name="saturation" format="appended" RangeMin="1" RangeMax="1" offset="640" />
+             <DataArray type="Float64" Name="sigma" NumberOfComponents="4" format="appended" RangeMin="6165.3746518" RangeMax="6171.3840692" offset="720" />
+         </PointData>
+         <CellData>
+             <Points>
+                 <DataArray type="Float64" Name="Points" NumberOfComponents="3" format="appended" RangeMin="0" RangeMax="0.032015621187" offset="944" />
+             </Points>
+             <Cells>
+                 <DataArray type="Int64" Name="connectivity" format="appended" RangeMin="" RangeMax="" offset="1028" />
+                 <DataArray type="Int64" Name="offsets" format="appended" RangeMin="" RangeMax="" offset="1100" />
+                 <DataArray type="UInt8" Name="types" format="appended" RangeMin="" RangeMax="" offset="1160" />
+             </Cells>
+         </Piece>
+     </UnstructuredGrid>
+     <AppendedData encoding="base64">
+         _AQAAAAAAAAAAGAAAAAAAAABMAAAAAAAAAAGwAAAAAAAAA=ef4z0PVM9c1t0DTTc2SUb9t0xJBAAQMQTCAQAAAAAAAAAAGAAAAAAAAAEEAAAAAAAAADAAAAAAAAA=EF5JYKMAAAQAQAQAQAAAAAAAAAAGAAAAAAAAACAAAAAAAAAHgAAAAAAAAA=ef7r0EdK317y0EP5c+0mp3KvTL4R0UBg1TXQ+6AQAAAAAAAAAAGAAAAAAAAACAAAAAAAAAHgAAAAAAAAA=ef6LdPZ85Dnk30U106012VJCuBn+x10GgAFd0//AQAAAAAAAAAAGAAAAAAAAACAAAAAAAAAHgAAAAAAAAA=EF5TOMFatkUq3V4J5h;qZBKm633wLAczDE0=AQAAAAAAAAAAGAAAAAAAAACAAAAAAAAAHgAAAAAAAAA=ef779r8E3tv/h9JMDCDwrf4XLABak1tMpA==AQAAAAAAAAAAGAAAAAAAAAIAAAAAAAAAAAGAAAAAAAAA=ef5JXVIarH+9QFW0W0pRnggB3Kh9HcaPT/XK/vkry2+gC/g6lYfLvoX08G1YPAF7dIdo=AQAAAAAAAAAAGAAAAAAAAAIAAAAAAAAAALWAAAAAAAAA=EF7JWFarH1+9QeUNJoBcRihHq0a3v/d9fouyMurD0DTMPC3UD460ZPHAG0Tney=AQAAAAAAAAAAGAAAAAAAAAAGAAAAAAAAAHgAAAAAAAAA=EF5JYMA0Zs0EA3tc4TU169vFVK3BKEcXBwCsHBDQAQAAAAAAAAAAGAAAAAAAAACAAAAAAAAA=EF5JYTAARjiNBKMWZQTQ0HAABw==AQAAAAAAAAAAGAAAAAAAAACAAAAAAAAA=ef5JYAAAAAAAU=AQAAAAAAAAAAGAAAAAAAAAEEAAAAAAAAACQAAAAAAAAA=EF7JBAACgAK
+     </AppendedData>
+ </VTKFile>
diff --git a/Tests/Data/RichardsMechanics/beacon_1a01_dsm_micromacro_inflowprobe.prj b/Tests/Data/RichardsMechanics/beacon_1a01_dsm_micromacro_inflowprobe.prj
new file mode 100644
index 0000000000..e79422da88

```

```

--- /dev/null
+++ b/Tests/Data/RichardsMechanics/beacon_1a01_dsm_micromacro_inflowprobe.prj
@@ -0,0 +1,179 @@
+<?xml version="1.0" encoding="ISO-8859-1"?>
+<OpenGeoSysProject>
+  <mesh axially_symmetric="true">beacon_1a01_domain.vtu</mesh>
+  <geometry>beacon_1a01.gml</geometry>
+  <processes>
+    <process>
+      <name>RM</name>
+      <type>RICHARDS_MECHANICS</type>
+      <integration_order>2</integration_order>
+      <micro_porosity>
+        <mass_exchange_coefficient>1e-13</mass_exchange_coefficient>
+        <nonlinear_solver>
+          <maximum_iterations>100</maximum_iterations>
+          <residuum_tolerance>1e-8</residuum_tolerance>
+          <increment_tolerance>1e-20</increment_tolerance>
+        </nonlinear_solver>
+      </micro_porosity>
+      <potential_exchange>
+        <enabled>true</enabled>
+        <mode>full_potential</mode>
+        <pressure_tolerance>0.0</pressure_tolerance>
+        <hamaker_constant>6e-20</hamaker_constant>
+        <specific_surface>4000</specific_surface>
+        <micro_solid_density_reference>2650</micro_solid_density_reference>
+        <micro_solid_volume_fraction_reference>0.6</micro_solid_volume_fraction_reference>
+        <!-- Deliberate unsaturated contrast case:
+          keep the microscale hydrated enough that the attractive
+          vdW term stays moderate, but drive the macro reduced
+          potential more negative so the exchange stays on the
+          drainage side and n_l decreases. -->
+        <initial_micro_water_content>0.2</initial_micro_water_content>
+        <local_nonlinear_solve_mode>scalar_micro_macro_mass_storage_mode</local_nonlinear_solve_mode>
+        <macro_porosity_update_mode>additive_macro_porosity_rate_mode</macro_porosity_update_mode>
+        <micro_solid_volume_fraction_mode>current_porosity_split</micro_solid_volume_fraction_mode>
+        <micro_liquid_density_reference>1300</micro_liquid_density_reference>
+        <micro_liquid_density_a>1.3</micro_liquid_density_a>
+        <micro_liquid_density_b>1.0</micro_liquid_density_b>
+        <fd_jacobian_for_exchange>false</fd_jacobian_for_exchange>
+        <micro_potential_convention>negative_attractive</micro_potential_convention>
+        <micro_water_content_swelling_slope>0.1</micro_water_content_swelling_slope>
+      </potential_exchange>
+      <constitutive_relation>
+        <type>LinearElasticIsotropic</type>
+        <youngs_modulus>E</youngs_modulus>
+        <poissons_ratio>nu</poissons_ratio>
+      </constitutive_relation>
+      <process_variables>
+        <pressure>pressure</pressure>
+        <displacement>displacement</displacement>
+      </process_variables>
+      <secondary_variables>
+        <secondary_variable name="sigma"/>
+        <secondary_variable name="swelling_stress"/>
+        <secondary_variable name="epsilon"/>
+        <secondary_variable name="velocity"/>
+        <secondary_variable name="saturation"/>
+        <secondary_variable name="porosity"/>
+        <secondary_variable name="transport_porosity"/>
+        <secondary_variable name="dry_density_solid"/>
+        <secondary_variable name="micro_saturation"/>
+        <secondary_variable name="micro_pressure"/>
+        <secondary_variable name="micro_water_content"/>
+        <secondary_variable name="micro_porosity"/>
+        <secondary_variable name="micro_exchange_source"/>
+      </secondary_variables>
+      <specific_body_force>0</specific_body_force>
+      <initial_stress>sigma0</initial_stress>
+      <mass_lumping>true</mass_lumping>
+    </process>
+  </processes>
+  <media>
+    <medium id="0">
+      <phases>
+        <phase>
+          <type>AqueousLiquid</type>
+          <properties>
+            <property><name>viscosity</name><type>Constant</type><value>1e-3</value></property>
+            <property><name>density</name><type>Constant</type><value>1e3</value></property>
+          </properties>
+        </phase>
+        <phase>
+          <type>Solid</type>
+          <properties>
+            <property><name>density</name><type>Constant</type><value>2780</value></property>
+            <property>
+              <name>swelling_stress_rate</name>
+              <type>SaturationDependentSwelling</type>
+              <swelling_pressures>1.3e7 1.3e7 1.3e7</swelling_pressures>
+              <exponents>1 1</exponents>
+              <lower_saturation_limit>0</lower_saturation_limit>
+              <upper_saturation_limit>1</upper_saturation_limit>
+            </property>
+          </properties>
+        </phase>
+      </phases>
+      <properties>
+        <property><name>transport_porosity</name><type>TransportPorosityFromMassBalance</type><initial_porosity>phi_tr0</initial_porosity><minimal_porosity>0</minimal_porosity><maximal_porosity>1</maximal_porosity></property>
+        <property><name>biot_coefficient</name><type>Constant</type><value>0.6</value></property>
+        <property><name>permeability</name><type>Constant</type><value>1e-18</value></property>
+        <property><name>porosity</name><type>PorosityFromMassBalance</type><initial_porosity>phi0</initial_porosity><minimal_porosity>0</minimal_porosity><maximal_porosity>1</maximal_porosity></property>
+        <property><name>reference_temperature</name><type>Constant</type><value>293.15</value></property>
+        <property><name>relative_permeability</name><type>Constant</type><value>1</value></property>
+        <property><name>saturation</name><type>SaturationVanGenuchten</type><residual_liquid_saturation>0.0</residual_liquid_saturation><residual_gas_saturation>0</residual_gas_saturation><exponent>0.5</exponent><p_b>1e6</p_b></property>
+        <property><name>saturation_micro</name><type>SaturationVanGenuchten</type><residual_liquid_saturation>0</residual_liquid_saturation><residual_gas_saturation>0</residual_gas_saturation><exponent>0.5</exponent><p_b>1e7</p_b></property>
+        <property><name>bishops_effective_stress</name><type>BishopsPowerLaw</type><exponent>1</exponent></property>
+      </properties>
+    </medium>
+  </media>
+  <time_loop>
+    <processes>

```

```

+
+   <process ref="RM">
+     <nonlinear_solver>nonlinear_solvers/nonlinear_solver<
+     <convergence_criterion>
+       <norm_type>NORM2</norm_type>
+       <type>PerComponentDeltaX</type>
+       <abstols>5e-8 1e-13 1e-13</abstols>
+     </convergence_criterion>
+     <time_discretization><type>BackwardEuler</type></time_discretization>
+     <time_stepping>
+       <type>IterationNumberBasedTimeStepping</type>
+       <t_initial>0.0</t_initial>
+       <t_end>1e4</t_end>
+       <initial_dt>10</initial_dt>
+       <minimum_dt>0.1</minimum_dt>
+       <maximum_dt>2e3</maximum_dt>
+       <number_iterations>1 2 3 4 5 6 7 8 9 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25</number_iterations>
+       <multiplier>1.225 1.2 1.175 1.15 1.125 1.1 1.075 1.05 1.025 0.975 0.95 0.925 0.9 0.875 0.85 0.825 0.8 0.775 0.75 0.725 0.7 0.675 0.65 0.625</multiplier>
+     </time_stepping>
+   </process>
+ </processes>
+ <output>
+   <type>VTK</type>
+   <prefix>beacon_1a01_dsm_micromacro_inflowprobe</prefix>
+   <suffix>_t_{:time}</suffix>
+   <fixed_output_times>1e2 1e3 5e3 1e4</fixed_output_times>
+   <variables><variable>pressure</variable><variable>displacement</variable><variable>saturation</variable><variable>micro_pressure</variable><variable>micro_saturation</variable><variable>micro_water_content</variable><variable>micro_porosity</variable><variable>micro_exchange_source</variable><variable>swelling_stress</variable><variable>sigma</variable></variables>
+ </output>
+ </time_loop>
+ <parameters>
+   <parameter><name>sigma</name><type>Function</type><expression>0</expression><expression>0</expression><expression>0</expression><expression>0</expression></parameter>
+   <parameter><name>E</name><type>Constant</type><value>50e6</value></parameter>
+   <parameter><name>nu</name><type>Constant</type><value>0.2</value></parameter>
+   <parameter><name>phi0</name><type>Constant</type><value>0.4</value></parameter>
+   <parameter><name>phi_tr0</name><type>Constant</type><value>0.3</value></parameter>
+   <parameter><name>displacement0</name><type>Constant</type><values>0 0</values></parameter>
+   <parameter><name>pressure_ic</name><type>Constant</type><values>-3e5</values></parameter>
+   <parameter><name>dirichlet0</name><type>Constant</type><value>0</value></parameter>
+   <parameter><name>top_pressure</name><type>Constant</type><value>-2e5</value></parameter>
+ </parameters>
+ <process_variables>
+   <process_variable>
+     <name>displacement</name>
+     <components>2</components>
+     <order>1</order>
+     <initial_condition>displacement0</initial_condition>
+     <boundary_conditions>
+       <boundary_condition><geometrical_set>beacon_1a01_geometry</geometrical_set><geometry>left</geometry><type>Dirichlet</type><component>0</component><parameter>dirichlet0</parameter></boundary_condition>
+       <boundary_condition><geometrical_set>beacon_1a01_geometry</geometrical_set><geometry>right</geometry><type>Dirichlet</type><component>0</component><parameter>dirichlet0</parameter></boundary_condition>
+       <boundary_condition><geometrical_set>beacon_1a01_geometry</geometrical_set><geometry>bottom</geometry><type>Dirichlet</type><component>1</component><parameter>dirichlet0</parameter></boundary_condition>
+       <boundary_condition><geometrical_set>beacon_1a01_geometry</geometrical_set><geometry>top</geometry><type>Dirichlet</type><component>1</component><parameter>dirichlet0</parameter></boundary_condition>
+     </boundary_conditions>
+   </process_variable>
+   <process_variable>
+     <name>pressure</name>
+     <components>1</components>
+     <order>1</order>
+     <initial_condition>pressure_ic</initial_condition>
+     <boundary_conditions>
+       <boundary_condition><geometrical_set>beacon_1a01_geometry</geometrical_set><geometry>top</geometry><type>Dirichlet</type><component>0</component><parameter>top_pressure</parameter></boundary_condition>
+     </boundary_conditions>
+   </process_variable>
+ </process_variables>
+ <nonlinear_solvers>
+   <nonlinear_solver><name>nonlinear_solver</name><type>Newton</type><max_iter>30</max_iter><linear_solver>general_linear_solver</linear_solver></nonlinear_solver>
+ </nonlinear_solvers>
+ <linear_solvers>
+   <linear_solver><name>general_linear_solver</name><eigen>solver_type>SparseLU</solver_type><scaling>true</scaling></eigen></linear_solver>
+ </linear_solvers>
+ </OpenGeoSysProject>
diff --git a/Tests/Data/RichardsMechanics/beacon_1a01_dsm_micromacroSmoke.prj b/Tests/Data/RichardsMechanics/beacon_1a01_dsm_micromacroSmoke.prj
new file mode 100644
index 0000000000..f9270bd0a2
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/beacon_1a01_dsm_micromacroSmoke.prj
@@ -0,0 +1,159 @@
<?xml version="1.0" encoding="ISO-8859-1"?>
<OpenGeoSysProject>
+   <mesh axially_symmetric="true">beacon_1a01_domain.vtu</mesh>
+   <geometry>beacon_1a01.gml</geometry>
+   <processes>
+     <process>
+       <name>RM</name>
+       <type>RICHARDS_MECHANICS</type>
+       <integration_order>2</integration_order>
+       <micro_porosity>
+         <mass_exchange_coefficient>1e-13</mass_exchange_coefficient>
+         <nonlinear_solver>
+           <maximum_iterations>100</maximum_iterations>
+           <residuum_tolerance>1e-8</residuum_tolerance>
+           <increment_tolerance>1e-20</increment_tolerance>
+         </nonlinear_solver>
+       </micro_porosity>
+       <potential_exchange>
+         <enabled>true</enabled>
+         <mode>full_potential</mode>
+         <pressure_tolerance>0.0</pressure_tolerance>
+         <hamaker_constant>1e-30</hamaker_constant>
+         <specific_surface>1.0</specific_surface>
+         <micro_solid_density_reference>2050</micro_solid_density_reference>
+         <micro_solid_volume_fraction_reference>0.6</micro_solid_volume_fraction_reference>
+         <initial_micro_water_content>0.1</initial_micro_water_content>
+         <fd_jacobian_for_exchange>false</fd_jacobian_for_exchange>
+       </potential_exchange>
+       <constitutive_relation>
+         <type>LinearElasticIsotropic</type>
+         <youngs_modulus>E</youngs_modulus>
+         <poissons_ratio>nu</poissons_ratio>
+       </constitutive_relation>
+     </process>
+     <process_variables>
+       <pressure>pressure</pressure>

```

```

+         <displacement>displacement</displacement>
+     </process_variables>
+     <secondary_variables>
+         <secondary_variable name="sigma"/>
+         <secondary_variable name="swelling_stress"/>
+         <secondary_variable name="epsilon"/>
+         <secondary_variable name="velocity"/>
+         <secondary_variable name="saturation"/>
+         <secondary_variable name="porosity"/>
+         <secondary_variable name="transport_porosity"/>
+         <secondary_variable name="dry_density_solid"/>
+         <secondary_variable name="micro_saturation"/>
+         <secondary_variable name="micro_pressure"/>
+     </secondary_variables>
+     <specific_body_force>0 0</specific_body_force>
+     <initial_stress>sigma0</initial_stress>
+     <mass_lumping>true</mass_lumping>
+ </process>
+ </processes>
+ <media>
+     <medium id="0">
+         <phases>
+             <phase>
+                 <type>AqueousLiquid</type>
+                 <properties>
+                     <property<name>viscosity</name><type>Constant</type><value>1e-3</value></property>
+                     <property<name>density</name><type>Constant</type><value>1e3</value></property>
+                 </properties>
+             </phase>
+             <phase>
+                 <type>Solid</type>
+                 <properties>
+                     <property<name>density</name><type>Constant</type><value>2780</value></property>
+                     <property>
+                         <name>swelling_stress_rate</name>
+                         <type>SaturationDependentSwelling</type>
+                         <swelling_pressures>1.3e7 1.3e7 1.3e7</swelling_pressures>
+                         <exponents>1 1 1</exponents>
+                         <lower_saturation_limit>0</lower_saturation_limit>
+                         <upper_saturation_limit>1</upper_saturation_limit>
+                     </property>
+                 </properties>
+             </phase>
+         </phases>
+         <properties>
+             <property<name>transport_porosity</name><type>TransportPorosityFromMassBalance</type><initial_porosity>phi_tr0</initial_porosity><minimal_porosity>0</minimal_porosity><maximal_porosity>1</maximal_porosity></property>
+             <property<name>biot_coefficient</name><type>Constant</type><value>0.6</value></property>
+             <property<name>permeability</name><type>Constant</type><value>1e-18</value></property>
+             <property<name>porosity</name><type>PorosityFromMassBalance</type><initial_porosity>phi0</initial_porosity><minimal_porosity>0</minimal_porosity><maximal_porosity>1</maximal_porosity></property>
+             <property<name>reference_temperature</name><type>Constant</type><value>293.15</value></property>
+             <property<name>relative_permeability</name><type>Constant</type><value>1</value></property>
+             <property<name>saturations</name><type>SaturationVanGenuchten</type><residual_liquid_saturation>0.0</residual_liquid_saturation><residual_gas_saturation>0</residual_gas_saturation><exponent>0.5</exponent><p_b>1e6</p_b></property>
+             <property<name>saturation_micro</name><type>SaturationVanGenuchten</type><residual_liquid_saturation>0</residual_liquid_saturation><residual_gas_saturation>0</residual_gas_saturation><exponent>0.5</exponent><p_b>1e7</p_b></property>
+             <property<name>bishops_effective_stress</name><type>BishopsPowerLaw</type><exponent>1</exponent></property>
+         </properties>
+     </medium>
+ </media>
+ <time_loop>
+     <processes>
+         <process ref="RM">
+             <nonlinear_solver>nonlinear_solver</nonlinear_solver>
+             <convergence_criterion>
+                 <norm_type>NORM2</norm_type>
+                 <type>PerComponentDeltaX</type>
+                 <abstols>5e-8 1e-13 1e-13</abstols>
+             </convergence_criterion>
+             <time_discretization><type>BackwardEuler</type></time_discretization>
+             <time_stepping>
+                 <type>FixedTimeStepping</type>
+                 <t_initial>0.0</t_initial>
+                 <t_end>1e3</t_end>
+                 <timesteps><pair><repeat>20</repeat><delta_t>50</delta_t></pair></timesteps>
+             </time_stepping>
+         </process>
+     </processes>
+     <output>
+         <type>VTK</type>
+         <prefix>beacon_1a01_dsm_micromacro_smoke</prefix>
+         <suffix>_t_{:time}</suffix>
+         <fixed_output_times>1e3</fixed_output_times>
+         <variables><variable>pressure</variable><variable>displacement</variable><variable>saturation</variable><variable>micro_pressure</variable><variable>micro_saturation</variable><variable>swelling_stress</variable><variable>sigma</variable></variables>
+     </output>
+ </time_loop>
+ <parameters>
+     <parameter><name>sigma0</name><type>Function</type><expression>0</expression><expression>0</expression><expression>0</expression><expression>0</expression></parameter>
+     <parameter><name>E</name><type>Constant</type><value>50e6</value></parameter>
+     <parameter><name>nu</name><type>Constant</type><value>0.2</value></parameter>
+     <parameter><name>phi0</name><type>Constant</type><value>0.4</value></parameter>
+     <parameter><name>phi_tr0</name><type>Constant</type><value>0.3</value></parameter>
+     <parameter><name>displacement0</name><type>Constant</type><values>0 0</values></parameter>
+     <parameter><name>pressure_ic</name><type>Constant</type><values>1e6</values></parameter>
+     <parameter><name>dirichlet0</name><type>Constant</type><value>0</value></parameter>
+     <parameter><name>top_pressure</name><type>Constant</type><value>2e3</value></parameter>
+ </parameters>
+ <process_variables>
+     <process_variable>
+         <name>displacement</name>
+         <components>2</components>
+         <order>1</order>
+         <initial_condition>displacement0</initial_condition>
+         <boundary_conditions>
+             <boundary_condition>geometrical_set=beacon_1a01_geometry</geometrical_set><geometry>left</geometry><type>Dirichlet</type><component>0</component><parameter>dirichlet0</parameter></boundary_condition>
+             <boundary_condition>geometrical_set=beacon_1a01_geometry</geometrical_set><geometry>right</geometry><type>Dirichlet</type><component>0</component><parameter>dirichlet0</parameter></boundary_condition>
+             <boundary_condition>geometrical_set=beacon_1a01_geometry</geometrical_set><geometry>bottom</geometry><type>Dirichlet</type><component>1</component><parameter>dirichlet0</parameter></boundary_condition>
+             <boundary_condition>geometrical_set=beacon_1a01_geometry</geometrical_set><geometry>top</geometry><type>Dirichlet</type><component>1</component><parameter>dirichlet0</parameter></boundary_condition>
+         </boundary_conditions>
+     </process_variable>
+ </process_variables>

```

```

+       <name>pressure</name>
+       <components>1</components>
+       <order>1</order>
+       <initial_condition>pressure_ic</initial_condition>
+       <boundary_conditions>
+       <boundary_condition><geometrical_set>beacon_1a01_geometry</geometrical_set><geometry>top</geometry><type>Dirichlet</type><component>0</component><parameter>top_pre
ssure</parameter></boundary_condition>
+       </boundary_conditions>
+       </process_variable>
+       </process_variables>
+       <nonlinear_solvers>
+       <nonlinear_solver><name>nonlinear_solver</name><type>Newton</type><max_iter>30</max_iter><linear_solver>general_linear_solver</linear_solver></nonlinear_solver>
+       </nonlinear_solvers>
+       <linear_solvers>
+       <linear_solver><name>general_linear_solver</name><eigen><solver_type>SparseLU</solver_type><scaling>true</scaling></eigen></linear_solver>
+       </linear_solvers>
+</OpenGeoSysProject>
diff --git a/Tests/Data/RichardsMechanics/beacon_1a01_dsm_micromacro_stressprobe.prj b/Tests/Data/RichardsMechanics/beacon_1a01_dsm_micromacro_stressprobe.prj
new file mode 100644
index 0000000000..e7055bf33b
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/beacon_1a01_dsm_micromacro_stressprobe.prj
@@ -0,0 +1,174 @@
<?xml version="1.0" encoding="ISO-8859-1"?>
<OpenGeoSysProject>
+   <mesh axially_symmetric="true">beacon_1a01_domain.vtu</mesh>
+   <geometry>beacon_1a01.gml</geometry>
+   <processes>
+       <process>
+           <name>RM</name>
+           <type>RICHARDS_MECHANICS</type>
+           <integration_order>2</integration_order>
+           <micro_porosity>
+               <mass_exchange_coefficient>1e-13</mass_exchange_coefficient>
+               <nonlinear_solver>
+                   <maximum_iterations>100</maximum_iterations>
+                   <residuum_tolerance>1e-8</residuum_tolerance>
+                   <increment_tolerance>1e-20</increment_tolerance>
+               </nonlinear_solver>
+           </micro_porosity>
+           <potential_exchange>
+               <enabled>true</enabled>
+               <mode>full_potential</mode>
+               <pressure_tolerance>0.0</pressure_tolerance>
+               <hamaker_constant>6e-20</hamaker_constant>
+               <specific_surface>100</specific_surface>
+               <micro_solid_density_reference>2650</micro_solid_density_reference>
+               <micro_solid_volume_fraction_reference>0.6</micro_solid_volume_fraction_reference>
+               <initial_micro_water_content>0.1</initial_micro_water_content>
+               <local_nonlinear_solve_mode>scalar_micro_macro_mass_storage_mode</local_nonlinear_solve_mode>
+               <macro_porosity_update_mode>additive_macro_porosity_rate_mode</macro_porosity_update_mode>
+               <micro_solid_volume_fraction_mode>current_porosity_split</micro_solid_volume_fraction_mode>
+               <micro_liquid_density_reference>1300</micro_liquid_density_reference>
+               <micro_liquid_density_a>1.3</micro_liquid_density_a>
+               <micro_liquid_density_b>1.0</micro_liquid_density_b>
+               <fd_jacobian_for_exchange>false</fd_jacobian_for_exchange>
+               <micro_potential_convention>negative_attractive</micro_potential_convention>
+               <micro_water_content_swelling_slope>0.1</micro_water_content_swelling_slope>
+           </potential_exchange>
+           <constitutive_relation>
+               <type>LinearElasticIsotropic</type>
+               <youngs_modulus>E</youngs_modulus>
+               <poissons_ratio>nu</poissons_ratio>
+           </constitutive_relation>
+           <process_variables>
+               <pressure>pressure</pressure>
+               <displacement>displacement</displacement>
+           </process_variables>
+           <secondary_variables>
+               <secondary_variable name="sigma"/>
+               <secondary_variable name="swelling_stress"/>
+               <secondary_variable name="epsilon"/>
+               <secondary_variable name="velocity"/>
+               <secondary_variable name="saturation"/>
+               <secondary_variable name="porosity"/>
+               <secondary_variable name="transport_porosity"/>
+               <secondary_variable name="dry_density_solid"/>
+               <secondary_variable name="micro_saturation"/>
+               <secondary_variable name="micro_pressure"/>
+               <secondary_variable name="micro_water_content"/>
+               <secondary_variable name="micro_porosity"/>
+               <secondary_variable name="micro_exchange_source"/>
+           </secondary_variables>
+           <specific_body_force>0</specific_body_force>
+           <initial_stress>sigma0</initial_stress>
+           <mass_lumping>true</mass_lumping>
+       </process>
+   </processes>
+   <media>
+       <medium id="0">
+           <phases>
+               <phase>
+                   <type>AqueousLiquid</type>
+                   <properties>
+                       <property><name>viscosity</name><type>Constant</type><value>1e-3</value></property>
+                       <property><name>density</name><type>Constant</type><value>1e3</value></property>
+                   </properties>
+               </phase>
+               <phase>
+                   <type>Solid</type>
+                   <properties>
+                       <property><name>density</name><type>Constant</type><value>2780</value></property>
+                       <property>
+                           <name>swelling_stress_rate</name>
+                           <type>SaturationDependentSwelling</type>
+                           <swelling_pressures>1.3e7 1.3e7 1.3e7</swelling_pressures>
+                           <exponents>1 1 1</exponents>
+                           <lower_saturation_limit>0</lower_saturation_limit>
+                           <upper_saturation_limit>1</upper_saturation_limit>
+                       </property>
+                   </properties>
+               </phase>
+           </phases>
+           <properties>
+               <property><name>transport_porosity</name><type>TransportPorosityFromMassBalance</type><initial_porosity>phi_tr0</initial_porosity><minimal_porosity>0</minimal_poro
sity><maximal_porosity>1</maximal_porosity></property>
+               <property><name>biot_coefficient</name><type>Constant</type><value>0.6</value></property>

```



```

+         <property><name>permeability</name><type>Constant</type><value>1e-18</value></property>
+         <property><name>porosity</name><type>PorosityFromMassBalance</type><initial_porosity>phi0</initial_porosity><minimal_porosity>0</minimal_porosity><maximal_porosity>1</maximal_porosity></property>
+         <property><name>reference_temperature</name><type>Constant</type><value>293.15</value></property>
+         <property><name>relative_permeability</name><type>Constant</type><value>1</value></property>
+         <property><name>saturation</name><type>SaturationVanGenuchten</type><residual_liquid_saturation>0.0</residual_liquid_saturation><residual_gas_saturation>0</residual_gas_saturation><exponent>0.5</exponent><p_b>1e6</p_b></property>
+         <property><name>saturation_micro</name><type>SaturationVanGenuchten</type><residual_liquid_saturation>0</residual_liquid_saturation><residual_gas_saturation>0</residual_gas_saturation><exponent>0.5</exponent><p_b>1e7</p_b></property>
+         <property><name>bishops_effective_stress</name><type>BishopsPowerLaw</type><exponent>1</exponent></property>
+     </properties>
+ </medium>
+ </media>
+ <time_loop>
+   <processes>
+     <process ref="RM">
+       <nonlinear_solver>nonlinear_solver</nonlinear_solver>
+       <convergence_criterion>
+         <norm_type>NORM2</norm_type>
+         <type>PerComponentDeltaX</type>
+         <abstols>5e-8 1e-13 1e-13</abstols>
+       </convergence_criterion>
+       <time_discretization><type>BackwardEuler</type></time_discretization>
+       <time_stepping>
+         <type>IterationNumberBasedTimeStepping</type>
+         <t_initial>0.0</t_initial>
+         <t_end>1e5</t_end>
+         <initial_dt>10</initial_dt>
+         <minimum_dt>0.1</minimum_dt>
+         <maximum_dt>2e3</maximum_dt>
+         <number_iterations>1 2 3 4 5 6 7 8 9 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25</number_iterations>
+         <multiplier>1.225 1.2 1.175 1.15 1.125 1.1 1.075 1.05 1.025 0.975 0.95 0.925 0.9 0.875 0.85 0.825 0.8 0.775 0.75 0.725 0.7 0.675 0.65 0.625</multiplier>
+       </time_stepping>
+     </process>
+   </processes>
+   <output>
+     <type>VTK</type>
+     <prefix>beacon_1a01_dsm_micromacro_stressprobe</prefix>
+     <suffix>_t(:time)</suffix>
+     <fixed_output_times>1e3 1e4 5e4 1e5</fixed_output_times>
+     <variables><variable>pressure</variable><variable>displacement</variable><variable>saturation</variable><variable>micro_pressure</variable><variable>micro_saturation</variable><variable>micro_water_content</variable><variable>micro_porosity</variable><variable>micro_exchange_source</variable><variable>swelling_stress</variable><variable>sigma</variable></variables>
+   </output>
+ </time_loop>
+ <parameters>
+   <parameter><name>sigma0</name><type>Function</type><expression>0</expression><expression>0</expression><expression>0</expression><expression>0</expression><expression>0</expression></parameter>
+   <parameter><name>E</name><type>Constant</type><value>50e6</value></parameter>
+   <parameter><name>nu</name><type>Constant</type><value>0.2</value></parameter>
+   <parameter><name>phi0</name><type>Constant</type><value>0.4</value></parameter>
+   <parameter><name>phi_tr0</name><type>Constant</type><value>0.3</value></parameter>
+   <parameter><name>displacement0</name><type>Constant</type><values>0 0</values></parameter>
+   <parameter><name>pressure_ic</name><type>Constant</type><values>-1e6</values></parameter>
+   <parameter><name>dirichlet0</name><type>Constant</type><value>0</value></parameter>
+   <parameter><name>top_pressure</name><type>Constant</type><value>2e3</value></parameter>
+ </parameters>
+ <process_variables>
+   <process_variable>
+     <name>displacement</name>
+     <components>2</components>
+     <order>1</order>
+     <initial_condition>displacement0</initial_condition>
+     <boundary_conditions>
+       <boundary_condition><geometrical_set>beacon_1a01_geometry</geometrical_set><geometry>left</geometry><type>Dirichlet</type><component>0</component><parameter>dirichlet0</parameter></boundary_condition>
+       <boundary_condition><geometrical_set>beacon_1a01_geometry</geometrical_set><geometry>right</geometry><type>Dirichlet</type><component>0</component><parameter>dirichlet0</parameter></boundary_condition>
+       <boundary_condition><geometrical_set>beacon_1a01_geometry</geometrical_set><geometry>bottom</geometry><type>Dirichlet</type><component>1</component><parameter>dirichlet0</parameter></boundary_condition>
+       <boundary_condition><geometrical_set>beacon_1a01_geometry</geometrical_set><geometry>top</geometry><type>Dirichlet</type><component>1</component><parameter>dirichlet0</parameter></boundary_condition>
+     </boundary_conditions>
+   </process_variable>
+   <process_variable>
+     <name>pressure</name>
+     <components>1</components>
+     <order>1</order>
+     <initial_condition>pressure_ic</initial_condition>
+     <boundary_conditions>
+       <boundary_condition><geometrical_set>beacon_1a01_geometry</geometrical_set><geometry>top</geometry><type>Dirichlet</type><component>0</component><parameter>top_pressure</parameter></boundary_condition>
+     </boundary_conditions>
+   </process_variable>
+ </process_variables>
+ <nonlinear_solvers>
+   <nonlinear_solver><name>nonlinear_solver</name><type>Newton</type><max_iter>30</max_iter><linear_solver>general_linear_solver</linear_solver></nonlinear_solver>
+ </nonlinear_solvers>
+ <linear_solvers>
+   <linear_solver><name>general_linear_solver</name><eigen>csolver_type>SparseLU</solver_type><scaling>true</scaling></eigen></linear_solver>
+ </linear_solvers>
+</OpenGeoSysProject>
diff --git a/Tests/Data/RichardsMechanics/beacon_1a01_reference_t_1000.000000.vtu b/Tests/Data/RichardsMechanics/beacon_1a01_reference_t_1000.000000.vtu
new file mode 100644
index 0000000000..b995a87bc5
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/beacon_1a01_reference_t_1000.000000.vtu
@@ -0,0 +1,32 @@
<?xml version="1.0"?>
<VTKFile type="UnstructuredGrid" version="1.0" byte_order="LittleEndian" header_type="UInt64" compressor="vtkZLibDataCompressor">
  <UnstructuredGrid>
    <FieldData>
      <DataArray type="Int8" Name="OGS_VERSION" NumberOfTuples="19" format="appended" RangeMin="45" RangeMax="103" offset="0" />
    </FieldData>
    <Piece NumberOfPoints="4" NumberOfCells="1" />
    <PointData>
      <DataArray type="Float64" Name="displacement" NumberOfComponents="2" format="appended" RangeMin="0" RangeMax="0" offset="80" />
      <DataArray type="Float64" Name="micro_pressure" format="appended" RangeMin="-2.1387930721e-16" RangeMax="-2.1324910821e-16" offset="140" />
      <DataArray type="Float64" Name="micro_saturation" format="appended" RangeMin="0.9990188599" RangeMax="1.000002" offset="228" />
      <DataArray type="Float64" Name="pressure" format="appended" RangeMin="-964190.65292" RangeMax="2000" offset="312" />
      <DataArray type="Float64" Name="saturation" format="appended" RangeMin="0.7288060223" RangeMax="1.0475197139" offset="392" />
      <DataArray type="Float64" Name="sigma" NumberOfComponents="4" format="appended" RangeMin="0" RangeMax="0" offset="476" />
      <DataArray type="Float64" Name="swelling_stress" NumberOfComponents="4" format="appended" RangeMin="0" RangeMax="0" offset="536" />
    </PointData>
  </UnstructuredGrid>
</VTKFile>

```

```

+ <CellData>
+ </CellData>
+ <Points>
+ <DataArray type="Float64" Name="Points" NumberOfComponents="3" format="appended" RangeMin="0" RangeMax="0.032015621187" offset="596"
+ />
+ </Points>
+ <Cells>
+ <DataArray type="Int64" Name="connectivity" format="appended" RangeMin="" RangeMax="" offset="680" />
+ <DataArray type="Int64" Name="offsets" format="appended" RangeMin="" RangeMax="" offset="752" />
+ <DataArray type="UInt8" Name="types" format="appended" RangeMin="" RangeMax="" offset="812" />
+ </Cells>
+ </Piece>
+ </UnstructuredGrid>
+ <AppendedData encoding="base64">
+ _AQAAAAAAAAAABMAAAAAAAAAGwAAAAAAAAA=ef4z0zPM9c1tD0TTTcZ5U0yt0xJBAAgMQTCAQAAAAAAAAAAGAAAAAAAAEAAAAAAAAADAAAAAAAAA=ef5jYKAMAAAAQAABAQAAAAAAAAAAGAAAAAAAACAAAAAAAAAAHw
AAAAAAAA=ef57b9r58CCldxs+Q0LP/DcfNuxet+cFLAYa0Kv7A=AAAAAAAAAAGAAAAAAAAACAAAAAAAHgAAAAAAAAA=ef6zCM90+/b9vb01Ljb6ck+CieGdVRlUBgA0Yw9gAQAAAAAAAAAAGAAAAAAAAACAAAAAAAGAAAAAAAA
=ef6Tbf225uj1A0Sj0AgWN8BgNAK00C0g=AAAAAAAAAAGAAAAAAAAACAAAAAAAHgAAAAAAAAA=ef7L6njAmBj03D4L5j/sEeJYcuID/RB0D0D6Kg+MAQAAAAAAAAAAGAAAAAAAAIAAAAAAAAAAAGAAAAAAAAA=ef5jYBhYAAAAgAA
BAQAAAAAAAAAAGAAAAAAAAIAAAAAAAAAAAGAAAAAAAAA=ef5jYBhYAAAAgAAABAQAAAAAAAAAAGAAAAAAAAAAGAAAAAAAAAHgAAAAAAAAA=ef5jYMA0Zs0EA3tc4tU169wfvk3BkECXBwC5MhBD0AQAAAAAAAAAAGAAAAAAAAAAGAAAAAAAAEw
AAAAAAAA=ef5jYIAR1jNBKwZ0TQAHAABw=AAAAAAAAAAGAAAAAAAAAAGAAAAAAAAAAGAAAAAAAAA=ef5jYAAAAAoAU=AAAAAAAAAAGAAAAAAAAEAAAAAAAAACQAAAAAAAAA=ef7jBAAACgAK
+ </AppendedData>
+</VTKFile>
diff --git a/Tests/Data/RichardsMechanics/beacon_1b.gml b/Tests/Data/RichardsMechanics/beacon_1b.gml
new file mode 100644
index 0000000000..57a680ff8e
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/beacon_1b.gml
@@ -0,0 +1,17 @@
+<?xml version="1.0" encoding="ISO-8859-1"?>
+<?xml-stYLESHEET type="text/xsl" href="OpenGeoSysGLI.xsl"?>
+<OpenGeoSysGLI xmlns:xsl="http://www.w3.org/2001/XMLSchema-instance" xmlns:ogs="http://www.opengeosys.org">
+ <name>beacon_1b_geometry</name>
+ <points>
+ <point id="0" x="0.0" y="0.0" z="0.0" name="origin"/>
+ <point id="1" x="0.0" y="0.10515" z="0.0"/>
+ <point id="2" x="0.12" y="0.0" z="0.0"/>
+ <point id="3" x="0.12" y="0.10515" z="0.0"/>
+ </points>
+ <polyLines>
+ <polyline id="0" name="left"><pnt>0</pnt><pnt>1</pnt></polyline>
+ <polyline id="1" name="right"><pnt>2</pnt><pnt>3</pnt></polyline>
+ <polyline id="2" name="bottom"><pnt>0</pnt><pnt>2</pnt></polyline>
+ <polyline id="3" name="top"><pnt>1</pnt><pnt>3</pnt></polyline>
+ </polyLines>
+</OpenGeoSysGLI>
diff --git a/Tests/Data/RichardsMechanics/beacon_1b_domain.vtu b/Tests/Data/RichardsMechanics/beacon_1b_domain.vtu
new file mode 100644
index 0000000000..ab8879784b
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/beacon_1b_domain.vtu
@@ -0,0 +1,24 @@
+<?xml version="1.0"?>
+<VTKFile type="UnstructuredGrid" version="0.1" byte_order="LittleEndian">
+ <UnstructuredGrid>
+ <Piece NumberOfPoints="4" NumberOfCells="1">
+ <PointData>
+ <CellData>
+ <DataArray type="Int32" Name="MaterialIDs" format="ascii">0</DataArray>
+ </CellData>
+ <Points>
+ <DataArray type="Float64" NumberOfComponents="3" format="ascii">
+ 0.0 0.0 0.0
+ 0.12 0.0 0.0
+ 0.12 0.10515 0.0
+ 0.0 0.10515 0.0
+ </DataArray>
+ </Points>
+ <Cells>
+ <DataArray type="Int32" Name="connectivity" format="ascii">0 1 2 3</DataArray>
+ <DataArray type="Int32" Name="offsets" format="ascii">4</DataArray>
+ <DataArray type="UInt8" Name="types" format="ascii">9</DataArray>
+ </Cells>
+ </Piece>
+ </UnstructuredGrid>
+</VTKFile>
diff --git a/Tests/Data/RichardsMechanics/beacon_1b_dsm_micromacro_smoke.prj b/Tests/Data/RichardsMechanics/beacon_1b_dsm_micromacro_smoke.prj
new file mode 100644
index 0000000000..4f782c3b59
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/beacon_1b_dsm_micromacro_smoke.prj
@@ -0,0 +1,84 @@
+<?xml version="1.0" encoding="ISO-8859-1"?>
+<OpenGeoSysProject>
+ <mesh axially_symmetric="true">beacon_1b_domain.vtu</mesh>
+ <geometry>beacon_1b.gml</geometry>
+ <processes>
+ <process>
+ <name>RM</name>
+ <type>RICHARDS_MECHANICS</type>
+ <integration_order>2</integration_order>
+ <micro_porosity>
+ <mass_exchange_coefficient>1e-13</mass_exchange_coefficient>
+ <nonlinear_solver><maximum_iterations>100</maximum_iterations><residuum_tolerance>1e-8</residuum_tolerance><increment_tolerance>1e-20</increment_tolerance></nonlin
ear_solver>
+ </micro_porosity>
+ <potential_exchange>
+ <enabled>true</enabled>
+ <mode>full_potential</mode>
+ <pressure_tolerance>0.0</pressure_tolerance>
+ <hamaker_constant>1e-30</hamaker_constant>
+ <specific_surface>1.0</specific_surface>
+ <micro_solid_density_reference>2650</micro_solid_density_reference>
+ <micro_solid_volume_fraction_reference>0.6</micro_solid_volume_fraction_reference>
+ <initial_micro_water_content>0.1</initial_micro_water_content>
+ <fd_jacobian_for_exchange>false</fd_jacobian_for_exchange>
+ </potential_exchange>
+ <constitutive_relation>type=LinearElasticIsotropic</type><youngs_modulus>E</youngs_modulus><poissons_ratio>nu</poissons_ratio></constitutive_relation>
+ <process_variables><pressure>pressure</pressure><displacement>displacement</displacement></process_variables>
+ <secondary_variables>
+ <secondary_variable name="sigma"/><secondary_variable name="swelling_stress"/><secondary_variable name="epsilon"/><secondary_variable name="velocity"/><secondary_v
ariable name="saturation"/><secondary_variable name="porosity"/><secondary_variable name="transport_porosity"/><secondary_variable name="dry_density_solid"/><secondary_variable nam
e="micro_saturation"/><secondary_variable name="micro_pressure"/>
+ </secondary_variables>
+ <specific_body_force>0</specific_body_force>
+ <initial_stress>sigma0</initial_stress>
+ <mass_lumping>true</mass_lumping>
+ </process>
+ </processes>

```



```

index 0000000000..029f23e82c
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/beacon_1c.gml
@@ -0,0 +1,19 @@
+<?xml version="1.0" encoding="ISO-8859-1"?>
+<?xml-stYLESHEET type="text/xml" href="OpenGeoSysGLI.xsl"?>
+<OpenGeoSysGLI xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:ogs="http://www.opengeosys.org">
+  <name>beacon_1c_geometry</name>
+  <points>
+    <point id="0" x="0.0" y="0.0" z="0.0" name="origin"/>
+    <point id="1" x="0.0" y="0.0485" z="0.0"/>
+    <point id="2" x="0.0" y="0.1" z="0.0"/>
+    <point id="3" x="0.05" y="0.0" z="0.0"/>
+    <point id="4" x="0.05" y="0.0485" z="0.0"/>
+    <point id="5" x="0.05" y="0.1" z="0.0"/>
+  </points>
+  <polylines>
+    <polyline id="0" name="left"><pnt>0</pnt><pnt>1</pnt><pnt>2</pnt></polyline>
+    <polyline id="1" name="right"><pnt>3</pnt><pnt>4</pnt><pnt>5</pnt></polyline>
+    <polyline id="2" name="bottom"><pnt>0</pnt><pnt>3</pnt><pnt>4</pnt></polyline>
+    <polyline id="3" name="top"><pnt>2</pnt><pnt>5</pnt><pnt>4</pnt></polyline>
+  </polylines>
+</OpenGeoSysGLI>
diff --git a/Tests/Data/RichardsMechanics/beacon_1c_domain.vtu b/Tests/Data/RichardsMechanics/beacon_1c_domain.vtu
new file mode 100644
index 0000000000..c3cee8524f
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/beacon_1c_domain.vtu
@@ -0,0 +1,26 @@
+<?xml version="1.0"?>
+<VTKFile type="UnstructuredGrid" version="0.1" byte_order="LittleEndian">
+  <UnstructuredGrid>
+    <Piece NumberOfPoints="6" NumberOfCells="2">
+      <PointData>
+        <CellData>
+          <DataArray type="Int32" Name="MaterialIDs" format="ascii">0 1</DataArray>
+        </CellData>
+        <Points>
+          <DataArray type="Float64" NumberOfComponents="3" format="ascii">
+            0.0 0.0 0.0
+            0.05 0.0 0.0
+            0.0 0.0485 0.0
+            0.05 0.0485 0.0
+            0.0 0.1 0.0
+            0.05 0.1 0.0
+          </DataArray>
+        </Points>
+        <Cells>
+          <DataArray type="Int32" Name="connectivity" format="ascii">0 1 3 2 3 5 4</DataArray>
+          <DataArray type="Int32" Name="offsets" format="ascii">4 8</DataArray>
+          <DataArray type="UInt8" Name="types" format="ascii">9 9</DataArray>
+        </Cells>
+      </Piece>
+    </UnstructuredGrid>
+</VTKFile>
diff --git a/Tests/Data/RichardsMechanics/beacon_1c_dsm_micromacro_smoke.prj b/Tests/Data/RichardsMechanics/beacon_1c_dsm_micromacro_smoke.prj
new file mode 100644
index 0000000000..81cefa178c
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/beacon_1c_dsm_micromacro_smoke.prj
@@ -0,0 +1,125 @@
+<?xml version="1.0" encoding="ISO-8859-1"?>
+<OpenGeoSysProject>
+  <mesh axially_symmetric="true">beacon_1c_domain.vtu</mesh>
+  <geometry>beacon_1c.gml</geometry>
+  <processes>
+    <process>
+      <name>RM</name>
+      <type>RICHARDS_MECHANICS</type>
+      <integration_order>2</integration_order>
+      <micro_porosity>
+        <mass_exchange_coefficient>1e-13</mass_exchange_coefficient>
+        <nonlinear_solver><maximum_iterations>100</maximum_iterations><residuum_tolerance>1e-8</residuum_tolerance><increment_tolerance>1e-20</increment_tolerance></nonlin
ear_solver>
+      </micro_porosity>
+      <potential_exchange>
+        <enabled>true</enabled>
+        <mode>full_potential</mode>
+        <pressure_tolerance>0.0</pressure_tolerance>
+        <hamaker_constant>1e-30</hamaker_constant>
+        <specific_surface>1.0</specific_surface>
+        <micro_solid_density_reference>2650</micro_solid_density_reference>
+        <micro_solid_volume_fraction_reference>0.6</micro_solid_volume_fraction_reference>
+        <initial_micro_water_content>0.1</initial_micro_water_content>
+        <!-- DSMMicroMacro-style liquid-density EOS metadata taken from the
+          1a01 dsm_micromacro-role alignment case so the scalar dsm_micromacro
+          mass-storage path remains well-posed for the 1c media. -->
+        <micro_liquid_density_reference>1300</micro_liquid_density_reference>
+        <micro_liquid_density_a>1.3</micro_liquid_density_a>
+        <micro_liquid_density_b>1.0</micro_liquid_density_b>
+        <fd_jacobian_for_exchange>false</fd_jacobian_for_exchange>
+        <medium id="0">
+          <specific_surface>0.8</specific_surface>
+          <micro_solid_volume_fraction_reference>0.65</micro_solid_volume_fraction_reference>
+          <initial_micro_water_content>0.08</initial_micro_water_content>
+          <micro_liquid_density_reference>1300</micro_liquid_density_reference>
+          <micro_liquid_density_a>1.3</micro_liquid_density_a>
+          <micro_liquid_density_b>1.0</micro_liquid_density_b>
+        </medium>
+        <medium id="1">
+          <specific_surface>1.5</specific_surface>
+          <micro_solid_volume_fraction_reference>0.45</micro_solid_volume_fraction_reference>
+          <initial_micro_water_content>0.14</initial_micro_water_content>
+          <micro_liquid_density_reference>1300</micro_liquid_density_reference>
+          <micro_liquid_density_a>1.3</micro_liquid_density_a>
+          <micro_liquid_density_b>1.0</micro_liquid_density_b>
+        </medium>
+      </potential_exchange>
+      <constitutive_relation id="0,1"><type>LinearElasticIsotropic</type><youngs_modulus>E</youngs_modulus><poissons_ratio>nu</poissons_ratio></constitutive_relation>
+      <process_variables><pressure>pressure</pressure><displacement>displacement</displacement></process_variables>
+      <secondary_variables>
+        <secondary_variable name="sigma"/><secondary_variable name="swelling_stress"/><secondary_variable name="epsilon"/><secondary_variable name="velocity"/><secondary_v
ariable name="saturation"/><secondary_variable name="porosity"/><secondary_variable name="transport_porosity"/><secondary_variable name="dry_density_solid"/><secondary_variable nam
e="micro_saturation"/><secondary_variable name="micro_pressure"/>
+      </secondary_variables>
+      <specific_body_force>0 0</specific_body_force>
+      <initial_stress>sigma0</initial_stress>

```

```

+      <mass_lumping>true</mass_lumping>
+    </process>
+  </processes>
+  <media>
+    <medium id="0">
+      <phases>
+        <phase><type>AqueousLiquid</type><properties><property><name>viscosity</name><type>Constant</type><value>1e-3</value></property><property><name>density</name><type>Constant</type><value>1e3</value></property></properties></phase>
+      </phases>
+      <properties>
+        <property><name>transport_porosity</name><type>TransportPorosityFromMassBalance</type><initial_porosity>phi_tr0_block</initial_porosity><minimal_porosity>0</minimal_porosity><maximal_porosity>1</maximal_porosity></property>
+        <property><name>biot_coefficient</name><type>Constant</type><value>1.0</value></property>
+        <property><name>permeability</name><type>Constant</type><value>1e-18</value></property>
+        <property><name>porosity</name><type>PorosityFromMassBalance</type><initial_porosity>phi0_block</initial_porosity><minimal_porosity>0</minimal_porosity><maximal_porosity>1</maximal_porosity></property>
+        <property><name>reference_temperature</name><type>Constant</type><value>293.15</value></property>
+        <property><name>relative_permeability</name><type>Constant</type><value>1</value></property>
+        <property><name>saturation</name><type>SaturationVanGenuchten</type><residual_liquid_saturation>0.0</residual_liquid_saturation><residual_gas_saturation>0</residual_gas_saturation><exponent>0.5</exponent><p_b>1e6</p_b></property>
+        <property><name>saturation_micro</name><type>SaturationVanGenuchten</type><residual_liquid_saturation>0</residual_liquid_saturation><residual_gas_saturation>0</residual_gas_saturation><exponent>0.5</exponent><p_b>1e7</p_b></property>
+        <property><name>bishops_effective_stress</name><type>BishopsPowerLaw</type><exponent>1</exponent></property>
+      </properties>
+    </medium>
+    <medium id="1">
+      <phases>
+        <phase><type>AqueousLiquid</type><properties><property><name>viscosity</name><type>Constant</type><value>1e-3</value></property><property><name>density</name><type>Constant</type><value>1e3</value></property></properties></phase>
+      </phases>
+      <properties>
+        <property><name>transport_porosity</name><type>TransportPorosityFromMassBalance</type><initial_porosity>phi_tr0_pellet</initial_porosity><minimal_porosity>0</minimal_porosity><maximal_porosity>1</maximal_porosity></property>
+        <property><name>biot_coefficient</name><type>Constant</type><value>1.0</value></property>
+        <property><name>permeability</name><type>Constant</type><value>1e-15</value></property>
+        <property><name>porosity</name><type>PorosityFromMassBalance</type><initial_porosity>phi0_pellet</initial_porosity><minimal_porosity>0</minimal_porosity><maximal_porosity>1</maximal_porosity></property>
+        <property><name>reference_temperature</name><type>Constant</type><value>293.15</value></property>
+        <property><name>relative_permeability</name><type>Constant</type><value>1</value></property>
+        <property><name>saturation</name><type>SaturationVanGenuchten</type><residual_liquid_saturation>0.0</residual_liquid_saturation><residual_gas_saturation>0</residual_gas_saturation><exponent>0.5</exponent><p_b>5e5</p_b></property>
+        <property><name>saturation_micro</name><type>SaturationVanGenuchten</type><residual_liquid_saturation>0</residual_liquid_saturation><residual_gas_saturation>0</residual_gas_saturation><exponent>0.5</exponent><p_b>5e6</p_b></property>
+        <property><name>bishops_effective_stress</name><type>BishopsPowerLaw</type><exponent>1</exponent></property>
+      </properties>
+    </medium>
+  </media>
+  <time_loop>
+    <processes><process ref="RM"><nonlinear_solver>nonlinear_solver</nonlinear_solver><convergence_criterion>norm_type=NORM2</norm_type><type>PerComponentDelta</type><abstol>5e-8</abstol><reltol>1e-13</reltol><convergence_criterion>time_discretization</time_discretization><type>BackwardEuler</type><time_discretization><time_stepping><type>FixedTimeStepping</type><t_initial>0.0</t_initial><t_end>1e3</t_end><time_steps><pair><repeat>20</repeat><delta_t>50</delta_t></pair></time_steps></time_stepping></process></processes>
+    <output><type>VTk</type><prefix>beacon_1c_dsm_micromacro_smoke</prefix><suffix>t_{time}</suffix><fixed_output_times>variables</variable>pressure</variable>displacement</variable>saturation</variable>porosity</variable>transport_porosity</variable>micro_pressure</variable>micro_saturation</variable>swelling_stress</variable>sigma</variable></output>
+  </time_loop>
+  <parameters>
+    <parameter><name>sigma0</name><type>Function</type><expression>0</expression><expression>0</expression><expression>0</expression><expression>0</expression></parameter>
+    <parameter><name>E</name><type>Constant</type><value>50e6</value></parameter>
+    <parameter><name>nu</name><type>Constant</type><value>0.2</value></parameter>
+    <parameter><name>phi0_block</name><type>Constant</type><value>0.35</value></parameter>
+    <parameter><name>phi_tr0_block</name><type>Constant</type><value>0.25</value></parameter>
+    <parameter><name>phi0_pellet</name><type>Constant</type><value>0.675</value></parameter>
+    <parameter><name>phi_tr0_pellet</name><type>Constant</type><value>0.55</value></parameter>
+    <parameter><name>displacement0</name><type>Constant</type><value>0</value></parameter>
+    <parameter><name>pressure_ic</name><type>Constant</type><value>-1e6</value></parameter>
+    <parameter><name>dirichlet0</name><type>Constant</type><value>0</value></parameter>
+    <parameter><name>top_pressure</name><type>Constant</type><value>1e4</value></parameter>
+    <parameter><name>bottom_pressure</name><type>Constant</type><value>0</value></parameter>
+  </parameters>
+  <process_variables>
+    <process_variable><name>displacement</name><components>2</components><order>1</order><initial_condition>displacement0</initial_condition><boundary_conditions>
+      <boundary_condition><name>beacon_1c_geometry</name><geometrical_set>beacon_1c_geometry</geometrical_set><type>Dirichlet</type><component>0</component><parameter>dirichlet0</parameter></boundary_condition>
+      <boundary_condition><name>beacon_1c_geometry</name><geometrical_set>beacon_1c_geometry</geometrical_set><type>Dirichlet</type><component>0</component><parameter>dirichlet0</parameter></boundary_condition>
+      <boundary_condition><name>beacon_1c_geometry</name><geometrical_set>beacon_1c_geometry</geometrical_set><type>Dirichlet</type><component>1</component><parameter>dirichlet0</parameter></boundary_condition>
+      <boundary_condition><name>beacon_1c_geometry</name><geometrical_set>beacon_1c_geometry</geometrical_set><type>Dirichlet</type><component>1</component><parameter>dirichlet0</parameter></boundary_condition>
+    </boundary_conditions></process_variable>
+    <process_variable><name>pressure</name><components>1</components><order>1</order><initial_condition>pressure_ic</initial_condition><boundary_conditions>
+      <boundary_condition><name>beacon_1c_geometry</name><geometrical_set>beacon_1c_geometry</geometrical_set><type>Dirichlet</type><component>0</component><parameter>top_pressure</parameter></boundary_condition>
+      <boundary_condition><name>beacon_1c_geometry</name><geometrical_set>beacon_1c_geometry</geometrical_set><type>Dirichlet</type><component>0</component><parameter>bottom_pressure</parameter></boundary_condition>
+    </boundary_conditions></process_variable>
+  </process_variables>
+  <nonlinear_solvers><nonlinear_solver><name>nonlinear_solver</name><type>Newton</type><max_iter>30</max_iter><linear_solver>general_linear_solver</linear_solver></nonlinear_solvers>
+  <linear_solvers><linear_solver><name>general_linear_solver</name><eigen><solver_type>SparseU</solver_type><scaling>true</scaling></eigen></linear_solver></linear_solvers>
+</OpenGeoSysProject>
diff --git a/Tests/Data/RichardsMechanics/beacon_1c_reference_t_1000.000000.vtu b/Tests/Data/RichardsMechanics/beacon_1c_reference_t_1000.000000.vtu
new file mode 100644
index 0000000000..26d604f210
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/beacon_1c_reference_t_1000.000000.vtu
@@ -0,0 +1,34 @@
<?xml version="1.0"?>
<VTkFile type="UnstructuredGrid" version="1.0" byte_order="LittleEndian" header_type="UInt64" compressor="vtkLibDataCompressor">
  <UnstructuredGrid>
    <FieldData>
      <DataArray type="Int8" Name="OGS_VERSION" NumberOfTuples="19" format="appended" RangeMin="45" RangeMax="103" offset="0" />
    </FieldData>
    <Piece NumberOfPoints="6" NumberOfCells="2" />
    <PointData>
      <DataArray type="Float64" Name="displacement" NumberOfComponents="2" format="appended" RangeMin="0" RangeMax="3.885874694e-06" offset="80" />
      <DataArray type="Float64" Name="micro_pressure" format="appended" RangeMin="-2.7112926238e-16" RangeMax="-1.1865074546e-16" offset="160" />
      <DataArray type="Float64" Name="micro_saturation" format="appended" RangeMin="0.99949992927" RangeMax="1.0000071429" offset="260" />
      <DataArray type="Float64" Name="porosity" format="appended" RangeMin="0.34994746054" RangeMax="0.67502440884" offset="356" />
    </PointData>
  </UnstructuredGrid>

```

```

+ <DataArray types="Float64" Names="pressure" format="appended" RangeMin="-225724.96148" RangeMax="10000" offset="460" />
+ <DataArray types="Float64" Names="saturation" format="appended" RangeMin="0.95150072204" RangeMax="1.0164353976" offset="544" />
+ <DataArray types="Float64" Names="sigma" NumberOfComponents="4" format="appended" RangeMin="137.51017651" RangeMax="4721.1827267" offset="644" />
+ <DataArray types="Float64" Names="swelling_stress" NumberOfComponents="4" format="appended" RangeMin="0" RangeMax="0" offset="824" />
+ <DataArray types="Float64" Names="transport_porosity" format="appended" RangeMin="0.26994764654" RangeMax="0.53502340084" offset="884" />
+ </PointData>
+ <CellData>
+ </CellData>
+ <Points>
+ <DataArray types="Float64" Names="Points" NumberOfComponents="3" format="appended" RangeMin="0" RangeMax="0.11180339887" offset="984" />
+ </Points>
+ <Cells>
+ <DataArray types="Int64" Name="connectivity" format="appended" RangeMin="" RangeMax="" offset="1084" />
+ <DataArray types="Int64" Name="offsets" format="appended" RangeMin="" RangeMax="" offset="1164" />
+ <DataArray types="UInt8" Name="types" format="appended" RangeMin="" RangeMax="" offset="1228" />
+ </Cells>
+ </Piece>
+ </UnstructuredGrid>
+ <AppendedData encoding="base64">
+ _AQAAAAAAAAAAGAAAAAAAAABAAAAAAAAAGwAAAAAAAAA=ef4z0zPVm9c1DDTTc2SU0ytx8jBAAQMTCQAQAAAAAAAAAAGAAAAAAAAAAGAAAAAAAAA=ef5jYCA0qXbS+X6XNgH4+uh8XEBAA5NBjs=AQAAAAAAAAAAGg
AAAAAAAAAAAAAAAAAAGAAAAAAAAA=ef6737L/b3Hn5j0PoL5+YvbkV7Ve8y9Bfd2VTn8z:f8wKAwCRLt2AQAAAAAAAAAAGAAAAAAAAAAGAAAAAAAAA=ef778x8E3tv/g93mLpPcnv9+zb28Dpfekp9ayM3yPw1IAVHHHxw
AQAAAAAAAAAAGAAAAAAAAAAGAAAAAAAAAAGAAAAAAAAA=ef5rXud7o5v1m0bLP537Y2YZ0de6brEH77U1Hz8x8ar8P5gMak+EcFg=AQAAAAAAAAAAGAAAAAAAAAAGAAAAAAAAAAGAAAAAAAAA=ef5jYEAfSekNj7v5T4to8GCHYcdDQ
A6JoLxw=AQAAAAAAAAAAGAAAAAAAAAAGAAAAAAAAAAGAAAAAAAAA=ef5r/hGveUvgg30b1fapYfIyseidvRaUvjmh+HCgBwf7e1AaAF2ZGPs=AQAAAAAAAAAAGAAAAAAAAAAGAAAAAAAAAAGAAAAAAAAA=ef47WfptU7yxAMwfrGKD3xwAf
f0XSNy9j7b0H5t60yMLADVP7DkbubV5k4HIDQCQdg/IAfw+XLXb:XXAMwofQCQdg/BCofJLIxolH8RMcIPQGBxjfwB91p175hgCIPAAbZmfPAQAAAAAAAAAAGAAAAAAAAAAGAAAAAAAAAAGAAAAAAAAA=ef5jYBjaAAAwABAQAAAAAAAAA
AgAAAAAAAAAAGAAAAAAAAAAGAAAAAAAAAAGAAAAAAAAA=ef5L1Uyqu+R20T4D5s+fP0TR0nzTXhLKqS1TTX8p99BeG0DAP7vFTU=AQAAAAAAAAAAGAAAAAAAAAAGAAAAAAAAAAGAAAAAAAAA=ef5jYMA0Zs8EqZX20K0ZFrlu+/z3yqq4Pew9u1q/E4M9ej
IAH+V1Sw=AQAAAAAAAAAAGAAAAAAAAAAGAAAAAAAAAAGAAAAAAAAA=ef5jYTAARIjNDKjWZG1YOCuZoSIAAJQABU=AQAAAAAAAAAAGAAAAAAAAAAGAAAAAAAAAAGAAAAAAAAA=ef5jYAA0igNAACQA0B=AQAAAAAAAAAAGAAAAAAAAAAG
AAAAAGAAAAAAAAA=ef7j5AQAA0BAEw=
+ </AppendedData>
+ </VTKFile>
diff --git a/Tests/Data/RichardsMechanics/double_porosity_swelling_dsm_micromacro_constbc.xml b/Tests/Data/RichardsMechanics/double_porosity_swelling_dsm_micromacro_constbc.xml
new file mode 100644
index 0000000000..fd2923d6a
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/double_porosity_swelling_dsm_micromacro_constbc.xml
@@ -0,0 +1,48 @@
+<?xml version="1.0" encoding="ISO-8859-1"?>
+<OpenGeoSysProjectDiff base_file="double_porosity_swelling.prj">
+ <add sel="/processes/process[1]">
+ <potential_exchange>
+ <enabled=true/<enabled>
+ <mode=full_potential</mode>
+ <pressure_tolerance>0.0</pressure_tolerance>
+ <hamaker_constant>1e-30</hamaker_constant>
+ <specific_surface>1.0</specific_surface>
+ <micro_solid_density_reference>2650</micro_solid_density_reference>
+ <micro_solid_volume_fraction_reference>0.6</micro_solid_volume_fraction_reference>
+ <initial_micro_water_content>0.1</initial_micro_water_content>
+ <fd_jacobian_for_exchange>false</fd_jacobian_for_exchange>
+ </potential_exchange>
+ </add>
+ <!-- Keep the macro field as simple as possible: the initial pressure and the
+ top Dirichlet pressure are both held at the same constant suction. This
+ isolates the kept local DSM constitutive chain at process level. -->
+ <replace msel="/parameters/parameter[name='pressure_ic']/values/text()">
+ -1e7
+ </replace>
+ <replace msel="/parameters/parameter[name='dirichlet_pressure']/value/text()">
+ -1e7
+ </replace>
+ <replace msel="/curves/curve[name='pressure_ramp']/coords/text()">
+ 0.0 1e3
+ </replace>
+ <replace msel="/curves/curve[name='pressure_ramp']/values/text()">
+ 1 1
+ </replace>
+ <replace msel="/time_loop/processes/process/time_stepping/t_end/text()">
+ 1e3
+ </replace>
+ <replace msel="/time_loop/processes/process/time_stepping/initial_dt/text()">
+ 10
+ </replace>
+ <replace msel="/time_loop/processes/process/time_stepping/maximum_dt/text()">
+ 100
+ </replace>
+ <replace msel="/time_loop/output/fixed_output_times/text()">
+ 1e3
+ </replace>
+ <replace msel="/time_loop/output/prefix/text()">
+ double_porosity_swelling_dsm_micromacro_constbc
+ </replace>
+</OpenGeoSysProjectDiff>
diff --git a/Tests/Data/RichardsMechanics/double_porosity_swelling_dsm_micromacro_wetting.xml b/Tests/Data/RichardsMechanics/double_porosity_swelling_dsm_micromacro_wetting.xml
new file mode 100644
index 0000000000..384085ef53
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/double_porosity_swelling_dsm_micromacro_wetting.xml
@@ -0,0 +1,59 @@
+<?xml version="1.0" encoding="ISO-8859-1"?>
+<OpenGeoSysProjectDiff base_file="double_porosity_swelling.prj">
+ <add sel="/processes/process[1]">
+ <potential_exchange>
+ <enabled=true/<enabled>
+ <mode=full_potential</mode>
+ <pressure_tolerance>0.0</pressure_tolerance>
+ <hamaker_constant>1e-30</hamaker_constant>
+ <specific_surface>1.0</specific_surface>
+ <micro_solid_density_reference>2650</micro_solid_density_reference>
+ <micro_solid_volume_fraction_reference>0.6</micro_solid_volume_fraction_reference>
+ <fd_jacobian_for_exchange>false</fd_jacobian_for_exchange>
+ <check_local_jacobian>true</check_local_jacobian>
+ <local_jacobian_perturbation>1e-8</local_jacobian_perturbation>
+ <local_jacobian_relative_tolerance>1e-3</local_jacobian_relative_tolerance>
+ </potential_exchange>
+ </add>
+ <replace msel="/processes/process[1]/micro_porosity/mass_exchange_coefficient/text()">
+ 1e-13
+ </replace>
+ <replace msel="/parameters/parameter[name='pressure_ic']/values/text()">
+ -1e7
+ </replace>
+ <replace msel="/parameters/parameter[name='phi_tr0']/value/text()">

```

+J625rH+8MQ0dy/GbDX6UDpdb4wuulymc1599vyHz2e3RJy6SkPCRx4wRL5Jc/42c+Uh6jfnZ/qFIGL6q5h9HzsT1/6p/vy1JNxvlfAasG5SLhYIVu905Yuf7ixmnvq1RR1VuupGRZPrUxIqLqQv6+w99rXXX0Z+lox6+33gt9d1hob6HU9c


```

+ {
+     auto const it = column.find(key);
+     if (it == column.end())
+     {
+         return fallback;
+     }
+     return std::stod(fields.at(it--second));
+ };
+
+ std::vector<DSMMicroMacroOverlapBaselineRow> rows;
+ std::string line;
+ while (std::getline(in, line))
+ {
+     if (line.empty())
+     {
+         continue;
+     }
+
+     auto const fields = splitCommaLine(line);
+     rows.push_back(
+     {
+         .step = static_cast<int>(get_value(fields, "step")),
+         .pressure = get_value(fields, "pressure"),
+         .epsilon_v_total =
+             get_optional_value(fields, "epsilon_v_total", 0.0),
+         .delta_epsilon_v =
+             get_optional_value(fields, "delta_epsilon_v", 0.0),
+         .saturation = get_value(fields, "S_L"),
+         .mu_LR = get_value(fields, "mu_LR"),
+         .n_l = get_value(fields, "n_l"),
+         .phi_m = get_value(fields, "phi_m"),
+         .phi_M = get_value(fields, "phi_M"),
+         .phi = get_optional_value(fields, "phi",
+             get_value(fields, "phi_m") +
+             get_value(fields, "phi_M")),
+         .rho_LR = get_value(fields, "rho_LR"),
+         .mu_LR = get_value(fields, "mu_LR"),
+         .rho_l_hat = get_value(fields, "rho_l_hat"),
+         .delta_epsilon_sw =
+             get_optional_value(fields, "delta_epsilon_sw", 0.0),
+         .epsilon_sw = get_value(fields, "epsilon_sw"),
+         .stress_xx = get_value(fields, "sigma_S_xx");
+     }
+
+     return rows;
+ }
+
+ double comparisonTolerance(double const a, double const b,
+     double const rel = 1e-10,
+     double const abs = 1e-14)
+ {
+     return abs + rel * std::max(std::abs(a), std::abs(b));
+ }
+
+ double isotropicStressFromSwelling(double const epsilon_sw, double const E,
+     double const nu)
+ {
+     double const bulk_modulus = E / (3.0 * (1.0 - 2.0 * nu));
+     return -bulk_modulus * epsilon_sw;
+ }
+
+ double referenceMicroSolidVolumeFraction(
+     double const n_l, double const phi, double const phi_M_prev,
+     double const phi_m_prev, double const volumetric_strain,
+     double const volumetric_strain_prev,
+     PotentialExchangeParameters const& potential_exchange_params)
+ {
+     if (potential_exchange_params.micro_solid_volume_fraction_mode ==
+         MicroSolidVolumeFractionMode::Reference)
+     {
+         return std::max(1e-16, potential_exchange_params.micro_solid_volume_fraction_reference);
+     }
+ }
+
+ auto const split = computeTransportPorosityUpdate(
+     phi, phi_M_prev, phi_m_prev, n_l, volumetric_strain,
+     volumetric_strain_prev, potential_exchange_params.macro_porosity_update_mode);
+ return std::max(1e-16, 1.0 - split.phi_M - split.phi_m);
+ }
+
+ *ReducedMicroLiquidDensityData solveReferenceReducedMicroLiquidDensity(
+     double const n_l, double const rho_LR, double const nS,
+     PotentialExchangeParameters const& potential_exchange_params)
+ {
+     auto const solve_rho = [&](double const n_eval)
+     {
+         double const n_l_safe = std::max(1e-16, n_eval);
+         double const nS_safe = std::max(1e-16, nS);
+         double const rho_SR =
+             std::max(1e-16, potential_exchange_params.micro_solid_density_reference);
+         double const rho_l0 =
+             std::max(1e-16, potential_exchange_params.micro_liquid_density_reference);
+         double const a_rho = std::max(1e-16, potential_exchange_params.micro_liquid_density_a);
+         double const b_rho = std::max(1e-16, potential_exchange_params.micro_liquid_density_b);
+         double const denominator = nS_safe * rho_SR;
+
+         auto const rhs = [&](double const rho_LR)
+         {
+             double const omega_l =
+                 std::max(1e-16, n_l_safe * rho_LR / denominator);
+             return std::pair(
+                 omega_l,
+                 rho_l0 * std::exp(-a_rho * std::pow(omega_l, b_rho)) +
+                 rho_LR);
+         };
+
+         double rho_LR =
+             rho_LR +
+             rho_l0 *
+             std::exp(-a_rho *
+                 std::pow(std::max(1e-16, n_l_safe * rho_LR /
+                     denominator),
+                     b_rho));
+
+         constexpr int max_iterations = 40;
+         for (int iter = 0; iter < max_iterations; ++iter)
+         {
+             auto const [omega_l, target] = rhs(rho_LR);
+             double const residual = rho_LR - target;
+             if (std::abs(residual) <=

```

```

+         1e-14 * std::max(1.0, std::abs(rho_LR)))
+     {
+         return std::pair{rho_LR, omega_L};
+     }
+
+     double const h = 1e-8 * std::max(1.0, std::abs(rho_LR));
+     double const rho_plus = rho_LR + h;
+     double const rho_minus = std::max(1e-16, rho_LR - h);
+     auto const [omega_plus, target_plus] = rhs(rho_plus);
+     auto const [omega_minus, target_minus] = rhs(rho_minus);
+     (void)omega_L;
+     (void)omega_plus;
+     (void)omega_minus;
+     double const g_plus = rho_plus - target_plus;
+     double const g_minus = rho_minus - target_minus;
+     double const jacobian = (g_plus - g_minus) / (rho_plus - rho_minus);
+     EXPECT_TRUE(std::isfinite(jacobian));
+     EXPECT_GT(std::abs(jacobian), 1e-20);
+     if (!std::isfinite(jacobian) && std::abs(jacobian) > 1e-20)
+     {
+         break;
+     }
+
+     double const candidate =
+         std::max(1e-16, rho_LR - residual / jacobian);
+     if (std::abs(candidate - rho_LR) <=
+         1e-14 * std::max(1.0, std::abs(rho_LR)))
+     {
+         auto const [omega_candidate, _] = rhs(candidate);
+         (void)_;
+         return std::pair(candidate, omega_candidate);
+     }
+     rho_LR = candidate;
+ }
+ auto const [omega_L, _] = rhs(rho_LR);
+ (void)_;
+ return std::pair{rho_LR, omega_L};
+ };
+
+ double const n_L_safe = std::max(1e-16, n_L);
+ auto const [rho_LR, omega_L] = solve_rho(n_L_safe);
+ double const h = 1e-8 * std::max(1.0, std::abs(n_L_safe));
+ double const n_plus = n_L_safe + h;
+ double const n_minus = std::max(1e-16, n_L_safe - h);
+ double const rho_plus = solve_rho(n_plus).first;
+ double const rho_minus = solve_rho(n_minus).first;
+ double const drho_LR_dnl = (rho_plus - rho_minus) / (n_plus - n_minus);
+
+ return {rho_LR = rho_LR,
+         .omega_L = omega_L,
+         .drho_LR_dnl = drho_LR_dnl,
+         .drho_L_dnl = rho_LR + n_L_safe * drho_LR_dnl};
+ }
+
+ DsmMicromacroReferenceSinglePointData solveDsmMicromacroReferenceSinglePoint(
+     double const p_L, double const n_L_prev, double const dt,
+     double const rho_LR, double const alpha_bar, double const mu,
+     double const phi, PotentialExchangeParameters const& potential_exchange_params,
+     double const volumetric_strain = 0.0,
+     double const volumetric_strain_prev = 0.0)
+ {
+     constexpr double n_L_floor = 1e-16;
+     double const phi_ceiling =
+         potential_exchange_params.local_nonlinear_solve_mode !=
+         LocalNonlinearSolveMode::ScalarExchange &&
+         std::isfinite(phi)
+         ? std::max(n_L_floor, phi)
+         : std::numeric_limits<double>::infinity();
+     double const volumetric_strain_rate =
+         dt > 0.0 ? (volumetric_strain - volumetric_strain_prev) / dt : 0.0;
+     bool const use_mass_storage =
+         potential_exchange_params.local_nonlinear_solve_mode ==
+         LocalNonlinearSolveMode::ScalarReferenceMassStorage;
+     double const n5_prev = potential_exchange_params.micro_solid_volume_fraction_mode ==
+         MicroSolidVolumeFractionMode::Reference
+         ? potential_exchange_params.micro_solid_volume_fraction_reference
+         : std::max(1e-16, 1.0 - 0.0 - n_L_prev);
+     auto const prev_micro_liquid_density =
+         use_mass_storage
+         ? std::optional<ReducedMicroLiquidDensityData>{
+             solveReferenceReducedMicroLiquidDensity(
+                 n_L_prev, rho_LR, n5_prev, potential_exchange_params)}
+         : std::nullopt;
+     double const rho_L_prev =
+         prev_micro_liquid_density
+         ? n_L_prev * prev_micro_liquid_density->rho_LR
+         : 0.0;
+
+     auto const macro_potential =
+         computeYoungLaplaceMacroPotential(p_L, rho_LR, potential_exchange_params.pressure_tolerance);
+     double const alpha_M_effective = alpha_bar * rho_LR / mu;
+
+     auto const eval_exchange = [&](double const n_L)
+     {
+         double const active_n5 = referenceMicroSolidVolumeFraction(
+             n_L, phi, 0.0, n_L_prev, volumetric_strain, volumetric_strain_prev,
+             potential_exchange_params);
+         double const rho_LR_for_potential =
+             use_mass_storage
+             ? solveReferenceReducedMicroLiquidDensity(
+                 n_L, rho_LR, active_n5, potential_exchange_params)
+             : rho_LR;
+         auto const micro_potential = computeVanDerWaalsMicroPotential(
+             n_L, rho_LR_for_potential, active_n5,
+             potential_exchange_params.micro_solid_density_reference, potential_exchange_params.hamaker_constant,
+             potential_exchange_params.specific_surface,
+             microPotentialSignFactor(potential_exchange_params.micro_potential_convention));
+         auto const exchange = computePotentialDrivenMassExchange(
+             alpha_M_effective, macro_potential.mu_LR, micro_potential.mu_LR);
+         return std::pair{micro_potential, exchange};
+     };
+
+     auto const residual = [&](double const n_L)
+     {
+         auto const [micro_potential, exchange] = eval_exchange(n_L);
+         (void)micro_potential;

```

```

+     if (use_mass_storage)
+     {
+         double const active_n5 = referenceMicroSolidVolumeFraction(
+             n_l, phi, 0.0, n_l_prev, volumetric_strain,
+             volumetric_strain_prev, potential_exchange_params);
+         auto const micro_liquid_density =
+             solveReferenceReducedMicroLiquidDensity(
+                 n_l, rho_LR, active_n5, potential_exchange_params);
+         double residual = n_l * micro_liquid_density.rho_LR - rho_l_prev -
+             dt * exchange.rho_l_hat;
+         residual -= dt * n_l * micro_liquid_density.rho_LR *
+             volumetric_strain_rate;
+         return residual;
+     }
+
+     double residual = n_l - n_l_prev - dt * exchange.rho_l_hat / rho_LR;
+     if (potential_exchange_params.local_nonlinear_solve_mode !=
+         LocalNonlinearSolveMode::ScalarExchange)
+     {
+         residual -= dt * n_l * volumetric_strain_rate;
+     }
+     return residual;
+ };
+
+ double n_l = std::clamp(n_l_prev, n_l_floor, phi_ceiling);
+ constexpr int max_iterations = 40;
+ constexpr double residual_tolerance = 1e-14;
+ constexpr double increment_tolerance = 1e-14;
+ bool converged = false;
+
+ for (int iter = 0; iter < max_iterations; ++iter)
+ {
+     double const r = residual(n_l);
+     if (std::abs(r) <=
+         residual_tolerance * std::max(1.0, std::abs(n_l_prev)))
+     {
+         converged = true;
+         break;
+     }
+
+     double const h = 1e-8 * std::max(1.0, std::abs(n_l));
+     double const n_l_plus = n_l + h;
+     double const n_l_minus = std::max(n_l_floor, n_l - h);
+     double const denom = n_l_plus - n_l_minus;
+     EXPECT_GT(denom, 0.0);
+     if (!denom > 0.0)
+     {
+         return {};
+     }
+
+     double const jacobian =
+         (residual(n_l_plus) - residual(n_l_minus)) / denom;
+     EXPECT_TRUE(std::isfinite(jacobian));
+     EXPECT_GT(std::abs(jacobian), 1e-20);
+     if (!std::isfinite(jacobian) && std::abs(jacobian) > 1e-20)
+     {
+         return {};
+     }
+
+     double step = -r / jacobian;
+     double n_l_candidate = std::clamp(n_l + step, n_l_floor, phi_ceiling);
+
+     // Basic backtracking to keep the independently coded reference solve
+     // robust while remaining distinct from the production helper.
+     double candidate_residual = residual(n_l_candidate);
+     int backtracking_steps = 0;
+     while (std::abs(candidate_residual) > std::abs(r) &&
+         backtracking_steps < 12)
+     {
+         step *= 0.5;
+         n_l_candidate = std::clamp(n_l + step, n_l_floor, phi_ceiling);
+         candidate_residual = residual(n_l_candidate);
+         ++backtracking_steps;
+     }
+
+     if (std::abs(n_l_candidate - n_l) <=
+         increment_tolerance * std::max(1.0, std::abs(n_l)))
+     {
+         n_l = n_l_candidate;
+         converged = true;
+         break;
+     }
+
+     n_l = n_l_candidate;
+ }
+
+ EXPECT_TRUE(converged);
+ if (!converged)
+ {
+     return {};
+ }
+
+ auto const [micro_potential, exchange] = eval_exchange(n_l);
+ double const n_l_ref = std::max(
+     1e-16, potential_exchange_params.initial_micro_water_content.value_or(
+         potential_exchange_params.micro_solid_volume_fraction_reference));
+ double const phi_safe = std::max(0.0, phi);
+ double const phi_m = std::clamp(n_l, 0.0, phi_safe);
+
+ return {
+     .n_l = n_l,
+     .micro_potential = micro_potential,
+     .exchange = exchange,
+     .p_L_m = -rho_LR * micro_potential.mu_LR,
+     .S_L_m = n_l / n_l_ref,
+     .phi_M = phi_safe - phi_m,
+     .phi_m = phi_m,
+ };
+ }
+
+ double referenceDnLDpl(double const p_L, double const n_l_prev, double const dt,
+     double const rho_LR, double const alpha_bar,
+     double const mu, double const phi,
+     PotentialExchangeParameters const& potential_exchange_params,
+     double const volumetric_strain = 0.0,
+     double const volumetric_strain_prev = 0.0)
+ {

```

```

+ double const h = 1e-8 * std::max(1.0, std::abs(p_L));
+ auto const plus = solveDsmMicromacroReferenceSinglePoint(
+     p_L + h, n_L_prev, dt, rho_LR, alpha_bar, mu, phi, potential_exchange_params,
+     volumetric_strain, volumetric_strain_prev);
+ auto const minus = solveDsmMicromacroReferenceSinglePoint(
+     p_L - h, n_L_prev, dt, rho_LR, alpha_bar, mu, phi, potential_exchange_params,
+     volumetric_strain, volumetric_strain_prev);
+ return (plus.n_L - minus.n_L) / (2.0 * h);
+}
+
+double referenceDrhoLRHatDpL(double const p_L, double const n_L_prev,
+    double const dt, double const rho_LR,
+    double const alpha_bar, double const mu,
+    double const phi,
+    PotentialExchangeParameters const& potential_exchange_params,
+    double const volumetric_strain = 0.0,
+    double const volumetric_strain_prev = 0.0)
+{
+    double const h = 1e-8 * std::max(1.0, std::abs(p_L));
+    auto const plus = solveDsmMicromacroReferenceSinglePoint(
+        p_L + h, n_L_prev, dt, rho_LR, alpha_bar, mu, phi, potential_exchange_params,
+        volumetric_strain, volumetric_strain_prev);
+    auto const minus = solveDsmMicromacroReferenceSinglePoint(
+        p_L - h, n_L_prev, dt, rho_LR, alpha_bar, mu, phi, potential_exchange_params,
+        volumetric_strain, volumetric_strain_prev);
+    return ((-plus.exchange.rho_L_hat) - (-minus.exchange.rho_L_hat)) /
+        (2.0 * h);
+}
+
+enum class CoupledExchangeReferenceMode
+{
+    pressure_proxy,
+    full_potential_vdw
+};
+
+struct RepresentativeCoupledExchangeState
+{
+    char const* name = "";
+    CoupledExchangeReferenceMode mode =
+        CoupledExchangeReferenceMode::pressure_proxy;
+    double p_L = 0.0;
+    double p_L_m = 0.0;
+    double pressure_tolerance = 0.0;
+    double n_L_prev = 0.0;
+    double dt = 0.0;
+    double rho_LR = 0.0;
+    double drho_LR_dpl = 0.0;
+    double alpha_bar = 0.0;
+    double mu = 0.0;
+    double phi = 0.0;
+    double volumetric_strain = 0.0;
+    double volumetric_strain_prev = 0.0;
+};
+
+double linearizedDensityAtPressure(double const p_L_eval, double const p_L_ref,
+    double const rho_LR_ref,
+    double const drho_LR_dpl)
+{
+    return std::max(1e-16, rho_LR_ref + drho_LR_dpl * (p_L_eval - p_L_ref));
+}
+
+double referenceCoupledRhoLRHat(
+    RepresentativeCoupledExchangeState const& state, double const p_L_eval,
+    PotentialExchangeParameters const& potential_exchange_params)
+{
+    double const rho_LR_eval = linearizedDensityAtPressure(
+        p_L_eval, state.p_L, state.rho_LR, state.drho_LR_dpl);
+
+    if (state.mode == CoupledExchangeReferenceMode::pressure_proxy)
+    {
+        double const alpha_M_effective = state.alpha_bar * rho_LR_eval / state.mu;
+        double const mu_LR_active = p_L_eval / rho_LR_eval;
+        double const mu_LR_active = state.p_L_m / rho_LR_eval;
+        auto const exchange = computePotentialDrivenMassExchange(
+            alpha_M_effective, mu_LR_active, mu_LR_active);
+        return exchange.rho_L_hat;
+    }
+
+    auto potential_exchange_params_eval = potential_exchange_params;
+    potential_exchange_params_eval.pressure_tolerance =
+        state.pressure_tolerance;
+    auto const reference = solveDsmMicromacroReferenceSinglePoint(
+        p_L_eval, state.n_L_prev, state.dt, rho_LR_eval, state.alpha_bar,
+        state.mu, state.phi, potential_exchange_params_eval,
+        state.volumetric_strain,
+        state.volumetric_strain_prev);
+    return reference.exchange.rho_L_hat;
+}
+
+double referenceCoupledDrhoLRHatDpL(
+    RepresentativeCoupledExchangeState const& state,
+    PotentialExchangeParameters const& potential_exchange_params)
+{
+    double const h = 1e-8 * std::max(1.0, std::abs(state.p_L));
+    double const plus = referenceCoupledRhoLRHat(state, state.p_L + h, potential_exchange_params);
+    double const minus = referenceCoupledRhoLRHat(state, state.p_L - h, potential_exchange_params);
+    return (plus - minus) / (2.0 * h);
+}
+
+struct ProductionCoupledExchangeData
+{
+    double rho_L_hat = 0.0;
+    double drho_L_hat_dpl = 0.0;
+    bool converged = true;
+};
+
+ProductionCoupledExchangeData productionCoupledExchangeData(
+    RepresentativeCoupledExchangeState const& state,
+    PotentialExchangeParameters const& potential_exchange_params)
+{
+    double const beta_LR = state.drho_LR_dpl / state.rho_LR;
+
+    if (state.mode == CoupledExchangeReferenceMode::pressure_proxy)
+    {
+        auto const data = computePotentialExchangeUpdate(
+            state.alpha_bar, state.mu, state.p_L, state.p_L_m, state.rho_LR,
+            beta_LR, state.pressure_tolerance, false, false, 0.0, 0.0,

```

```

+         false, 0.0, false,
+         potential_exchange_params.fd_jacobian_perturbation);
+     return {
+         .rho_L_hat = data.exchange.rho_L_hat,
+         .drho_L_hat_dpl = data.drho_L_hat_dpl_direct,
+         .converged = true,
+     };
+ }
+
+ auto const macro_potential = computeYoungLaplaceMacroPotential(
+     state.p_L, state.rho_LR, state.pressure_tolerance);
+ auto const n_L_update = solveImplicitMicroWaterContent(
+     state.n_L_prev, state.dt, state.rho_LR, state.alpha_bar, state.mu,
+     macro_potential,
+     {phi = state.phi,
+      .volumetric_strain = state.volumetric_strain,
+      .volumetric_strain_prev = state.volumetric_strain_prev},
+     potential_exchange_params);
+ double const dn_L_dpl = computeImplicitNLDpl(
+     state.n_L_prev, state.p_L, state.dt, state.rho_LR,
+     state.drho_LR_dpl, state.alpha_bar, state.mu,
+     macro_potential, n_L_update.micro_potential, n_L_update.exchange,
+     {phi = state.phi,
+      .volumetric_strain = state.volumetric_strain,
+      .volumetric_strain_prev = state.volumetric_strain_prev},
+     potential_exchange_params);
+ double const dmu_LR_vdw_dpl =
+     n_L_update.micro_potential.dmu_LR_dnl * dn_L_dpl +
+     n_L_update.micro_potential.dmu_LR_drho_LR * state.drho_LR_dpl;
+
+ auto const data = computePotentialExchangeUpdate(
+     state.alpha_bar, state.mu, state.p_L, state.p_L_m, state.rho_LR,
+     beta_LR, state.pressure_tolerance, true, true,
+     n_L_update.micro_potential.mu_LR,
+     n_L_update.micro_potential.dmu_LR_drho_LR, true, dmu_LR_vdw_dpl,
+     false, potential_exchange_params.fd_jacobian_perturbation);
+
+     return {
+         .rho_L_hat = data.exchange.rho_L_hat,
+         .drho_L_hat_dpl = data.drho_L_hat_dpl_direct,
+         .converged = n_L_update.converged,
+     };
+ }
+ // namespace
+
+ *TEST(RichardsMechanics, DSMMacroSingleIntegrationPointReferencePath)
+ {
+     PotentialExchangeParameters potential_exchange_params;
+     potential_exchange_params.enabled = true;
+     potential_exchange_params.pressure_tolerance = 0.0;
+     potential_exchange_params.hamaker_constant = 1e-30;
+     potential_exchange_params.specific_surface = 1.0;
+     potential_exchange_params.micro_solid_density_reference = 2650.0;
+     potential_exchange_params.micro_solid_volume_fraction_reference = 0.6;
+     potential_exchange_params.initial_micro_water_content = 0.1;
+     potential_exchange_params.local_jacobian_perturbation = 1e-8;
+
+     double const p_L = -1.0e7;
+     double const n_L_prev = 0.1;
+     double const dt = 100.0;
+     double const rho_LR = 1000.0;
+     double const drho_LR_dpl = 0.0;
+     double const alpha_bar = 1.0e-13;
+     double const mu = 1.0e-3;
+     double const phi = 0.4;
+     double const phi_prev = 0.4;
+
+     auto const macro_potential =
+         computeYoungLaplaceMacroPotential(p_L, rho_LR, potential_exchange_params.pressure_tolerance);
+     auto const ogs_update = solveImplicitMicroWaterContent(
+         n_L_prev, dt, rho_LR, alpha_bar, mu, macro_potential,
+         {phi = phi, .volumetric_strain = 0.0, .volumetric_strain_prev = 0.0},
+         potential_exchange_params);
+     ASSERT_TRUE(ogs_update.converged);
+
+     auto const reference = solveDSMMacroReferenceSinglePoint(
+         p_L, n_L_prev, dt, rho_LR, alpha_bar, mu, phi, potential_exchange_params);
+
+     EXPECT_NEAR(ogs_update.n_L, reference.n_L,
+         comparisonTolerance(ogs_update.n_L, reference.n_L));
+     EXPECT_NEAR(ogs_update.micro_potential.mu_LR, reference.micro_potential.mu_LR,
+         comparisonTolerance(ogs_update.micro_potential.mu_LR,
+             reference.micro_potential.mu_LR,
+             1e-10, 1e-18));
+     EXPECT_NEAR(ogs_update.exchange.rho_L_hat, reference.exchange.rho_L_hat,
+         comparisonTolerance(ogs_update.exchange.rho_L_hat,
+             reference.exchange.rho_L_hat,
+             1e-10, 1e-18));
+     EXPECT_NEAR(ogs_update.exchange.rho_L_hat, reference.exchange.rho_L_hat,
+         comparisonTolerance(ogs_update.exchange.rho_L_hat,
+             reference.exchange.rho_L_hat,
+             1e-10, 1e-18));
+
+     auto const compatibility_output =
+         computeCompatibilityMicroHydraulicOutput(ogs_update.n_L, rho_LR, potential_exchange_params);
+     EXPECT_NEAR(compatibility_output.p_L_m, reference.p_L_m,
+         comparisonTolerance(compatibility_output.p_L_m,
+             reference.p_L_m, 1e-10, 1e-12));
+     EXPECT_NEAR(compatibility_output.S_L_m, reference.S_L_m,
+         comparisonTolerance(compatibility_output.S_L_m,
+             reference.S_L_m));
+
+     auto const transport_porosity_update =
+         computeTransportPorosityUpdate(
+             phi, phi_prev - n_L_prev, n_L_prev, ogs_update.n_L,
+             /*volumetric_strain=*/0.0, /*volumetric_strain_prev=*/0.0,
+             MacroPorosityUpdateMode::AlgebraicSplint);
+     EXPECT_NEAR(transport_porosity_update.phi_M, reference.phi_M,
+         comparisonTolerance(transport_porosity_update.phi_M,
+             reference.phi_M));
+     EXPECT_NEAR(transport_porosity_update.phi_m, reference.phi_m,
+         comparisonTolerance(transport_porosity_update.phi_m,
+             reference.phi_m));
+     EXPECT_NEAR(transport_porosity_update.phi_M_prev, phi_prev - n_L_prev,
+         comparisonTolerance(transport_porosity_update.phi_M_prev,
+             phi_prev - n_L_prev));
+     EXPECT_NEAR(transport_porosity_update.phi_m_prev, n_L_prev,

```

```

+         comparisonTolerance(transport_porosity_update.phi_m_prev,
+                               n_l_prev));
+
+ double const analytic_dn_l_dpl = computeImplicitNDpl(
+     n_l_prev, p_L, dt, rho_LR, drho_LR_dpl, alpha_bar, mu, macro_potential,
+     ogs_update.micro_potential, ogs_update.exchange,
+     {phi = phi, .volumetric_strain = 0.0, .volumetric_strain_prev = 0.0},
+     potential_exchange_params);
+ double const reference_dn_l_dpl = referenceDnDpl(
+     p_L, n_l_prev, dt, rho_LR, alpha_bar, mu, phi, potential_exchange_params);
+
+ EXPECT_NEAR(analytic_dn_l_dpl, reference_dn_l_dpl,
+             comparisonTolerance(analytic_dn_l_dpl, reference_dn_l_dpl,
+                                 5e-5, 1e-18));
+
+ auto const fd_diagnostic = computeLocalJacobianDiagnosticData(
+     n_l_prev, p_L, dt, rho_LR, drho_LR_dpl, alpha_bar, mu,
+     potential_exchange_params.pressure_tolerance,
+     {phi = phi, .volumetric_strain = 0.0, .volumetric_strain_prev = 0.0},
+     potential_exchange_params);
+ EXPECT_NEAR(fd_diagnostic.fd_dn_l_dpl, reference_dn_l_dpl,
+             comparisonTolerance(fd_diagnostic.fd_dn_l_dpl,
+                                 reference_dn_l_dpl, 5e-5, 1e-18));
+
+ double const reference_drho_L_hat_dpl = referenceDrhoHatDpl(
+     p_L, n_l_prev, dt, rho_LR, alpha_bar, mu, phi, potential_exchange_params);
+ EXPECT_NEAR(fd_diagnostic.fd_drho_L_hat_dpl, reference_drho_L_hat_dpl,
+             comparisonTolerance(fd_diagnostic.fd_drho_L_hat_dpl,
+                                 reference_drho_L_hat_dpl,
+                                 5e-5, 1e-18));
+
+ }
+
+ TEST(RichardsMechanics, DSMMicroMacroBranchSensitivityNearMacroPotentialTransition)
+ {
+     PotentialExchangeParameters potential_exchange_params;
+     potential_exchange_params.enabled = true;
+     potential_exchange_params.pressure_tolerance = 100.0;
+     potential_exchange_params.hamaker_constant = 1e-30;
+     potential_exchange_params.specific_surface = 1.0;
+     potential_exchange_params.micro_solid_density_reference = 2650.0;
+     potential_exchange_params.micro_solid_volume_fraction_reference = 0.6;
+     potential_exchange_params.initial_micro_water_content = 0.1;
+     potential_exchange_params.local_jacobian_perturbation = 1e-8;
+
+     double const n_l_prev = 0.1;
+     double const dt = 100.0;
+     double const rho_LR = 1000.0;
+     double const alpha_bar = 1.0e-13;
+     double const mu = 1.0e-3;
+     double const phi = 0.4;
+
+     std::array<double, 5> const pressures = {
+         -150.0,
+         -100.0,
+         -99.999,
+         -50.0,
+         0.0,
+     };
+
+     std::array<bool, 5> const saturated_expectation = {
+         false,
+         false,
+         true,
+         true,
+         true,
+     };
+
+     struct CaseResult
+     {
+         double p_L = 0.0;
+         bool saturated_branch = false;
+         double mu_LR = 0.0;
+         double n_l = 0.0;
+         double rho_L_hat = 0.0;
+         double p_L_m = 0.0;
+         double S_L_m = 0.0;
+     };
+
+     std::array<CaseResult, 5> results;
+
+     for (std::size_t i = 0; i < pressures.size(); ++i)
+     {
+         double const p_L = pressures[i];
+         auto const macro_potential = computeYoungLaplaceMacroPotential(
+             p_L, rho_LR, potential_exchange_params.pressure_tolerance);
+
+         EXPECT_EQ(macro_potential.saturated_branch, saturated_expectation[i]);
+         if (saturated_expectation[i])
+         {
+             EXPECT_DOUBLE_EQ(macro_potential.mu_LR, 0.0);
+             EXPECT_DOUBLE_EQ(macro_potential.dmu_LR_dpl, 0.0);
+         }
+         else
+         {
+             EXPECT_LT(macro_potential.mu_LR, 0.0);
+             EXPECT_DOUBLE_EQ(macro_potential.mu_LR, p_L / rho_LR);
+         }
+
+         auto const ogs_update = solveImplicitMicroWaterContent(
+             n_l_prev, dt, rho_LR, alpha_bar, mu, macro_potential,
+             {phi = phi, .volumetric_strain = 0.0, .volumetric_strain_prev = 0.0},
+             potential_exchange_params);
+         ASSERT_TRUE(ogs_update.converged);
+
+         auto const reference = solveDSMMicroMacroReferenceSinglePoint(
+             p_L, n_l_prev, dt, rho_LR, alpha_bar, mu, phi, potential_exchange_params);
+         auto const compatibility_output =
+             computeCompatibilityMicroHydraulicOutput(ogs_update.n_l, rho_LR,
+                                                       potential_exchange_params);
+
+         EXPECT_NEAR(ogs_update.n_l, reference.n_l,
+             comparisonTolerance(ogs_update.n_l, reference.n_l));
+         EXPECT_NEAR(ogs_update.exchange.rho_L_hat, reference.exchange.rho_L_hat,
+             comparisonTolerance(ogs_update.exchange.rho_L_hat,
+                                 reference.exchange.rho_L_hat,
+                                 1e-10, 1e-18));
+         EXPECT_NEAR(compatibility_output.p_L_m, reference.p_L_m,
+             comparisonTolerance(compatibility_output.p_L_m,

```

```

+         reference.p_L_m, 1e-10, 1e-12));
+ EXPECT_NEAR(compatibility_output.S_L_m, reference.S_L_m,
+             comparisonTolerance(compatibility_output.S_L_m,
+                                 reference.S_L_m));
+
+ // Current kept DSM branch: mu_LR <= 0 on the macro side, mu_LR > 0 on
+ // the vdw microscale side. The exchange law therefore stays
+ // sign-locked to non-increasing micro water content.
+ EXPECT_LE(ogs_update.exchange.rho_L_hat, 0.0);
+ EXPECT_LE(ogs_update.n_L,
+           n_L_prev + comparisonTolerance(ogs_update.n_L, n_L_prev,
+                                           0.0, 1e-18));
+
+ results[i] = {
+     .p_L = p_L,
+     .saturated_branch = macro_potential.saturated_branch,
+     .mu_LR = macro_potential.mu_LR,
+     .n_L = ogs_update.n_L,
+     .rho_L_hat = ogs_update.exchange.rho_L_hat,
+     .p_L_m = compatibility_output.p_L_m,
+     .S_L_m = compatibility_output.S_L_m,
+ };
+ }
+
+ // Once the macro state is on the saturated helper branch, the active macro
+ // potential is identically zero, so the local DSM update becomes invariant
+ // with respect to further increases in p_L as long as rho_LR stays fixed.
+ for (std::size_t i = 3; i < results.size(); ++i)
+ {
+     EXPECT_NEAR(results[2].n_L, results[i].n_L,
+                 comparisonTolerance(results[2].n_L, results[i].n_L));
+     EXPECT_NEAR(results[2].rho_L_hat, results[i].rho_L_hat,
+                 comparisonTolerance(results[2].rho_L_hat,
+                                     results[i].rho_L_hat, 1e-10, 1e-18));
+     EXPECT_NEAR(results[2].p_L_m, results[i].p_L_m,
+                 comparisonTolerance(results[2].p_L_m, results[i].p_L_m,
+                                     1e-10, 1e-12));
+     EXPECT_NEAR(results[2].S_L_m, results[i].S_L_m,
+                 comparisonTolerance(results[2].S_L_m, results[i].S_L_m));
+ }
+
+ // The kept branch remains monotone with respect to p_L: less negative
+ // pressures produce less drying, but never net wetting.
+ for (std::size_t i = 1; i < results.size(); ++i)
+ {
+     double const n_L_tolerance =
+         comparisonTolerance(results[i - 1].n_L, results[i].n_L);
+     EXPECT_LE(results[i - 1].n_L, results[i].n_L + n_L_tolerance);
+
+     double const rho_L_hat_tolerance = comparisonTolerance(
+         results[i - 1].rho_L_hat, results[i].rho_L_hat, 1e-10, 1e-18);
+     EXPECT_LE(results[i - 1].rho_L_hat,
+               results[i].rho_L_hat + rho_L_hat_tolerance);
+ }
+ }
+
+ TEST(RichardsMechanics, DSMMicroMacroNegativeAttractiveMicroPotentialAdmitsWetting)
+ {
+     PotentialExchangeParameters potential_exchange_params;
+     potential_exchange_params.enabled = true;
+     potential_exchange_params.pressure_tolerance = 1.0;
+     potential_exchange_params.hamaker_constant = 6.0e-20;
+     potential_exchange_params.specific_surface = 1000.0;
+     potential_exchange_params.micro_solid_density_reference = 2650.0;
+     potential_exchange_params.micro_solid_volume_fraction_reference = 0.6;
+     potential_exchange_params.micro_potential_convention =
+         MicroPotentialConvention::NegativeAttractive;
+     potential_exchange_params.initial_micro_water_content = 0.03;
+     potential_exchange_params.local_jacobian_perturbation = 1e-8;
+
+     double const p_L = 0.0;
+     double const n_L_prev = 0.03;
+     double const dt = 1.0;
+     double const rho_LR = 1000.0;
+     double const alpha_bar = 1.0e-9;
+     double const mu = 1.0e-3;
+     double const phi = 0.4;
+
+     auto const macro_potential =
+         computeYoungLaplaceMacroPotential(p_L, rho_LR, potential_exchange_params.pressure_tolerance);
+     ASSERT_TRUE(macro_potential.saturated_branch);
+     ASSERT_DOUBLE_EQ(macro_potential.mu_LR, 0.0);
+
+     auto const ogs_update = solveImplicitMicroWaterContent(
+         n_L_prev, dt, rho_LR, alpha_bar, mu, macro_potential,
+         {phi = phi, .volumetric_strain = 0.0, .volumetric_strain_prev = 0.0},
+         potential_exchange_params);
+     ASSERT_TRUE(ogs_update.converged);
+
+     auto const reference = solveDsmMicromacroReferenceSinglePoint(
+         p_L, n_L_prev, dt, rho_LR, alpha_bar, mu, phi, potential_exchange_params);
+     auto const compatibility_output =
+         computeCompatibilityMicroHydraulicOutput(ogs_update.n_L, rho_LR, potential_exchange_params);
+
+     EXPECT_LT(ogs_update.micro_potential.mu_LR, 0.0);
+     EXPECT_GT(ogs_update.exchange.rho_L_hat, 0.0);
+     EXPECT_LT(ogs_update.exchange.rho_L_hat, 0.0);
+     EXPECT_GT(ogs_update.n_L, n_L_prev);
+     EXPECT_GT(compatibility_output.p_L_m, 0.0);
+
+     EXPECT_NEAR(ogs_update.n_L, reference.n_L,
+                 comparisonTolerance(ogs_update.n_L, reference.n_L));
+     EXPECT_NEAR(ogs_update.exchange.rho_L_hat, reference.exchange.rho_L_hat,
+                 comparisonTolerance(ogs_update.exchange.rho_L_hat,
+                                     reference.exchange.rho_L_hat,
+                                     1e-10, 1e-18));
+     EXPECT_NEAR(compatibility_output.p_L_m, reference.p_L_m,
+                 comparisonTolerance(compatibility_output.p_L_m,
+                                     reference.p_L_m, 1e-10, 1e-12));
+     EXPECT_NEAR(compatibility_output.S_L_m, reference.S_L_m,
+                 comparisonTolerance(compatibility_output.S_L_m,
+                                     reference.S_L_m));
+ }
+
+ TEST(RichardsMechanics, DSMMicroMacroVdwRelaxationStressIncrement)
+ {
+     PotentialExchangeParameters potential_exchange_params;

```



```

+ potential_exchange_params.enabled = true;
+ potential_exchange_params.micro_potential_convention =
+   MicroPotentialConvention::NegativeAttractive;
+ potential_exchange_params.vdw_relaxation_stress_gain = 10.0;
+
+
+ auto const& identity2 = MathLib::KelvinVector::Invariants<
+   MathLib::KelvinVector::kelvin_vector_dimensions(2)>::identity2;
+
+
+ auto const compressive_increment =
+   computeVdwRelaxationStressIncrement<2>(5.0, 3.0, potential_exchange_params);
+ EXPECT_NEAR((compressive_increment + 20.0 * identity2).norm(), 0.0, 1e-14);
+
+
+ auto const no_relaxation_increment =
+   computeVdwRelaxationStressIncrement<2>(3.0, 5.0, potential_exchange_params);
+ EXPECT_NEAR(no_relaxation_increment.norm(), 0.0, 1e-14);
+
+
+ potential_exchange_params.vdw_relaxation_stress_gain = 0.0;
+ auto const zero_gain_increment =
+   computeVdwRelaxationStressIncrement<2>(5.0, 3.0, potential_exchange_params);
+ EXPECT_NEAR(zero_gain_increment.norm(), 0.0, 1e-14);
+
+
+ potential_exchange_params.vdw_relaxation_stress_gain = 10.0;
+ potential_exchange_params.micro_potential_convention = MicroPotentialConvention::PositiveReduced;
+ auto const unsupported_convention_increment =
+   computeVdwRelaxationStressIncrement<2>(5.0, 3.0, potential_exchange_params);
+ EXPECT_NEAR(unsupported_convention_increment.norm(), 0.0, 1e-14);
+}
+
+
+TEST(RichardsMechanics, DSWMicroMacroMicroWaterContentStressIncrement)
+{
+   PotentialExchangeParameters potential_exchange_params;
+   potential_exchange_params.enabled = true;
+   potential_exchange_params.micro_water_content_stress_gain = 10.0;
+
+
+   auto const& identity2 = MathLib::KelvinVector::Invariants<
+     MathLib::KelvinVector::kelvin_vector_dimensions(2)>::identity2;
+
+
+   auto const compressive_increment =
+     computeMicroWaterContentStressIncrement<2>(0.2, 0.3, potential_exchange_params);
+   EXPECT_NEAR((compressive_increment + 1.0 * identity2).norm(), 0.0, 1e-14);
+
+
+   auto const no_growth_increment =
+     computeMicroWaterContentStressIncrement<2>(0.3, 0.2, potential_exchange_params);
+   EXPECT_NEAR(no_growth_increment.norm(), 0.0, 1e-14);
+
+
+   potential_exchange_params.micro_water_content_stress_gain = 0.0;
+   auto const zero_gain_increment =
+     computeMicroWaterContentStressIncrement<2>(0.2, 0.3, potential_exchange_params);
+   EXPECT_NEAR(zero_gain_increment.norm(), 0.0, 1e-14);
+}
+
+
+TEST(RichardsMechanics, DSWMicroMacroMicroPorositySwellingStressIncrement)
+{
+   using KM = MathLib::KelvinVector::KelvinMatrixType<2>;
+
+
+   PotentialExchangeParameters potential_exchange_params;
+   potential_exchange_params.enabled = true;
+   potential_exchange_params.micro_water_content_swelling_slope = 0.1;
+
+
+   auto const& identity2 = MathLib::KelvinVector::Invariants<
+     MathLib::KelvinVector::kelvin_vector_dimensions(2)>::identity2;
+   KM C_el = KM::Identity();
+
+
+   auto const loading_increment =
+     computeReferenceMicroPorositySwellingStressIncrement<2>(
+       0.2, 0.3, C_el, potential_exchange_params);
+   auto const expected_loading = -(0.1 * (0.3 - 0.2) / 3.0) * identity2;
+   EXPECT_NEAR((loading_increment - expected_loading).norm(), 0.0, 1e-14);
+
+
+   auto const unloading_increment =
+     computeReferenceMicroPorositySwellingStressIncrement<2>(
+       0.3, 0.2, C_el, potential_exchange_params);
+   auto const expected_unloading = -(0.1 * (0.2 - 0.3) / 3.0) * identity2;
+   EXPECT_NEAR((unloading_increment - expected_unloading).norm(), 0.0, 1e-14);
+
+
+   potential_exchange_params.micro_water_content_swelling_slope = 0.0;
+   auto const disabled_increment =
+     computeReferenceMicroPorositySwellingStressIncrement<2>(
+       0.2, 0.3, C_el, potential_exchange_params);
+   EXPECT_NEAR(disabled_increment.norm(), 0.0, 1e-14);
+}
+
+
+TEST(RichardsMechanics, DSWMicroMacroAlignedSwellingIgnoresExploratoryGains)
+{
+   using KM = MathLib::KelvinVector::KelvinMatrixType<2>;
+   using KV = MathLib::KelvinVector::KelvinVectorType<2>;
+
+
+   PotentialExchangeParameters potential_exchange_params;
+   potential_exchange_params.enabled = true;
+   potential_exchange_params.local_nonLinear_solve_mode =
+     LocalNonLinearSolveMode::ScalarReferenceMassStorage;
+   potential_exchange_params.micro_water_content_swelling_slope = 0.1;
+   potential_exchange_params.vdw_relaxation_stress_gain = 100.0;
+   potential_exchange_params.micro_water_content_stress_gain = 100.0;
+
+
+   KM C_el = KM::Identity();
+   double const phi_m_prev = 0.2;
+   double const phi_m = 0.3;
+   double const n_l_prev = 0.2;
+   double const n_l = 0.3;
+   double const p_L_m_prev = 5.0;
+   double const p_L_m = 3.0;
+
+
+   KV const expected =
+     computeReferenceMicroPorositySwellingStressIncrement<2>(
+       phi_m_prev, phi_m, C_el, potential_exchange_params);
+   KV const actual =
+     computeSwellingStressIncrement<2>(
+       phi_m_prev, phi_m, n_l_prev, n_l, p_L_m_prev, p_L_m, C_el, potential_exchange_params);
+
+   EXPECT_NEAR((actual - expected).norm(), 0.0, 1e-14);
+}
+
+
+TEST(RichardsMechanics, DSWMicroMacroTransportPorositySplitRecomposesTotalPorosity)
+{
+   auto const split = computeTransportPorosityUpdate(
+     0.4, 0.27, 0.08, 0.1, 0.0, 0.0,

```

```

+ MacroPorosityUpdateMode::AlgebraicSplit);
+
+ EXPECT_NEAR(split.phi_m, 0.1, 1e-14);
+ EXPECT_NEAR(split.phi_M, 0.3, 1e-14);
+ EXPECT_NEAR(split.phi_m_prev, 0.08, 1e-14);
+ EXPECT_NEAR(split.phi_M_prev, 0.27, 1e-14);
+ EXPECT_NEAR(split.phi_M + split.phi_m, 0.4, 1e-14);
+ EXPECT_NEAR(split.phi_M_prev + split.phi_m_prev, 0.35, 1e-14);
+
+ auto const clamped = computeTransportPorosityUpdate(
+     0.25, 0.0, 0.2, 0.4, 0.0, 0.0,
+     MacroPorosityUpdateMode::AlgebraicSplit);
+ EXPECT_NEAR(clamped.phi_m, 0.25, 1e-14);
+ EXPECT_NEAR(clamped.phi_M, 0.0, 1e-14);
+ EXPECT_NEAR(clamped.phi_M + clamped.phi_m, 0.25, 1e-14);
+ }
+
+ TEST(RichardsMechanics, DSMMicroMacroAdditiveMacroPorosityRateUpdate)
+ {
+     double const phi_M_prev = 0.30;
+     double const phi_m_prev = 0.10;
+     double const phi_m = 0.11;
+     double const volumetric_strain_prev = 0.0;
+     double const volumetric_strain = 1.0e-3;
+
+     auto const split = computeTransportPorosityUpdate(
+         0.4, phi_M_prev, phi_m_prev, phi_m, volumetric_strain,
+         volumetric_strain_prev,
+         MacroPorosityUpdateMode::ReferenceAdditiveRate);
+
+     double const delta_eps_v = volumetric_strain - volumetric_strain_prev;
+     double const expected_phi_M =
+         (phi_M_prev + (1.0 - phi_m) * delta_eps_v - (phi_m - phi_m_prev)) /
+         (1.0 + delta_eps_v);
+
+     EXPECT_NEAR(split.phi_m, phi_m, 1e-14);
+     EXPECT_NEAR(split.phi_m_prev, phi_m_prev, 1e-14);
+     EXPECT_NEAR(split.phi_M_prev, phi_M_prev, 1e-14);
+     EXPECT_NEAR(split.phi_M, expected_phi_M, 1e-14);
+     EXPECT_NEAR(split.phi_M + split.phi_m, expected_phi_M + phi_m, 1e-14);
+     EXPECT_GT(split.phi_M + split.phi_m, phi_M_prev + phi_m_prev);
+ }
+
+ TEST(RichardsMechanics, DSMMicroMacroCurrentPorositySplitMicroSolidFractionMode)
+ {
+     PotentialExchangeParameters potential_exchange_params;
+     potential_exchange_params.hamaker_constant = 6.0e-20;
+     potential_exchange_params.specific_surface = 4000.0;
+     potential_exchange_params.micro_solid_density_reference = 2650.0;
+     potential_exchange_params.micro_solid_volume_fraction_reference = 0.6;
+     potential_exchange_params.micro_solid_volume_fraction_mode =
+         MicroSolidVolumeFractionMode::CurrentPorositySplit;
+     potential_exchange_params.macro_porosity_update_mode = MacroPorosityUpdateMode::AlgebraicSplit;
+     potential_exchange_params.initial_micro_water_content = 0.1;
+
+     double const n_l = 0.1;
+     double const rho_LR = 1000.0;
+     PotentialExchangeLocalSolveContext const local_context{
+         .phi = 0.25,
+         .phi_M_prev = 0.15,
+         .phi_m_prev = 0.1,
+         .volumetric_strain = 0.0,
+         .volumetric_strain_prev = 0.0};
+
+     auto const active_nS =
+         computeActiveMicroSolidVolumeFraction(n_l, local_context, potential_exchange_params);
+     EXPECT_NEAR(active_nS, 0.75, 1e-12);
+
+     auto const active_output = computeCompatibilityMicroHydraulicOutput(
+         n_l, rho_LR, local_context, potential_exchange_params);
+     auto const reference_output =
+         computeCompatibilityMicroHydraulicOutput(n_l, rho_LR, potential_exchange_params);
+
+     double const expected_ratio =
+         std::pow(active_nS / potential_exchange_params.micro_solid_volume_fraction_reference, 3.0);
+     EXPECT_NEAR(active_output.micro_potential.mu_LR /
+         reference_output.micro_potential.mu_LR,
+         expected_ratio, 1e-12);
+ }
+
+ TEST(RichardsMechanics, DSMMicroMacroReducedMicroLiquidDensityEOSReferencePath)
+ {
+     PotentialExchangeParameters potential_exchange_params;
+     potential_exchange_params.micro_solid_density_reference = 2650.0;
+     potential_exchange_params.micro_solid_volume_fraction_reference = 0.8;
+     potential_exchange_params.micro_liquid_density_reference = 1300.0;
+     potential_exchange_params.micro_liquid_density_a = 1.3;
+     potential_exchange_params.micro_liquid_density_b = 1.0;
+     potential_exchange_params.local_nonlinear_solve_mode =
+         LocalNonlinearSolveMode::ScalarReferenceMassStorage;
+
+     double const n_l = 0.1;
+     double const rho_LR = 1000.0;
+     double const nS = 0.8;
+
+     auto const production =
+         computeReducedMicroLiquidDensity(n_l, rho_LR, nS, potential_exchange_params);
+     auto const reference =
+         solveReferenceReducedMicroLiquidDensity(n_l, rho_LR, nS, potential_exchange_params);
+
+     EXPECT_NEAR(production.rho_LR, reference.rho_LR,
+         comparisonTolerance(production.rho_LR, reference.rho_LR,
+             1e-9, 1e-14));
+     EXPECT_NEAR(production.omega_l, reference.omega_l,
+         comparisonTolerance(production.omega_l, reference.omega_l,
+             1e-9, 1e-14));
+     EXPECT_NEAR(production.drho_l_dn_l, reference.drho_l_dn_l,
+         comparisonTolerance(production.drho_l_dn_l,
+             reference.drho_l_dn_l, 1e-7, 1e-12));
+ }
+
+ TEST(RichardsMechanics, DSMMicroMacroScalarStorageLocalSolveReferencePath)
+ {
+     PotentialExchangeParameters potential_exchange_params;
+     potential_exchange_params.enabled = true;
+     potential_exchange_params.pressure_tolerance = 0.0;
+     potential_exchange_params.hamaker_constant = 6.0e-20;

```

```

+ potential_exchange_params.specific_surface = 1000.0;
+ potential_exchange_params.micro_solid_density_reference = 2650.0;
+ potential_exchange_params.micro_solid_volume_fraction_reference = 0.6;
+ potential_exchange_params.micro_potential_convention =
+   MicroPotentialConvention::NegativeAttractive;
+ potential_exchange_params.local_nonlinear_solve_mode =
+   LocalNonLinearSolveMode::ScalarReferenceStorage;
+ potential_exchange_params.initial_micro_water_content = 0.03;
+
+ double const p_L = 0.0;
+ double const n_L_prev = 0.03;
+ double const dt = 100.0;
+ double const rho_LR = 1000.0;
+ double const alpha_bar = 1.0e-9;
+ double const mu = 1.0e-3;
+ double const phi = 0.031;
+ double const volumetric_strain_prev = 0.0;
+ double const volumetric_strain = 1.0e-3;
+
+ auto const macro_potential =
+   computeYoungLaplaceMacroPotential(p_L, rho_LR, potential_exchange_params.pressure_tolerance);
+ auto const ogs_update = solveImplicitMicroWaterContent(
+   n_L_prev, dt, rho_LR, alpha_bar, mu, macro_potential,
+   {phi = phi,
+    .volumetric_strain = volumetric_strain,
+    .volumetric_strain_prev = volumetric_strain_prev},
+   potential_exchange_params);
+ ASSERT_TRUE(ogs_update.converged);
+
+ auto const reference = solveDsmMicromacroReferenceSinglePoint(
+   p_L, n_L_prev, dt, rho_LR, alpha_bar, mu, phi, potential_exchange_params,
+   volumetric_strain, volumetric_strain_prev);
+ EXPECT_NEAR(ogs_update.n_L, reference.n_L,
+   comparisonTolerance(ogs_update.n_L, reference.n_L));
+ EXPECT_LE(ogs_update.n_L, phi + comparisonTolerance(ogs_update.n_L, phi));
+
+ auto potential_exchange_params_scalar = potential_exchange_params;
+ potential_exchange_params_scalar.local_nonlinear_solve_mode =
+   LocalNonLinearSolveMode::ScalarExchange;
+ auto const scalar_update = solveImplicitMicroWaterContent(
+   n_L_prev, dt, rho_LR, alpha_bar, mu, macro_potential,
+   {phi = phi,
+    .volumetric_strain = volumetric_strain,
+    .volumetric_strain_prev = volumetric_strain_prev},
+   potential_exchange_params_scalar);
+ ASSERT_TRUE(scalar_update.converged);
+ EXPECT_GT(scalar_update.n_L, phi);
+ EXPECT_LE(ogs_update.n_L,
+   scalar_update.n_L +
+   comparisonTolerance(ogs_update.n_L, scalar_update.n_L));
+}
+
+TEST(RichardsMechanics, DSMMicroMacroScalarMassStorageLocalSolveReferencePath)
+{
+   PotentialExchangeParameters potential_exchange_params;
+   potential_exchange_params.enabled = true;
+   potential_exchange_params.pressure_tolerance = 0.0;
+   potential_exchange_params.hamaker_constant = 6.0e-20;
+   potential_exchange_params.specific_surface = 1000.0;
+   potential_exchange_params.micro_solid_density_reference = 2650.0;
+   potential_exchange_params.micro_solid_volume_fraction_reference = 0.6;
+   potential_exchange_params.micro_liquid_density_reference = 1300.0;
+   potential_exchange_params.micro_liquid_density_a = 1.3;
+   potential_exchange_params.micro_liquid_density_b = 1.0;
+   potential_exchange_params.micro_potential_convention =
+     MicroPotentialConvention::NegativeAttractive;
+   potential_exchange_params.local_nonlinear_solve_mode =
+     LocalNonLinearSolveMode::ScalarReferenceMassStorage;
+   potential_exchange_params.initial_micro_water_content = 0.03;
+
+   double const p_L = 0.0;
+   double const n_L_prev = 0.03;
+   double const dt = 100.0;
+   double const rho_LR = 1000.0;
+   double const alpha_bar = 1.0e-9;
+   double const mu = 1.0e-3;
+   double const phi = 0.031;
+   double const volumetric_strain_prev = 0.0;
+   double const volumetric_strain = 1.0e-3;
+
+   auto const macro_potential =
+     computeYoungLaplaceMacroPotential(p_L, rho_LR, potential_exchange_params.pressure_tolerance);
+   auto const ogs_update = solveImplicitMicroWaterContent(
+     n_L_prev, dt, rho_LR, alpha_bar, mu, macro_potential,
+     {phi = phi,
+      .volumetric_strain = volumetric_strain,
+      .volumetric_strain_prev = volumetric_strain_prev},
+     potential_exchange_params);
+   ASSERT_TRUE(ogs_update.converged);
+
+   auto const reference = solveDsmMicromacroReferenceSinglePoint(
+     p_L, n_L_prev, dt, rho_LR, alpha_bar, mu, phi, potential_exchange_params,
+     volumetric_strain, volumetric_strain_prev);
+   EXPECT_NEAR(ogs_update.n_L, reference.n_L,
+     comparisonTolerance(ogs_update.n_L, reference.n_L,
+       1e-8, 1e-14));
+
+   double const analytic_dn_L_dpl = computeImplicitNDpl(
+     n_L_prev, p_L, dt, rho_LR, 0.0, alpha_bar, mu, macro_potential,
+     ogs_update.micro_potential, ogs_update.exchange,
+     {phi = phi,
+      .volumetric_strain = volumetric_strain,
+      .volumetric_strain_prev = volumetric_strain_prev},
+     potential_exchange_params);
+   double const reference_dn_L_dpl = referenceDnLDpl(
+     p_L, n_L_prev, dt, rho_LR, alpha_bar, mu, phi, potential_exchange_params,
+     volumetric_strain, volumetric_strain_prev);
+   EXPECT_NEAR(analytic_dn_L_dpl, reference_dn_L_dpl,
+     comparisonTolerance(analytic_dn_L_dpl, reference_dn_L_dpl,
+       1e-6, 1e-12));
+}
+
+TEST(RichardsMechanics, DSMMicroMacroMassStorageCoupledSolveResiduals)
+{
+   PotentialExchangeParameters potential_exchange_params;
+   potential_exchange_params.enabled = true;
+   potential_exchange_params.pressure_tolerance = 0.0;

```

```

+ potential_exchange_params.hamaker_constant = 6.0e-20;
+ potential_exchange_params.specific_surface = 1000.0;
+ potential_exchange_params.micro_solid_density_reference = 2650.0;
+ potential_exchange_params.micro_solid_volume_fraction_reference = 0.6;
+ potential_exchange_params.micro_liquid_density_reference = 1300.0;
+ potential_exchange_params.micro_liquid_density_a = 1.3;
+ potential_exchange_params.micro_liquid_density_b = 1.0;
+ potential_exchange_params.micro_potential_convention =
+   MicroPotentialConvention::NegativeAttractive;
+ potential_exchange_params.local_nonlinear_solve_mode =
+   LocalNonLinearSolveMode::ScalarReferenceMassStorage;
+ potential_exchange_params.macro_porosity_update_mode =
+   MacroPorosityUpdateMode::ReferenceAdditiveRate;
+ potential_exchange_params.micro_solid_volume_fraction_mode =
+   MicroSolidVolumeFractionMode::CurrentPorositySplit;
+ potential_exchange_params.initial_micro_water_content = 0.05;
+
+ double const p_L = 0.0;
+ double const n_L_prev = 0.05;
+ double const dt = 100.0;
+ double const rho_LR = 1000.0;
+ double const alpha_bar = 1.0e-9;
+ double const mu = 1.0e-3;
+ double const phi = 0.26;
+ double const phi_M_prev = 0.18;
+ double const phi_m_prev = 0.08;
+ double const volumetric_strain_prev = 0.0;
+ double const volumetric_strain = 1.0e-3;
+
+ PotentialExchangeLocalSolveContext const local_context{
+   .phi = phi,
+   .phi_M_prev = phi_M_prev,
+   .phi_m_prev = phi_m_prev,
+   .volumetric_strain = volumetric_strain,
+   .volumetric_strain_prev = volumetric_strain_prev,
+ };
+
+ auto const macro_potential =
+   computeYoungLaplaceMacroPotential(p_L, rho_LR, potential_exchange_params.pressure_tolerance);
+ auto const prev_micro_liquid_density =
+   computePreviousMicroLiquidDensity(n_L_prev, rho_LR, local_context,
+   potential_exchange_params);
+ double const rho_L_prev = n_L_prev * prev_micro_liquid_density.rho_LR;
+
+ auto const coupled_update = solveReferenceMassStorageCoupledState(
+   n_L_prev, rho_L_prev, prev_micro_liquid_density.rho_LR, dt, rho_LR,
+   alpha_bar, mu, macro_potential, local_context, potential_exchange_params);
+ ASSERT_TRUE(coupled_update.converged);
+ EXPECT_GT(coupled_update.n_L, 0.0);
+ EXPECT_LE(coupled_update.n_L, phi + comparisonTolerance(coupled_update.n_L,
+   phi, 1e-10, 1e-12));
+ EXPECT_GT(coupled_update.rho_LR, 0.0);
+
+ double const active_nS = computeActiveMicroSolidVolumeFraction(
+   coupled_update.n_L, local_context, potential_exchange_params);
+ auto const micro_potential = computeVanDerWaalsMicroPotential(
+   coupled_update.n_L, coupled_update.rho_LR, active_nS,
+   potential_exchange_params.micro_solid_density_reference, potential_exchange_params.hamaker_constant,
+   potential_exchange_params.specific_surface,
+   microPotentialSignFactorFromParameters(potential_exchange_params));
+ auto const exchange = computePotentialDrivenMassExchange(
+   alpha_bar * rho_LR / mu, macro_potential.mu_LR,
+   micro_potential.mu_LR);
+
+ double const rho_L = coupled_update.n_L * coupled_update.rho_LR;
+ double const volumetric_strain_rate =
+   (volumetric_strain - volumetric_strain_prev) / dt;
+ double const mass_residual =
+   rho_L - rho_L_prev - dt * exchange.rho_L_hat -
+   dt * rho_L * volumetric_strain_rate;
+ auto const density = computeReducedMicroLiquidDensity(
+   coupled_update.n_L, rho_LR, active_nS, potential_exchange_params);
+ double const density_residual = coupled_update.rho_LR - density.rho_LR;
+
+ double const residual_norm =
+   std::abs(mass_residual) / std::max(1.0, std::abs(rho_L_prev)) +
+   std::abs(density_residual) /
+   std::max(1.0, std::abs(coupled_update.rho_LR));
+ EXPECT_LE(residual_norm, 1e-8);
+}
+
+*TEST(RichardsMechanics, DSMMicroMacroOverlapTransferBaseLineHistory)
+{
+   auto const baseline_rows = loadDSMMicroMacroOverlapBaselineRows(
+     TestInfoLib::TestInfo::data_path +
+     "/RichardsMechanics/DSMMicroMacroOverlapTransferBaseLine.csv");
+   ASSERT_EQ(baseline_rows.size(), 5);
+
+   PotentialExchangeParameters potential_exchange_params;
+   potential_exchange_params.enabled = true;
+   // Match the shared dsm_micromacro/MFront nonnegative branch: p_L = 0 belongs to
+   // the saturated Young-Laplace side.
+   potential_exchange_params.pressure_tolerance = 1e-12;
+   potential_exchange_params.hamaker_constant = 6.0e-20;
+   potential_exchange_params.specific_surface = 100.0;
+   potential_exchange_params.micro_solid_density_reference = 2470.0;
+   potential_exchange_params.micro_solid_volume_fraction_reference = 0.8;
+   potential_exchange_params.micro_liquid_density_reference = 1300.0;
+   potential_exchange_params.micro_liquid_density_a = 1.3;
+   potential_exchange_params.micro_liquid_density_b = 1.0;
+   potential_exchange_params.micro_potential_convention =
+     MicroPotentialConvention::NegativeAttractive;
+   potential_exchange_params.local_nonlinear_solve_mode =
+     LocalNonLinearSolveMode::ScalarReferenceMassStorage;
+   potential_exchange_params.macro_porosity_update_mode = MacroPorosityUpdateMode::AlgebraicSplit;
+   potential_exchange_params.initial_micro_water_content = 0.1;
+   potential_exchange_params.micro_water_content_swelling_slope = 0.1;
+
+   double const dt = 1.0;
+   double const rho_LR = 1000.0;
+   double const alpha_bar = 1.0e-6;
+   double const mu = 1.0e-3;
+   double const phi = 0.2;
+   double const E = 1.0e10;
+   double const nu = 0.25;
+
+   double n_L_prev = 0.1;

```

```

+ double rho_LR_prev = potential_exchange_params.micro_liquid_density_reference;
+ double epsilon_sw = 0.0;
+
+ for (auto const& row : baseline_rows)
+ {
+     auto const local_context = PotentialExchangeLocalSolveContext{
+         .phi = phi,
+         .phi_M_prev = phi - n_L_prev,
+         .phi_m_prev = n_L_prev,
+         .volumetric_strain = 0.0,
+         .volumetric_strain_prev = 0.0,
+     };
+
+     auto const macro_potential = computeYoungLaplaceMacroPotential(
+         row.pressure, rho_LR, potential_exchange_params.pressure_tolerance);
+     double const rho_L_prev = n_L_prev * rho_LR_prev;
+     auto const coupled_update = solveReferenceMassStorageCoupledState(
+         n_L_prev, rho_L_prev, rho_LR_prev, dt, rho_LR, alpha_bar, mu,
+         macro_potential, local_context, potential_exchange_params);
+     ASSERT_TRUE(coupled_update.converged);
+
+     auto const transport = computeTransportPorosityUpdate(
+         phi, phi - n_L_prev, n_L_prev, coupled_update.n_L, 0.0, 0.0,
+         potential_exchange_params.macro_porosity_update_mode);
+     double const delta_epsilon_sw = potential_exchange_params.micro_water_content_swelling_slope *
+         (transport.phi_m - transport.phi_m_prev);
+     epsilon_sw += delta_epsilon_sw;
+     double const sigma_xx =
+         isotropicStressFromSwelling(epsilon_sw, E, nu);
+
+     EXPECT_TRUE(macro_potential.saturated_branch);
+     EXPECT_NEAR(macro_potential.mu_LR, row.mu_LR, 1e-14);
+     EXPECT_NEAR(1.0, row.saturation, 1e-14);
+     EXPECT_NEAR(coupled_update.n_L, row.n_L,
+         comparisonTolerance(coupled_update.n_L, row.n_L, 5e-3, 1e-10));
+     EXPECT_NEAR(transport.phi_m, row.phi_m,
+         comparisonTolerance(transport.phi_m, row.phi_m, 5e-3,
+             1e-10));
+     EXPECT_NEAR(transport.phi_M, row.phi_M,
+         comparisonTolerance(transport.phi_M, row.phi_M, 5e-3,
+             1e-10));
+     EXPECT_NEAR(coupled_update.rho_LR, row.rho_LR,
+         comparisonTolerance(coupled_update.rho_LR, row.rho_LR,
+             1e-3, 1e-6));
+     EXPECT_NEAR(coupled_update.micro_potential.mu_LR, row.mu_LR,
+         comparisonTolerance(coupled_update.micro_potential.mu_LR,
+             row.mu_LR, 1e-2, 1e-8));
+     EXPECT_NEAR(coupled_update.exchange.rho_L_hat, row.rho_L_hat,
+         comparisonTolerance(coupled_update.exchange.rho_L_hat,
+             row.rho_L_hat, 1e-2, 1e-8));
+     EXPECT_NEAR(epsilon_sw, row.epsilon_sw,
+         comparisonTolerance(epsilon_sw, row.epsilon_sw, 5e-3,
+             1e-10));
+     EXPECT_NEAR(sigma_xx, row.stress_xx,
+         comparisonTolerance(sigma_xx, row.stress_xx, 5e-3, 1e-4));
+     EXPECT_NEAR(transport.phi_m + transport.phi_M, phi, 1e-14);
+
+     n_L_prev = coupled_update.n_L;
+     rho_LR_prev = coupled_update.rho_LR;
+ }
+ }
+
+ *TEST(RichardsMechanics, DSMMicroMacroStrainCoupledOverlapBaselineHistory)
+ {
+     auto const overlap_rows = loadDSMMicroMacroOverlapBaselineRows(
+         TestInfoLib::TestInfo::data_path +
+         "/RichardsMechanics/DSMMicroMacroOverlapTransferBaseline.csv");
+     auto const strain_rows = loadDSMMicroMacroOverlapBaselineRows(
+         TestInfoLib::TestInfo::data_path +
+         "/RichardsMechanics/DSMMicroMacroStrainCoupledOverlapBaseline.csv");
+     ASSERT_EQ(overlap_rows.size(), 5);
+     ASSERT_EQ(strain_rows.size(), 5);
+
+     PotentialExchangeParameters potential_exchange_params;
+     potential_exchange_params.enabled = true;
+     potential_exchange_params.pressure_tolerance = 1e-12;
+     potential_exchange_params.hamaker_constant = 6.0e-20;
+     potential_exchange_params.specific_surface = 100.0;
+     potential_exchange_params.micro_solid_density_reference = 2470.0;
+     potential_exchange_params.micro_solid_volume_fraction_reference = 0.8;
+     potential_exchange_params.micro_liquid_density_reference = 1300.0;
+     potential_exchange_params.micro_liquid_density_a = 1.3;
+     potential_exchange_params.micro_liquid_density_b = 1.0;
+     potential_exchange_params.micro_potential_convention =
+         MicroPotentialConvention::NegativeAttractive;
+     potential_exchange_params.local_nonlinear_solve_mode =
+         LocalNonLinearSolveMode::ScalarReferenceMassStorage;
+     potential_exchange_params.macro_porosity_update_mode = MacroPorosityUpdateMode::AlgebraicSplit;
+     potential_exchange_params.initial_micro_water_content = 0.1;
+     potential_exchange_params.micro_water_content_swelling_slope = 0.1;
+
+     double const dt = 1.0;
+     double const rho_LR = 1000.0;
+     double const alpha_bar = 1.0e-6;
+     double const mu = 1.0e-3;
+     double const E = 1.0e10;
+     double const nu = 0.25;
+
+     auto const& overlap_anchor = overlap_rows.back();
+
+     double n_L_prev = overlap_anchor.n_L;
+     double rho_LR_prev = overlap_anchor.rho_LR;
+     double epsilon_sw = overlap_anchor.epsilon_sw;
+     double sigma_xx_prev = overlap_anchor.stress_xx;
+     double volumetric_strain_prev = 0.0;
+     double const phi = overlap_anchor.phi;
+
+     for (auto const& row : strain_rows)
+     {
+         auto const local_context = PotentialExchangeLocalSolveContext{
+             .phi = phi,
+             .phi_M_prev = phi - n_L_prev,
+             .phi_m_prev = n_L_prev,
+             .volumetric_strain = row.epsilon_v_total,
+             .volumetric_strain_prev = volumetric_strain_prev,
+         };
+     }

```

```

+     auto const macro_potential = computeYoungLaplaceMacroPotential(
+         row.pressure, rho_LR, potential_exchange_params.pressure_tolerance);
+     double const rho_l_prev = n_l_prev * rho_LR_prev;
+     auto const coupled_update = solveReferenceMassStorageCoupledState(
+         n_l_prev, rho_l_prev, rho_LR_prev, dt, rho_LR, alpha_bar, mu,
+         macro_potential, local_context, potential_exchange_params);
+     ASSERT_TRUE(coupled_update.converged);
+
+     auto const transport = computeTransportPorosityUpdate(
+         phi, phi - n_l_prev, n_l_prev, coupled_update.n_l,
+         row.epsilon_v_total, volumetric_strain_prev,
+         potential_exchange_params.macro_porosity_update_mode);
+
+     double const delta_epsilon_sw = potential_exchange_params.micro_water_content_swelling_slope *
+         (transport.phi_m - transport.phi_m_prev);
+     epsilon_sw += delta_epsilon_sw;
+     double const bulk_modulus = E / (3.0 * (1.0 - 2.0 * nu));
+     double const sigma_xx =
+         sigma_xx_prev + bulk_modulus * row.delta_epsilon_v -
+         bulk_modulus * delta_epsilon_sw;
+
+     EXPECT_TRUE(macro_potential.saturated_branch);
+     EXPECT_NEAR(macro_potential.mu_LR, row.mu_LR, 1e-14);
+     EXPECT_NEAR(1.0, row.saturation, 1e-14);
+     EXPECT_NEAR(coupled_update.n_l, row.n_l,
+         comparisonTolerance(coupled_update.n_l, row.n_l, 5e-3,
+             1e-10));
+     EXPECT_NEAR(transport.phi_m, row.phi_m,
+         comparisonTolerance(transport.phi_m, row.phi_m, 5e-3,
+             1e-10));
+     EXPECT_NEAR(transport.phi_M, row.phi_M,
+         comparisonTolerance(transport.phi_M, row.phi_M, 5e-3,
+             1e-10));
+     EXPECT_NEAR(coupled_update.rho_LR, row.rho_LR,
+         comparisonTolerance(coupled_update.rho_LR, row.rho_LR,
+             1e-3, 1e-6));
+     EXPECT_NEAR(coupled_update.micro_potential.mu_LR, row.mu_LR,
+         comparisonTolerance(coupled_update.micro_potential.mu_LR,
+             row.mu_LR, 1e-2, 1e-8));
+     EXPECT_NEAR(coupled_update.exchange.rho_l_hat, row.rho_l_hat,
+         comparisonTolerance(coupled_update.exchange.rho_l_hat,
+             row.rho_l_hat, 1e-2, 1e-8));
+     EXPECT_NEAR(delta_epsilon_sw, row.delta_epsilon_sw,
+         comparisonTolerance(delta_epsilon_sw, row.delta_epsilon_sw,
+             5e-3, 1e-12));
+     EXPECT_NEAR(epsilon_sw, row.epsilon_sw,
+         comparisonTolerance(epsilon_sw, row.epsilon_sw, 5e-3,
+             1e-10));
+     EXPECT_NEAR(sigma_xx, row.stress_xx,
+         comparisonTolerance(sigma_xx, row.stress_xx, 5e-3, 1e-4));
+     EXPECT_NEAR(transport.phi_m + transport.phi_M, row.phi,
+         comparisonTolerance(transport.phi_m + transport.phi_M,
+             row.phi, 1e-12, 1e-14));
+
+     n_l_prev = coupled_update.n_l;
+     rho_LR_prev = coupled_update.rho_LR;
+     sigma_xx_prev = sigma_xx;
+     volumetric_strain_prev = row.epsilon_v_total;
+ }
+}
+
+*TEST(RichardsMechanics, PotentialExchangeTangentRepresentativeStates)
+{
+     PotentialExchangeParameters potential_exchange_params;
+     potential_exchange_params.enabled = true;
+     potential_exchange_params.pressure_tolerance = 0.0;
+     potential_exchange_params.hamaker_constant = 1e-30;
+     potential_exchange_params.specific_surface = 1.0;
+     potential_exchange_params.micro_solid_density_reference = 2650.0;
+     potential_exchange_params.micro_solid_volume_fraction_reference = 0.6;
+     potential_exchange_params.initial_micro_water_content = 0.1;
+     potential_exchange_params.fd_jacobian_perturbation = 1e-8;
+
+     std::array<RepresentativeCoupledExchangeState, 3> const states = {{
+         {
+             .name = "pressure_proxy_unsaturated",
+             .mode = CoupledExchangeReferenceMode::pressure_proxy,
+             .p_L = -1.0e7,
+             .p_L_M = -2.0e7,
+             .pressure_tolerance = 0.0,
+             .n_l_prev = 0.1,
+             .dt = 100.0,
+             .rho_LR = 1000.0,
+             .drho_LR_dpl = 1.0e-7,
+             .alpha_bar = 1.0e-13,
+             .mu = 1.0e-3,
+             .phi = 0.4,
+         },
+         {
+             .name = "full_potential_unsaturated",
+             .mode = CoupledExchangeReferenceMode::full_potential_vdw,
+             .p_L = -1.0e7,
+             .p_L_M = 0.0,
+             .pressure_tolerance = 0.0,
+             .n_l_prev = 0.1,
+             .dt = 100.0,
+             .rho_LR = 1000.0,
+             .drho_LR_dpl = 1.0e-7,
+             .alpha_bar = 1.0e-13,
+             .mu = 1.0e-3,
+             .phi = 0.4,
+         },
+         {
+             .name = "full_potential_saturated_helper_branch",
+             .mode = CoupledExchangeReferenceMode::full_potential_vdw,
+             .p_L = -50.0,
+             .p_L_M = 0.0,
+             .pressure_tolerance = 100.0,
+             .n_l_prev = 0.1,
+             .dt = 100.0,
+             .rho_LR = 1000.0,
+             .drho_LR_dpl = 1.0e-7,
+             .alpha_bar = 1.0e-13,
+             .mu = 1.0e-3,
+             .phi = 0.4,
+         },
+     }},
+ };
+}

```



```

+ };
+ auto f_nS = [=](double const x)
+ {
+     return computeVanDerWaalsMicroPotential(n_l, rho_LR, x, rho_SR, A, Sa)
+         .mu_LR;
+ };
+ auto f_rhoSR = [=](double const x)
+ {
+     return computeVanDerWaalsMicroPotential(n_l, rho_LR, nS, x, A, Sa)
+         .mu_LR;
+ };
+
+ EXPECT_NEAR(mu.dmu_LR_dnl, centralDiff(f_nl, n_l, 1e-8), 1e-4);
+ EXPECT_NEAR(mu.dmu_LR_dns, centralDiff(f_nS, nS, 1e-8), 1e-3);
+ EXPECT_NEAR(mu.dmu_LR_drho_SR, centralDiff(f_rhoSR, rho_SR, 1e-4), 1e-6);
+}
+
+*TEST(RichardsMechanics, PotentialExchangeVanDerWaalsMicroPotentialNegativeAttractiveConvention)
+{
+    double const n_l = 0.03;
+    double const rho_LR = 1000.0;
+    double const nS = 0.6;
+    double const rho_SR = 2700.0;
+    double const A = 6.0e-20;
+    double const Sa = 1000.0;
+
+    auto const mu = computeVanDerWaalsMicroPotential(
+        n_l, rho_LR, nS, rho_SR, A, Sa, -1.0);
+
+    EXPECT_GT(mu.omega_l, 0.0);
+    EXPECT_LT(mu.mu_LR, 0.0);
+    EXPECT_GT(mu.dmu_LR_dnl, 0.0);
+    EXPECT_LT(mu.dmu_LR_dns, 0.0);
+    EXPECT_LT(mu.dmu_LR_drho_SR, 0.0);
+    EXPECT_DOUBLE_EQ(mu.dmu_LR_drho_LR, 0.0);
+
+    auto f_nl = [=](double const x)
+    {
+        return computeVanDerWaalsMicroPotential(
+            x, rho_LR, nS, rho_SR, A, Sa, -1.0)
+            .mu_LR;
+    };
+    auto f_nS = [=](double const x)
+    {
+        return computeVanDerWaalsMicroPotential(
+            n_l, rho_LR, x, rho_SR, A, Sa, -1.0)
+            .mu_LR;
+    };
+    auto f_rhoSR = [=](double const x)
+    {
+        return computeVanDerWaalsMicroPotential(
+            n_l, rho_LR, nS, x, A, Sa, -1.0)
+            .mu_LR;
+    };
+
+    EXPECT_NEAR(mu.dmu_LR_dnl, centralDiff(f_nl, n_l, 1e-8), 1e-4);
+    EXPECT_NEAR(mu.dmu_LR_dns, centralDiff(f_nS, nS, 1e-8), 1e-3);
+    EXPECT_NEAR(mu.dmu_LR_drho_SR, centralDiff(f_rhoSR, rho_SR, 1e-4), 1e-6);
+}
+
+*TEST(RichardsMechanics, PotentialExchangeSourceIoName)
+{
+    EXPECT_EQ(ioName(static_cast<MicroExchangeSourceTag*>(nullptr)),
+        "micro_exchange_source");
+}
+
+*TEST(RichardsMechanics, PotentialDrivenMassExchangeDataAndDerivatives)
+{
+    double const alpha_M = 2.5e-10;
+    double const mu_LR = -8.0;
+    double const mu_LR = 3.0;
+
+    auto const ex = computePotentialDrivenMassExchange(alpha_M, mu_LR, mu_LR);
+    EXPECT_DOUBLE_EQ(ex.rho_l_hat, alpha_M * (mu_LR - mu_LR));
+    EXPECT_DOUBLE_EQ(ex.rho_l_hat, -ex.rho_l_hat);
+    EXPECT_DOUBLE_EQ(ex.drho_l_hat_dmu_LR, alpha_M);
+    EXPECT_DOUBLE_EQ(ex.drho_l_hat_dmu_LR, -alpha_M);
+    EXPECT_DOUBLE_EQ(ex.drho_l_hat_dalpha_M, mu_LR - mu_LR);
+}
+
+*TEST(RichardsMechanics, PotentialExchangeDirectExchangeAlgebra)
+{
+    double const alpha_bar = 2.5e-10;
+    double const mu_LR = 0.0;
+    double const mu_LR = -3.0;
+
+    auto const ex = computePotentialDrivenMassExchange(alpha_bar, mu_LR,
+        mu_LR);
+    EXPECT_DOUBLE_EQ(ex.rho_l_hat, alpha_bar * (mu_LR - mu_LR));
+    EXPECT_DOUBLE_EQ(ex.rho_l_hat, -ex.rho_l_hat);
+    EXPECT_GT(ex.rho_l_hat, 0.0);
+}
+
+diff --git a/scripts/cmake/test/HistoryExtraction.cmake b/scripts/cmake/test/HistoryExtraction.cmake
+new file mode 100644
+index 0000000000..4417c7b8db
+--- /dev/null
++++ b/scripts/cmake/test/HistoryExtraction.cmake
+@@ -0,0 +1,51 @@
+ +if(NOT DEFINED PYTHON_EXECUTABLE)
+ +    message(FATAL_ERROR "PYTHON_EXECUTABLE is required.")
+ +endif()
+ +if(NOT DEFINED PROJECT_SOURCE_DIR)
+ +    message(FATAL_ERROR "PROJECT_SOURCE_DIR is required.")
+ +endif()
+ +if(NOT DEFINED BINARY_DIR)
+ +    message(FATAL_ERROR "BINARY_DIR is required.")
+ +endif()
+ +if(NOT DEFINED SOURCE_CSV)
+ +    message(FATAL_ERROR "SOURCE_CSV is required.")
+ +endif()
+ +if(NOT DEFINED OUTPUT_CSV)
+ +    message(FATAL_ERROR "OUTPUT_CSV is required.")
+ +endif()
+ +
+ +file(MAKE_DIRECTORY "${BINARY_DIR}")
+ +
+ +execute_process(

```



```

+ COMMAND
+     "${PYTHON_EXECUTABLE}"
+     "${PROJECT_SOURCE_DIR}/scripts/extract_observables.py"
+     --glob "${BINARY_DIR}/beacon_1a01_notebook_roles_t_*.vtu"
+     --output "${OUTPUT_CSV}"
+     --arrays
+     pressure
+     micro_pressure
+     micro_saturation
+     micro_water_content
+     micro_porosity
+     micro_exchange_source
+     swelling_stress
+     sigma
+     RESULT_VARIABLE extract_result
+)
+
+if(NOT extract_result EQUAL 0)
+  message(
+    FATAL_ERROR
+    "Notebook-role history extraction failed with exit code ${extract_result}."
+  )
+endif()
+
+if(NOT EXISTS "${OUTPUT_CSV}")
+  message(FATAL_ERROR "Notebook-role history CSV was not written.")
+endif()
+
+file(SIZE "${OUTPUT_CSV}" output_size)
+if(output_size EQUAL 0)
+  message(FATAL_ERROR "Notebook-role history CSV is empty.")
+endif()
diff --git a/scripts/extract_observables.py b/scripts/extract_observables.py
new file mode 100644
index 0000000000..47c7dbb45c
--- /dev/null
+++ b/scripts/extract_observables.py
@@ -0,0 +1,168 @@
#!/usr/bin/env python3
"""Extract simple time-history observables from a sequence of VTU files.

+
+The script is intentionally small and dependency-light:
+- it reads OGS VTK XML unstructured-grid outputs via `vtk`,
+- it searches point data first, then cell data,
+- it writes one CSV row per file,
+- and it stores per-component means for the requested arrays.
+
+This is a benchmark-observable extraction layer, not a calibration tool.
+"""
+
+from __future__ import annotations
+
+import argparse
+import csv
+import glob
+import re
+
+from pathlib import Path
+from typing import Iterable
+
+import numpy as np
+import vtk
+from vtk.util.numpy_support import vtk_to_numpy
+
+TIME_RE = re.compile(r"_t_([0-9.+\-eE]+\.\.vtu$")
+
+def parse_time_from_name(path: Path) -> float:
+    match = TIME_RE.search(path.name)
+    if not match:
+        raise ValueError(
+            f"Cannot parse time stamp from file name '{path.name}'. "
+            f"Expected suffix like '_t_1000.000000.vtu'."
+        )
+    return float(match.group(1))
+
+def load_unstructured_grid(path: Path) -> vtk.vtkUnstructuredGrid:
+    reader = vtk.vtkXMLUnstructuredGridReader()
+    reader.SetFileName(str(path))
+    reader.Update()
+    data = reader.GetOutput()
+    if data is None:
+        raise RuntimeError(f"Failed to read VTK file '{path}'.")
+    return data
+
+def find_array(dataset: vtk.vtkUnstructuredGrid, name: str, association: str):
+    if association in ("auto", "point"):
+        point_data = dataset.GetPointData()
+        if point_data is not None:
+            array = point_data.GetArray(name)
+            if array is not None:
+                return "point", vtk_to_numpy(array)
+
+    if association in ("auto", "cell"):
+        cell_data = dataset.GetCellData()
+        if cell_data is not None:
+            array = cell_data.GetArray(name)
+            if array is not None:
+                return "cell", vtk_to_numpy(array)
+
+    raise KeyError(f"Array '{name}' was not found in point or cell data.")
+
+def as_2d(values: np.ndarray) -> np.ndarray:
+    if values.ndim == 1:
+        return values.reshape(-1, 1)
+    return values
+
+def component_column_names(name: str, n_components: int) -> list[str]:
+    if n_components == 1:
+        return [f"{name}_mean"]
+    return [f"{name}_c{i}_mean" for i in range(n_components)]
+

```

```

+def iter_input_files(patterns: Iterable[str]) -> list[Path]:
+    files: list[Path] = []
+    for pattern in patterns:
+        matches = sorted(glob.glob(pattern))
+        files.extend(Path(match) for match in matches)
+        unique_files = sorted(set(files), key=parse_time_from_name)
+    return unique_files
+
+
+def main() -> int:
+    parser = argparse.ArgumentParser(
+        description="Extract per-file mean observables from VTU outputs."
+    )
+    parser.add_argument(
+        "--glob",
+        dest="patterns",
+        action="append",
+        required=True,
+        help="Glob pattern for input VTU files. Can be passed multiple times.",
+    )
+    parser.add_argument(
+        "--output",
+        required=True,
+        help="CSV file to write.",
+    )
+    parser.add_argument(
+        "--arrays",
+        nargs="+",
+        required=True,
+        help="VTK point/cell data array names to extract.",
+    )
+    parser.add_argument(
+        "--association",
+        choices=("auto", "point", "cell"),
+        default="auto",
+        help="Where to look for the arrays. Default: auto.",
+    )
+    args = parser.parse_args()
+
+    input_files = iter_input_files(args.patterns)
+    if not input_files:
+        raise SystemExit("No input VTU files matched the provided glob(s).")
+
+    rows = []
+    header = ["time_s"]
+    component_counts: dict[str, int] = {}
+
+    for path in input_files:
+        dataset = load_unstructured_grid(path)
+        time_s = parse_time_from_name(path)
+
+        row = {"time_s": time_s}
+        for name in args.arrays:
+            source, values = find_array(dataset, name, args.association)
+            values_2d = as_2d(np.asarray(values))
+            if name not in component_counts:
+                component_counts[name] = values_2d.shape[1]
+            header.extend(component_column_names(name, values_2d.shape[1]))
+            elif component_counts[name] != values_2d.shape[1]:
+                raise RuntimeError(
+                    f"Array '{name}' has inconsistent component count: "
+                    f"expected {component_counts[name]}, got {values_2d.shape[1]} in '{path.name}'."
+                )
+
+            means = values_2d.mean(axis=0)
+            for i, mean in enumerate(means):
+                column = f"{name}_mean" if values_2d.shape[1] == 1 else f"{name}_c{i}_mean"
+                row[column] = float(mean)
+
+            # Keep the association check explicit even though the CSV does not
+            # store it: the script is meant to fail early if the field moves.
+            _ = source
+
+        rows.append(row)
+
+    with open(args.output, "w", newline="", encoding="utf-8") as f:
+        writer = csv.writer(f)
+        writer.writerow(header)
+        for row in rows:
+            writer.writerow(
+                [row["time_s"]]
+                + [row[column] for column in header[1:]]
+            )
+
+    return 0
+
+
+if __name__ == "__main__":
+    raise SystemExit(main())
diff --git a/scripts/history_trend_check.py b/scripts/history_trend_check.py
new file mode 100644
index 0000000000..fb0aca77e0
--- /dev/null
+++ b/scripts/history_trend_check.py
@@ -0,0 +1,120 @@
+#!/usr/bin/env python3
+"""Check trend-level agreement between two notebook-role history CSV files.
+
+The notebook-role history extraction snapshot is meant to be a calibration
+baseline, not a brittle exact-value regression. This helper therefore verifies
+the invariants that matter for the current comparison step:
+- identical time stamps,
+- monotone growth of the micro water content / porosity proxy,
+- positive micro exchange source throughout,
+- growing stress magnitude,
+- and matching trend directions between the committed snapshot and the current
+  + extracted history.
+"""
+
+from __future__ import annotations
+
+import argparse
+import csv
+import math
+from pathlib import Path
+

```

```

+def load_csv(path: Path) -> List[dict[str, str]]:
+    with path.open(newLine="", encoding="utf-8") as f:
+        return list(csv.DictReader(f))
+
+
+def as_float_rows(rows: List[dict[str, str]], key: str) -> List[float]:
+    return [float(row[key]) for row in rows]
+
+
+def is_monotone_non_decreasing(values: List[float], tol: float = 0.0) -> bool:
+    return all(b >= a - tol for a, b in zip(values, values[1:]))
+
+
+def is_monotone_non_increasing(values: List[float], tol: float = 0.0) -> bool:
+    return all(b <= a + tol for a, b in zip(values, values[1:]))
+
+
+def trend_label(values: List[float], tol: float = 0.0) -> str:
+    delta = values[-1] - values[0]
+    if abs(delta) <= tol:
+        return "flat"
+    return "inc" if delta > 0.0 else "dec"
+
+
+def compare_times(reference: List[dict[str, str]], actual: List[dict[str, str]]) -> None:
+    if len(reference) != len(actual):
+        raise SystemExit(
+            f"Row count mismatch: reference has {len(reference)} rows, actual has "
+            f"{len(actual)} rows."
+        )
+
+    ref_times = as_float_rows(reference, "time_s")
+    act_times = as_float_rows(actual, "time_s")
+    if len(ref_times) < 2:
+        raise SystemExit("Need at least two time points for trend comparison.")
+
+    for idx, (ref_t, act_t) in enumerate(zip(ref_times, act_times)):
+        if not math.isclose(ref_t, act_t, rel_tol=0.0, abs_tol=1e-12):
+            raise SystemExit(
+                f"Time stamp mismatch at row {idx}: reference={ref_t}, actual={act_t}."
+            )
+
+
+def compare_trends(reference: List[dict[str, str]], actual: List[dict[str, str]]) -> None:
+    trend_columns = [
+        ("pressure_mean", "inc"),
+        ("micro_pressure_mean", "inc"),
+        ("micro_saturation_mean", "dec"),
+        ("micro_water_content_mean", "dec"),
+        ("micro_porosity_mean", "dec"),
+        ("micro_exchange_source_mean", "inc"),
+        ("swelling_stress_c0_mean", "inc"),
+        ("sigma_c0_mean", "inc"),
+    ]
+
+    for column, expected_reference_trend in trend_columns:
+        ref_values = as_float_rows(reference, column)
+        act_values = as_float_rows(actual, column)
+        ref_trend = trend_label(ref_values, tol=1e-15)
+        act_trend = trend_label(act_values, tol=1e-15)
+        print(
+            f"{column}: ref {ref_values[0]:.16g} -> {ref_values[-1]:.16g} "
+            f"({ref_trend}), act {act_values[0]:.16g} -> {act_values[-1]:.16g} "
+            f"({act_trend})"
+        )
+        if ref_trend != expected_reference_trend:
+            raise SystemExit(
+                f"Reference CSV trend for '{column}' changed: expected "
+                f"{expected_reference_trend}, got {ref_trend}."
+            )
+        if act_trend != ref_trend:
+            raise SystemExit(
+                f"Trend mismatch for '{column}': reference {ref_trend}, actual {act_trend}."
+            )
+
+    actual_sigma = as_float_rows(actual, "sigma_c0_mean")
+
+    if not (abs(actual_sigma[-1]) > abs(actual_sigma[0])):
+        raise SystemExit("Actual sigma_c0_mean magnitude did not grow.")
+
+
+def main() -> int:
+    parser = argparse.ArgumentParser(
+        description="Compare notebook-role history CSVs on trend rather than exact values."
+    )
+    parser.add_argument("--reference", required=True, help="Committed reference CSV.")
+    parser.add_argument("--actual", required=True, help="Extracted history CSV.")
+    args = parser.parse_args()
+
+    reference = load_csv(Path(args.reference))
+    actual = load_csv(Path(args.actual))
+    compare_times(reference, actual)
+    compare_trends(reference, actual)
+    return 0
+
+
+if __name__ == "__main__":
+    raise SystemExit(main())

```